

Методичка Прога 2-ой Сем

Оглавление:

1. Основы создания классов. Инкапсуляция

- Отличие C++ от C: включение функций в структуры (классы) как добавление поведения к характеристикам.
- Понятие класса, объекта (экземпляра класса), полей и методов.
- Инкапсуляция: объединение данных и методов, управление доступом.
- Спецификаторы доступа: private, protected, public.
- Различие между struct и class (доступ по умолчанию).
- Рекомендации по проектированию класса: поля – закрыты, интерфейс – открыт, служебные функции – закрыты.
- Указатель this: неявный параметр функций-членов, разрешение коллизий имён.

2. Конструкторы и деструкторы

- Конструкторы: назначение, имя, отсутствие возвращаемого значения.
- Конструктор без параметров (конструктор по умолчанию).
- Конструктор с параметрами.
- Конструктор со списком инициализаторов (:

5. Наследование и полиморфизм field(value), ...). Преимущества (эффективность, инициализация констант и ссылок).

- Конструктор с параметрами по умолчанию.
- Неявный конструктор по умолчанию: условия генерации, опасность неочевидных ошибок (исчезновение после объявления любого другого конструктора).
- Конструктор копирования: понятие, когда вызывается (явное создание копии, передача по значению, возврат по значению).
- Проблема автоматического конструктора копии (поверхностное копирование vs глубокое копирование) для классов с динамической памятью.
- Деструктор: имя (~), отсутствие аргументов, назначение (освобождение ресурсов).
- Неявный деструктор.

•Порядок вызова деструкторов (при уничтожении объекта).

3. Перегрузка операций

•Понятие перегрузки операций как способа придания операциям нового смысла для пользовательских типов.

•Синтаксис: operator@.

•Два способа перегрузки: как функция-член и как внешняя функция.

•Рекомендации по выбору способа (изменяет левый аргумент -> член; симметрия операндов или cout << obj -> внешняя).

•Перегрузка бинарных операций (+, +=, ==, !=).•Перегрузка унарных операций (префиксный и постфиксный ++, --): фиктивный параметр int для постфиксной формы.

•Перегрузка операций ввода/вывода (<<, >>) как внешних дружественных функций, возврат ссылки на поток.

•Перегрузка оператора индексации []: возврат по ссылке, константная и неконстантная версии.

•Перегрузка оператора приведения типа (operator double(), operator char*()).

•Ограничения перегрузки операций (нельзя перегрузить ::, .., sizeof, ?; и т.д.).

4. Управление ресурсами. Правило трёх и пяти

•Правило трёх: Если классу требуется явный деструктор, конструктор копирования или оператор присваивания копированием, то, скорее всего, нужны все три.

•Оператор копирующего присваивания (operator=): проверка на самоприсваивание (this != &other), корректное освобождение старого ресурса и выделение нового.

•Исключения в операторе присваивания: базовая гарантия, строгая гарантия (использование локальной копии и swap).

•Идиома copy-and-swap.

•Семантика перемещения (C++11): rvalue-ссылки (&&), конструктор перемещения, оператор присваивания перемещением.

•Правило пяти: деструктор, копирующий конструктор, копирующее присваивание, перемещающий конструктор, перемещающее присваивание.

5. Наследование и полиморфизм

•Наследование: отношение "is-a". Базовый и производный классы.

- Виды наследования (public, protected, private). Таблица доступности членов.
- Порядок вызова конструкторов и деструкторов при наследовании.
- Переопределение методов. Отличие переопределения от перегрузки.
- Совместимость типов: указатель/ссылка на базовый класс могут указывать на объект производного.
- **Полиморфизм:** статическое (раннее) и динамическое (позднее) связывание.
- **Виртуальные функции (virtual):** механизм позднего связывания.
- **Чисто виртуальные функции (= 0).** Абстрактные классы (нельзя создать объект).
- Таблица виртуальных методов (VMT), указатель vptr.
- Виртуальный деструктор (правило: если есть хотя одна виртуальная функция, деструктор должен быть виртуальным).
- **Преобразование типов в иерархии:**

upcast (от наследника к базовому) – неявное.

downcast (от базового к наследнику) – явное, static_cast, dynamic_cast. •dynamic_cast: безопасное приведение для полиморфных типов, возвращает nullptr для указателей или генерирует std::bad_cast для ссылок.

- **RTTI (Runtime Type Information):** typeid, std::type_info.
- Множественное наследование, проблема ромба (diamond problem), виртуальное наследование.
- Спецификаторы override и final.
- Закрытое наследование (private) как отношение "as-a" (реализовано посредством).

6. Дружественность (friend)

- Дружественные функции (доступ к закрытым членам класса).
- Дружественные классы.
- Сравнение дружественных функций с функциями доступа (геттерами/сеттерами).

7. Статические члены класса

- Статические поля: одна копия для всех объектов, инициализация вне класса.
- Статические методы: не имеют указателя this, могут обращаться только к статическим членам

- [Подсчёт количества объектов с помощью статического поля.](#)

8. Шаблоны (Templates)

- [Шаблоны функций и шаблоны классов.](#)
- [Параметры шаблонов: типы \(typename/class\), нетиповые параметры \(константы\), параметры-шаблоны.](#)
- [Специализация шаблонов \(полная и частичная\).](#)
- [Явная специализация.](#)
- [Особенности компиляции шаблонов \(код в заголовочных файлах\).](#)
- [Инстанцирование шаблона. Ключевое слово typename.](#)
- [Конструкторы преобразования и ключевое слово explicit.](#)

9. Обработка исключений (Exception Handling)

- [Конструкция try, catch, throw.](#)
- [Порядок обработчиков catch \(от частного к общему, catch\(...\)\).](#)
- [Исключения в конструкторах и деструкторах.](#)
- [Гарантии безопасности исключений \(базовая, строгая, отсутствие\).](#)

10. Умные указатели (Smart Pointers) – C++11 и новее

- [Проблемы с сырыми указателями: утечки памяти, висячие ссылки.](#)
- [std::unique_ptr: эксклюзивное владение, запрет копирования, перемещение через std::move.](#)
- [std::shared_ptr: подсчёт ссылок, совместное владение.](#)
- [std::weak_ptr: предотвращение циклических ссылок \(shared_ptr\), метод lock\(\).](#)
- [Фабричные функции: std::make_shared, std::make_unique.](#)
- [Проблема std::auto_ptr \(deprecated\).](#)

11. Отношения между классами

- [Наследование \("is-a"\).](#)
- [Включение/композиция \("has-a"\).](#)
- [Закрытое наследование \("as-a" или "реализовано через"\).](#)

12. STL (Standard Template Library)

• Основные компоненты STL: контейнеры, итераторы, алгоритмы, адаптеры, функторы, алло-
каторы.

• **Контейнеры:**

- Последовательные: vector, list, deque, array.
- Ассоциативные: set, multiset, map, multimap.
- Неупорядоченные: unordered_set, unordered_map.
- Адаптеры контейнеров: stack, queue, priority_queue.

• **Итераторы:** понятие, категории (ввода, вывода, однонаправленные, двунаправленные, произвольного доступа).

• Функции begin(), end(), rbegin(), rend().

• Итераторы вставки (back_inserter, front_inserter, inserter).

• Потоковые итераторы (istream_iterator, ostream_iterator).

• **Алгоритмы STL (основные):**

- Немодифицирующие: find, find_if, count, count_if, for_each, search, equal.
- Модифицирующие: copy, copy_if, transform, replace, remove, unique, reverse.
- Сортировка и операции с множествами: sort, set_intersection, set_union.

• **Функторы (функциональные объекты) и лямбда-выражения (C++11):**

• синтаксис []() {}, списки захвата ([=], [&], [a], [&b]), mutable.

• Класс std::function.

13. Продвинутые темы C++

• constexpr: вычисление на этапе компиляции, отличие от const.

• lvalue и rvalue: понятие, ссылки на rvalue (&&).

• Ковариантные возвращаемые типы виртуальных функций.

• Управление генерацией специальных функций: = default, = delete.

• Синглтон: понятие, запрет копирующего конструктора и оператора присваивания.

14. Практические задачи (примеры из лекций)

• Реализация класса Date.

- [Реализация класса Circle.](#)
- [Реализация класса Array \(динамический массив\).](#)
- [Реализация класса String.](#)
- [Реализация классов Person – Student \(наследование\).](#)
- [Реализация классов геометрических фигур с полиморфизмом \(shape, circle, rectangle\).](#)
- [Реализация класса-счётчика объектов \(Counter\).](#)
- [Реализация класса дроби \(frac\).](#)
- [Реализация бинарного дерева поиска \(рекурсивные структуры\).](#)
- [Реализация двусвязного списка и итератора к нему.](#)
- [Реализация шаблонного класса Stack.](#)
- [Реализация класса Matrix на основе vector<vector>.](#)
- [Использование map для частотного словаря.](#)
- [Использование set и set intersection для поиска студентов-задолжников.](#)

1. Основы создания классов. Инкапсуляция

Отличие C++ от C:

При внесении функций в структуры к характеристикам (**как у структур в C**) добавляется поведение. Так возникает концепция класса. В результате чего его поля (**характеристики**) и функции (**поведение**) рассматриваются как единое целое. Такое объединение получило название инкапсуляция (**encapsulation**).

Спецификаторы доступа: private, protected, public:

private (закрытый) – доступен только для членов класса

protected (защищенный) – доступен для членов класса и их наследников

public (открытый) – доступен для всех

Рекомендации по проектированию класса:

В основном рекомендуется делать **поля** – закрытыми, **интерфейсные функции** – открытыми, **служебные функции** – закрытыми.

Различие между struct и class (доступ по умолчанию):

Различие в доступе по умолчанию

описание	class <имя класса> { <поля и функции> };	struct <имя структуры (класса)>{ <поля и функции> };
спецификатор доступа по умолчанию	private	public

Следующие два объявления эквивалентны

```
struct A{  
    int f();  
    void g();  
private:  
    int i, j, k;  
};
```

```
int A::f() {  
    return i+j+k;  
}
```

```
void A::g() {  
    i=j=k=0;  
}
```

```
int main(){  
    A a;  
    B b;  
    a.f(); a.g();  
    b.f(); b.g();  
}
```

```
class B{  
    int i, j, k;  
public:  
    int f();  
    void g();  
};
```

```
int B::f() {  
    return i+j+k;  
}
```

```
void B::g() {  
    i=j=k=0;  
}
```

Понятие класса, объекта (экземпляра класса), полей и методов:

Класс определяет новый тип данных.

Переменная данного типа называется **объектом**.

Объект также называют **экземпляром класса**.

Поля класса — это его характеристики.

Методы класса - это функция или процедура, принадлежащая классу, которая определяет **поведение** объекта, которая также может изменять поля класса.

Инкапсуляция — объединение функций(поведения класса) и полей(характеристик класса), с возможностью регламентировать доступ к членам класса.

Указатель **this**: неявный параметр функций-членов, разрешение коллизий имён:

```
Date(int d, int m, int y) {  
    this->d = d;  
    this->m = m;  
    this->y = y;  
}
```

Ключевым словом **this** обозначается указатель на объект в функциях-членах класса.

Указатель на объект передаётся как **неявный параметр** во все функции, которые могут вызываться только для экземпляров классов (в том числе, конструкторы и деструкторы).

```
Date(int dd, int mm, int yy) {  
    d = dd;  
    this->m = mm;  
    this->y = yy;  
}
```

Если коллизии имён не возникают, то при обращении к членам класса его обычно опускают.

2. Конструкторы и деструкторы

Конструкторы: назначение, имя, отсутствие возвращаемого значения:

Чтобы обеспечить должную **инициализацию** каждого объекта, разработчик класса должен включить в него **специальную функцию**, которая называется **конструктором**

Имя конструктора должно совпадать с именем класса.

Конструктор предназначен для **инициализации** полей начальными значениями

```
Date(); //конструктор без параметров
```

```
Date(int d, int m, int y); //конструктор с параметрами
```

```
Date(14, 02, 2026) //пример вызова конструктора и создания объекта(т.е конструктор вызывается автоматически)
```

Конструктор с параметрами:

// Конструктор класса с параметрами

```
Date(int d, int m, int y) {  
    this->d = d;  
    this->m = m;  
    this->y = y;  
}
```

Конструктор со списком инициализаторов:

```
Date(int d, int m, int y) { //пример конструктора с параметрами  
    this->d = d;  
    this->m = m;  
    this->y = y;  
}(1)
```

В C++ для инициализации полей в конструкторе используются **списки инициализаторов**

```
class Date {  
    int d, m, y;  
    public:  
    Date(int d, int m, int y) : d(d), m(m), y(y) {}  
};
```

d(d), m(m), y(y) — **список инициализаторов**

В чем эффективность:

Если **список инициализаторов** (как в примере 1) не используется, то в случае **полей классовых типов** всё равно вначале будут вызваны **конструкторы без параметров** для таких полей, а затем в теле конструктора класса **значения полей будут перезаписаны** на основе аргументов конструктора. То есть выполняется двойная работа.

Поэтому лучше использовать список инициализаторов, т.к поля инициализируются **напрямую** в момент их создания. Вот так и живем)

Доп.тема — определение функций-членов класса:

Все функции, размещённые (определённые) внутри класса, автоматически помечаются модификатором **inline**. Определение внешних функций класса обычно осуществляется в *.cpp файлах

```

/* date.cpp */

#include "date.h"

    // Определение некоторой функции вне интерфейса класса

void Date::add_days(int n)

{

    ...//тело фцнкции

}

```

Конструктор без параметров (конструктор по умолчанию):

Конструктор без параметров называется конструктором **по умолчанию**. В случае если **явно не описан ни один конструктор**, компилятор **автоматически (неявно) сгенерирует конструктор по умолчанию**. Поведение неявно созданного конструктора будет таким же, как если бы он был объявлен явно без списка параметров и с пустым телом. (для динамической памяти необходимо самому прописывать конструктор по умолчанию, чтобы не получить ошибки при работе программы)

//Примеры

//Первая реализация

```

circle::circle(){

    x=10;

    y=10;

    r=10;

}

circle::circle(double x1, double y1, double r1){

    x=x1;

    y=y1;

    r=r1;

}

```

//Вторая реализация

```

circle::circle(double x1=10, double y1=10, double r1=10){

    x=x1;

    y=y1;

    r=r1;

}

```

Неявный конструктор по умолчанию: условия генерации, опасность неочевидных ошибок (исчезновение после объявления любого другого конструктора):

Аналогичная ошибка возникнет и при попытке объявить массив объектов класса Student:

```
main() {  
    Student students[3];  
}
```

Это происходит потому, что при создании массива объектов компилятор вставляет вызовы конструктора без параметров для каждого элемента массива.

Чтобы такой массив можно было создать, нужно либо добавить в класс конструктор без параметров, либо проводить инициализацию явно:

```
main() {  
    Student students[] = {Student("Vasya"), Student("Petya"), Student("Sasha")};  
}
```

После этого `main()` перестанет компилироваться, потому что исчезнет конструктор по умолчанию. Если в дальнейшем предполагается размещать объекты класса в массиве или в любом другом контейнере, необходимо **объявлять конструктор без параметров**.

Конструктор копирования: понятие, когда вызывается (явное создание копии, передача по значению, возврат по значению) + Проблема автоматического конструктора копии (поверхностное копирование vs глубокое копирование) для классов с динамической памятью:

Для чего нужен копирующий конструктор - определить, как создавать новый объект как копию существующего. Нужен для создания независимой копии ресурса (глубокое копирование), а не просто копирования указателя (поверхностное копирование).

Копирующие конструкторы настолько важны, что **компилятор автоматически генерирует копирующий конструктор**, если этого не делает программист.

Такой автоматически создаваемый конструктор копии является **конструктором с побитовым копированием** переменных-членов класса. Такое **копирование** называется **поверхностным**.

Три случая, когда вызывается конструктор копий:

- Явное создание нового объекта-копии:

```
myvector myv3(myv1);  
  
myvector myv2 = myv1;
```

- Вызов функции с передачей параметра по значению:

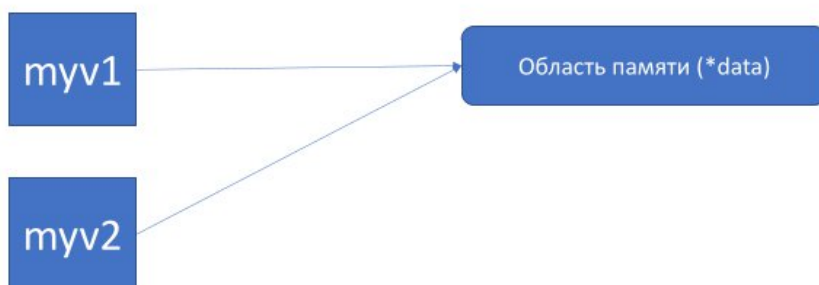
```
void f(myvector st) { /* ... */ }
```

- Возврат объекта из функции по значению:

```
myvector g() { /* ... return ...; */ }
```

Что касается проблем:

Что происходит в случае использования конструктора копии, создаваемого автоматически?



Вместо копии массива будет создана копия ссылки на него, т. е. ссылки двух разных объектов будут указывать на одно и то же место в памяти.

В каких случаях необходимо создавать конструктор копии?

Для классов, размещающих данные в динамической памяти, необходим конструктор копии.

Для классов, использующих размещение данных в динамической памяти, отсутствие конструктора копии является грубой ошибкой.

Необязательная часть:

//Пример

```
class myvector {
```

```
...
```

```
public:
```

```
};
```

здесь присутствуют случаи 1 и 3 вызова конструктора копий,

а значит должны создаваться две копии, но ...

```
myvector g() {
```

```
    return myvector();
```

```
}
```

```
int main() {  
    myvector myv1 = g();  
}
```

То есть, здесь должно произойти:

1. Явное создание нового объекта-копии

2. Возврат объекта из функции по значению

,но ... на самом деле, запуск данного примера покажет, что во время выполнения программы **не будет вызвано ни одного конструктора копий**.

Это результат работы оптимизирующего компилятора.

Заметим, что такая оптимизация может существенно повлиять на поведение программы в случае, когда конструктор копии содержит **побочные эффекты** (например: вывод на консоль).

Однако она производится подавляющим большинством современных компиляторов по умолчанию, потому что явно оговорена в стандарте языка.

Эта оптимизация носит название **Return Value Optimization (RVO)**.

Если специальными ключами компиляции запретить **RVO**, то будут вызвано ровно два конструктора копии, как и ожидалось.

В реальных программах просто стараются **не помещать дополнительный код** в конструктор копии.

Деструктор: имя (~), отсутствие аргументов, назначение (освобождение ресурсов) + неявный деструктор:

Синтаксис **деструктора** в целом схож с синтаксисом конструктора — **имя** функции тоже определяется **именем класса**. Необходим для того, чтобы освободить память

Но чтобы деструктор отличался от конструктора, его имя начинается с **префикса ~ (тильда)**.

Кроме того, **деструктор всегда один и у него нет аргументов**.

`~Date(){} - деструктор`

Неявный деструктор:

В случае отсутствия явного задания деструктора, компилятор неявно создаёт деструктор с пустым телом.

//Пример

```
class Student {  
    string name;  
public:
```

```
Student(string const & name):name(name) {}
~Student() {}
}
```

Деструктор автоматически вызывается компилятором при выходе объекта из области видимости или при уничтожении объекта, размещённого в динамической памяти.

При этом очищается память, занимаемая объектом.

Если перед уничтожением объекта **необходимо освободить** используемые **ресурсы** (например, динамическую память для размещения полей объекта, открытые файлы и т.д.), то **необходимо явно определить деструктор**, выполняющий эти действия.

Конструкторы и деструкторы обладают одной уникальной особенностью: они **не имеют возвращаемого значения**. В этом они принципиально отличаются от функций, возвращающих пустое значение (значение типа void)

Интересное замечание:

Этот факт справедлив и для очень больших объектов, таких как string или vector. **Чтобы не происходило полного копирования объектов, нужно передавать их по ссылке:**

```
void f(vector<int> &v);
void f(const string &s); // Изменить строку не получится
friend bool tofit(circle &a, rectangle &b);
А лучше так
friend bool tofit(circle &a, const rectangle &b);
```

Порядок вызова деструкторов (при уничтожении объекта) без учета наследования :

Если в классе выделяется память, то её необходимо освободить средствами этого же класса. Это должен делать деструктор.

```
Array::~Array( ){
    delete[] A;
}
```

Деструктор вызывается компилятором автоматически для каждого созданного объекта.

//Брал остальное из Gemini

1. Локальные (автоматические) объекты в пределах одного блока {}

Если в пределах одной функции или блока кода создано несколько локальных объектов, они распределяются в стековой памяти. При выходе из блока (по достижении закрывающей фи-

гурной скобки } или при вызове return) деструкторы вызываются в **строго обратном порядке их объявления**.

2. Порядок внутри сложного класса (Композиция / Агрегация)

Если класс содержит в себе другие объекты в качестве полей (членов класса), то при уничтожении экземпляра этого класса порядок вызовов следующий:

1. Сначала выполняется тело **деструктора самого внешнего класса**.
2. Затем автоматически вызываются **деструкторы всех встроенных объектов-полей**. Они уничтожаются в **обратном порядке их объявления** внутри класса (снизу вверх).

3. Статические и глобальные объекты

- **Глобальные объекты** и статические переменные в пределах одной единицы трансляции (файла) уничтожаются в порядке, обратном их созданию, уже **после** завершения работы функции main.
- **Локальные статические объекты** (static Trace s_obj; внутри функции) инициализируются только при первом заходе в функцию, а уничтожаются также после завершения main в порядке, обратном их инициализации.

Итоговая шпаргалка для экзамена:

Порядок конструирования всегда прямой, а порядок уничтожения — зеркально обратный:

Базовый класс → Поля класса (сверху вниз) → Сам класс (Конструктор)

⇓ Зеркальный разворот для деструкторов ⇓

Сам класс (Деструктор) → Поля класса (снизу вверх) → Базовый класс

3. Перегрузка операций

Понятие перегрузки операций как способа придания операциям нового смысла для пользовательских типов +синтаксис operator@:

В C++ для пользовательских типов данных возможна собственная реализация стандартных операций, которая называется **перегрузкой операций**.

Перегруженная операция — это **функция**, имя которой начинается со слова **operator**, за которым следует **символ операции**.

(Как я понял эти определения равносильны)

Перегрузка операции это определение операции с тем же именем, но с другими типами параметров.

@ - обозначение бинарной операции в рамках курса

Использовать перегруженную операцию можно двумя способами:

✓ традиционное использование операции

```
cout<<(a==b)<<endl;
```

✓ вызов функции

```
cout<<a.operator==(b)<<endl;
```

Два способа перегрузки: как функция-член и как внешняя функция:

Существует 2 способа перегрузки операции

1.Как **функция-член**

```
a.operator@(b)
```

2.Как **внешняя функция**

```
operator@(a, b)
```

Реализовывать необходимо **один из двух**.

При попытке реализовать оба варианта компилятор выдаст ошибку.

Рекомендации по выбору способа (изменяет левый аргумент -> член; симметрия операндов или cout << obj -> внешняя):

Функция-член:

- если операция изменяет левый аргумент (+=, -=, =)
- или оператор, который должен быть виртуальным (operator[])

Внешняя функция:

- если нужно обеспечить симметрию операндов (5 + BigInteger)
- или если левый операнд не является объектом нашего класса (cout << Date)

Количество аргументов в списке перегруженной операции зависит от двух факторов:

- категории операции – **унарная или бинарная**;
- способа определения операции

— **в виде глобальной (внешней) функции**

(один аргумент для унарной, два – для бинарных операций)

— **или функции класса**

(для унарных операций аргументы отсутствуют, для бинарных операций – один аргумент).

При определении функции-члена класса объект, для которого она будет вызываться, всегда будет её левым операндом.

Перегрузка бинарных операций (+, +=, ==, !=):

//Перегрузка операции +=

```
class Date{
private:
    void add_days(int n);
    ...
public:
    void operator+= (int n) { // inline
        add_days(n);
    }
};
```

//в main.cpp

d += 7;

d.operator+=(7);

//d.add_days(7); private

//Перегрузка == (одна из реализаций)

```
bool circle::equal(circle a){
    const double eps=0.0001;
    if (abs(x-a.x)<eps && abs(y-a.y)<eps && abs(r-a.r)<eps)
        return true;
    else
        return false;
}

bool circle::operator==(circle a){
    return equal(a);
}
```

//Перегрузка операции сравнения на равенство (вторая реализация)

```
class Date {
    int d, m, y;
public:
    //bool operator==(Date d1, Date d2) типичная ошибка
    bool operator==(Date const & other) {
        return d == other.d &&
            m == other.m &&
            y == other.y;
    }
};
```

Если операция бинарная и определена функцией-членом , то у функции будет один аргумент.

//Перегрузка операции != (как внешняя функция)

```
class Date {
    ...
public:
    ...
    friend bool operator!=(const Date &d1, const Date &d2) {
        return !(d1==d2);
    }
};
```

```
    }  
};
```

Как видим, использование ранее определенной операции `==` позволяет не обращаться к внутренним полям класса напрямую.

Если друг класса определен прямо в классе, то он остается внешней функцией и становится `inline`.

Так как описанная функция является `inline`, и если и операция `!=` тоже `inline`, то в результате будет произведена следующая замена.

//Перегрузка операции != (как член класса)

```
class Date {  
...  
public:  
...  
    bool operator!=(const Date &d2){  
        return !(*this==d2);  
    }  
};
```

//перегрузка + (как член класса)

```
class BigInteger {  
    int data[1024];  
public:  
    BigInteger & operator+=(BigInteger const & other) {  
        // Цикл по data: суммирование и перенос  
        // Возвращаем ссылку на себя  
        return *this;  
    }  
    BigInteger operator+(BigInteger const & bi2);  
};
```

```
BigInteger BigInteger::operator+(BigInteger const & bi2) {
```

```
    //функция не может возвращать ссылку, потому что возвращает новый объект
```

```
    BigInteger res(*this); //копия того объекта, для которого вызвана функция
```

```

        res += bi2; //res хранит bi1 + bi2

        return res;
}

```

//перегрузка + (как внешняя функция)

```

class BigInteger {
int data[1024];
public:
BigInteger & operator+=(BigInteger const & other) {
    // Цикл по data: суммирование и перенос
    // Возвращаем ссылку на себя
    return *this;
}
friend BigInteger operator+(BigInteger const & bi1, BigInteger const & bi2){
    // функция не может возвращать ссылку, потому что возвращает новый объект
    BigInteger res(bi1);
        // копия bi1
        res += bi2;
        // res хранит bi1 + bi2
        return res;
    }
};

```

//перегрузка * на примере класса frac(дроби) дополнительно

```

friend frac operator*(int n, const frac &f) {
    return frac(n * f.ch, f.zn);
}

```

Перегрузка унарных операций (префиксный и постфиксный ++, --): фиктивный параметр int для постфиксной формы:

Префиксная унарная операция может быть определена двумя способами:

- в виде функции-члена: a.operator@()

- в виде внешней функции: `operator@(a)`

//Перегрузка ++d

```
class Date {
```

```
private:
```

...В правило пяти включаются:

- Деструктор — автоматически вызывается при уничтожении объекта, гарантирует, что все выделенные ресурсы будут корректно освобождены.
- Конструктор копирования — позволяет создавать новые объекты как копии существующих.
- Оператор присваивания копированием — позволяет одному уже существующему объекту принять состояние другого существующего объекта.
- Конструктор перемещения — передаёт владение ресурсами от одного объекта к другому.
- Оператор присваивания перемещением — используется, когда существующему объекту присваивается значение rvalue.

```
void add_days(int a);
```

```
public:
```

```
...
```

```
// определение префиксного ++ инкремента
```

```
Date & operator++() {
```

```
add_days(1);
```

```
return *this;
```

```
}
```

```
};
```

Постфиксная унарная операция также может быть определена двумя способами:

- в виде функции-члена `operator@(int)`
- в виде внешней функции `operator@(a, int)`

//Операции d++

```
class Date {
```

```
private:
```

```
...
```

```

void add_days(int a);

public:

...

// определение постфиксного ++ инкремента
// d++ возвращает значение до увеличения, значит возвращать нужно копию, а не ссылку
Date operator++(int) {
    Date d = *this;
    //Date d = Date (*this);
    add_days(1); // ~ ++(*this);
    return d;
}

};

```

Из-за копирования всего объекта операция d++ будет выполняться дольше, поэтому надо выбирать в пользу ++d.

Это верно для своих типов, но для встроенных типов компилятор производит оптимизацию. В результате записи i++ и ++i, например, для переменных типа int, равноценны.

Перегрузка операций ввода/вывода (<<, >>) как внешних дружественных функций, возврат ссылки на поток:

```

friend ostream & operator<<(ostream & os, Date const & d){
    return os << d.d << '.' << d.m << '.' << d.y << '\n';
}

friend istream & operator>>(istream & in, Array & arr) {
    for (int i = 0; i < arr.n; i++) {
        in >> arr.values[i];
    }
    return in;
}

```

Перегрузка оператора индексации []: возврат по ссылке, константная и неконстантная версии:

Её нужно реализовывать **только** как **функцию-член класса**.

Важно, чтобы перегруженная операция доступа к элементу массива по индексу [] возвращала ссылку на элемент массива. Это обусловлено требованиями к соблюдению семантики.

//неконстантная версия

```
int& Array::operator [] (int i){  
    return A[i];  
}
```

//константная версия

```
int Array::operator [] (int i) const{  
    return values[i];  
}
```

//через const — говорим, что поля класса не будут изменены

Перегрузка оператора приведения типа (operator double(), operator char*()):

//перегрузка operator double() на примере класса frac

```
class frac {  
    int m, n;  
    ...  
public:  
    operator double()  
    {  
        return m/(double)n;  
    }  
};
```

//перегрузка operator char*()

```
Str::operator char*() const {  
    char *c=new char[length];  
    strcpy(c,sPtr);  
    return c;  
}
```

Доп.тема(этого в программе нет, просто оставаю это здесь).Неявный вызов операции приведения к char*

Если в программе в том месте, где ожидается char*, появился объект s определенного пользователя типа Str, то в этом случае компилятор для преобразования объекта в char* вызывает перегруженную функцию-операцию приведения operator char* и использует в выражении результирующий char*. Например:

```
cout << s;
```

Операция приведения для класса Str позволяет не перегружать операцию << (поместить в поток), предназначенную для вывода Str

//Str — пользовательская реализация класса String

Ограничения перегрузки операций (нельзя перегрузить ::, .., sizeof, ?: и т. д.):

Хотя С++ позволяет перегружать **почти все** операции, доступные в С++, **возможности** перегрузки **ограничены**.

В частности, **отсутствует** возможность **изменения приоритета** или **количества аргументов** у операций.

Кроме того, **рекомендуется не изменять семантику** операций.

Для некоторых операций перегрузка запрещена (sizeof, .., ::, ?: и пр.)

Следующие операции можно перегружать **только в классе** (методом класса)

1.присваивание =

2.вызов функции ()

3.индексация []

4.доступ через указатель ->

5.приведение типа

Операции можно перегружать **только как внешние**:

ввода/вывода

4. Управление ресурсами. Правило трёх и пяти

Правило трёх: Если классу требуется явный деструктор, конструктор копирования или оператор присваивания копированием, то, скорее всего, нужны все три:

//Более развернутый вариант формулировки

Если класс явно определяет (или требует определения) любой из перечисленных ниже трёх методов, это явный признак того, что класс управляет каким-либо ресурсом (памятью, файлом, сетевым соединением и т.д.).

В такой ситуации потребуется явно определить все три метода:

- **Деструктор** (~ClassName())

Зачем: Освободить захваченный ресурс, чтобы избежать утечек памяти.

- **Конструктор копирования** (ClassName(const ClassName &))

Зачем: Определить, как создавать новый объект как копию существующего. Нужен для создания независимой копии ресурса (глубокое копирование), а не просто копирования указателя (поверхностное копирование).

- **Оператор присваивания** (operator=(const ClassName &))

Зачем: Определить, как корректно заменить ресурс одного уже существующего объекта копией ресурса другого. Это сложнее конструктора копии, потому что нужно:

- Освободить старый ресурс текущего объекта.
- Создать копию ресурса из объекта справа (или переместить).
- Защитить объект от копирования самого себя (if (this != &other)).

//речь идет о динамической памяти)

//Вариант формулировки, который чуток попроще

Если классу нужен свой деструктор (чтобы попрощаться с ресурсом), значит, ему также нужны свои конструктор копирования и оператор присваивания (чтобы правильно размножать этот ресурс).

Если вы определили деструктор, но забыли про конструктор копии и присваивание, компилятор автоматически сгенерирует их за вас.

Но сделает это неправильно (просто скопирует указатели), что приведет к двукратному освобождению памяти и крашу программы.

Оператор копирующего присваивания (operator=): проверка на самоприсваивание (this != &other), корректное освобождение старого ресурса и выделение нового:

```
Array& Array::operator = (const Array &B) {
```

```
    if (this != &B) {
```

```
        delete[] A;
```

```
        n=B.n;
```

```
        A=new int[n];
```

```
        for (int i=0; i<n; i++)
```

```

        A[i]=B.A[i];
    }

    return *this;
}

```

Функция operator= должна быть обязательно функцией класса.

Игнорирование проверки самоприсваивания a=a в **случае использования динамической памяти** в определении класса может привести к **потере данных**.

Исключения в операторе присваивания: базовая гарантия, строгая гарантия (использование локальной копии и swap):

```

Array& Array::operator = (const Array &B) {
    if (this != &B) {
        delete[] A;
        n=B.n;
        A=new int[n];
        for (int i=0; i<n; i++)
            A[i]=B.A[i];
    }

    return *this;
}

```

В данной реализации операции могут появиться проблемы при возникновении **исключения в операции new**.

Поскольку к этому моменту уже выполнена операция delete[] A, объект после исключения в new останется в «**полуразрушенном**» состоянии.

Уровни гарантии безопасности кода:

В C++ выделяют три уровня гарантий безопасности кода при возникновении исключений:

- **базовый** – при возникновении исключения не возникает утечек ресурсов, однако объекты могут находиться в непредсказуемом состоянии;
- **строгий** – если во время операции произошло исключение, то объект будет находиться в том же состоянии, что до начала операции;
- **без исключений** – в данном коде не может возникнуть исключений.

В примере выше — указана **базовая гарантия**

//Пример строгой гарантии(без рисков) с использованием локальной копии

```
Array & Array::operator = (const Array &B) {  
    if (this != &B) {  
        n=B.n;  
        int * newdata = new int[n];  
        for (int i=0; i<n; i++)  
            newdata[i]=B.A[i];  
        delete[] A;  
        A = newdata;  
    }  
    return *this;  
}
```

Идиома copy-and-swap:

Предполагает реализацию операции копирующего присваивания с использованием конструктора копий.

При этом требуется вначале создать вспомогательную функцию-член swap(Array & other), для обмена содержимого текущего объекта с объектом other.

```
void Array::swap(Array & other) {  
    swap(n, other.n);  
    swap(A, other.A);  
}
```

```
Array& Array::operator=(Array other) { //вызов конструктора копии  
    this -> swap(other);  
    return *this;  
}
```

Идиома copy-and-swap позволяет разрабатывать **устойчивые к исключениям операторы присваивания** и **сокращает количество кода** в них **ценой** определения полезной **вспомогательной функции swap**.

Семантика перемещения (C++11): rvalue-ссылки (&&), конструктор перемещения, оператор присваивания перемещением:

Семантика перемещения появилась в стандарте C++11.

Она нацелена на **уменьшение количества создаваемых копий** объектов при выполнении конструктора копии и операции присваивания, которые вызываются для **rvalue** выражений.

Идея move-семантики

Не удалять временный объект и **не выделять** память для полей в новом объекте, а **инициализировать** поля в создаваемом объекте **ссылками** на поля временного объекта.

Конструктор копирования выделяет новую область памяти для хранения данных, вызывая оператор **new**, а **перемещающий конструктор** — **забирает** данные у переданного ему **временного объекта**.

При использовании **move-семантики** деструктор не должен освобождать память временного объекта, поскольку ссылкой на неё инициализируется поле в другом объекте.

Для этого в **move-конструкторе** и **move-операции присваивания** поле указатель временного объекта меняет значение на **nullptr**, а в деструкторе выделенная память освобождается только, если поле указатель не равен **nullptr**

//Пример правильной реализации move-конструктора можно подглядеть в Array

//еще пример move-конструктора

//move-конструктор

```
myvector(myvector&& v){
    name = "Move_Copy ( " + v.name+" )";
    cout << "move > " << name << " " << f << endl;
    size = v.size;
    vect = v.vect;
    v.vect = nullptr; // Не позволит сразу удалить временный объект
}
```

move-операция присваивания

//move-операция присваивания

```
myvector& operator=(myvector&& v){
    cout << name <<" operator-move= > " << v.name <<" " << f <<endl;
    if (vect != nullptr)
        delete[] vect;
    size = v.size;
    vect = v.vect;
    v.vect = nullptr;
    return *this;
}
```

//изменённый деструктор для move-семантики

```
~myvector(){  
    if (vect != nullptr) {  
        f--;  
        cout << "destructor > " << name << " " << f << endl;  
        delete[] vect;  
    }  
}
```

Чтобы отличать функции с перемещающей семантикой в стандарте C++11 введены **rvalue ссылки** — **myvector && v**.

При этом компилятор будет **использовать** функции с **перемещающей семантикой** только в случае, если параметром является rvalue выражение (временный объект).

//Итог

Ввиду наличия большого количества стандартных классов использовать move-конструкторы и move-операции присваивания приходится редко, однако знание такого механизма необходимо.

Правило пяти: деструктор, копирующий конструктор, копирующее присваивание, перемещающий конструктор, перемещающее присваивание:

Правило пяти — правило в C++, которое гласит, что если явно определяется один из пяти специальных методов класса, скорее всего, нужно явно определить и остальные четыре.

Это эволюция предшествующего правила трёх, адаптированная для учёта нововведений C++11, таких как семантика перемещения.

Правило пяти применяется, когда класс управляет ресурсами: динамической памятью, файловыми дескрипторами, сетевыми подключениями и т. д.

Цель правила — корректная обработка владения ресурсами и предотвращение таких проблем, как двойное освобождение или утечки памяти.

Копирование ресурсов может быть ресурсоёмким с точки зрения производительности, а перемещение позволяет избежать ненужных операций копирования, передавая владение ресурсами непосредственно новому объекту.

В правило пяти включаются:

- **Деструктор** — автоматически вызывается при уничтожении объекта, гарантирует, что все выделенные ресурсы будут корректно освобождены.

- **Конструктор копирования** — позволяет создавать новые объекты как копии существующих.
- **Оператор присваивания копированием** — позволяет одному уже существующему объекту принять состояние другого существующего объекта.
- **Конструктор перемещения** — передаёт владение ресурсами от одного объекта к другому.
- **Оператор присваивания перемещением** — используется, когда существующему объекту присваивается значение rvalue.

5. Наследование и полиморфизм

Отношения между классами — Наследование, Включение, Подobie.

Наследование: отношение "is-a". Базовый и производный классы:

Наследование (inheritance) – это способность класса приобретать свойства ранее определённого класса. Один класс может наследовать структуру и поведение другого класса

В языке C++ производный класс наследует все члены **базового** класса, **за исключением конструкторов, деструктора, перегруженной операции присваивания и определения друзей класса**

Таким образом, производный класс содержит в себе все данные-члены и функции-члены базового класса, добавляя к ним новые члены, определённые в нём самом

Кроме того, производный класс может изменять (переопределять) любую наследуемую функцию-член

Правила доступа

Раздел базового класса	Открытый (public)	Защищенный (protected)	Закрытый (private)
Доступность из функций базового класса, функций дружественных классов и из дружественных функций	Да	Да	Да
Доступность из функций классов-наследников базового	Да	Да	Нет
Доступность из других классов и функций	Да	Нет	Нет

Определение производного класса:

```
class <имя производного класса>: { public | protected | private } <имя базового класса>
```

Например:

```
class Student: public Person {...};
```

```
class Ball: public Sphere {...};
```

Открытое наследование(«is-a»):

Открытое наследование устанавливает между классами отношение «**является**», или в английской нотации «**is-a**»

При открытом наследовании всё, что характеризует объекты класса-предка, является справедливым и для объектов класса-наследника

Это свойство называется **совместимостью типов** объектов. Благодаря этому **объект производного класса можно применять вместо объекта базового класса, но не наоборот**

Например, **объекту базового типа** можно присвоить **объект производного типа, указателю на объект базового типа — указатель на объект производного**. Объект производного типа также может быть передан в качестве параметра в функцию, вместо объекта базового типа.

Порядок вызова конструкторов и деструкторов при наследовании:

Конструктор производного класса выполняется после конструктора базового класса. Это правило действует и в цепочках наследования любой длины

```
Ball::Ball(): Sphere(){ setName("NoName"); }
```

```
Ball::Ball(double iniRadius, const string iniName):
```

```
    Sphere(iniRadius), name(iniName){ }
```

//контекст: класс Ball наследуется от класса Sphere

Для указания, **какой именно из конструкторов** базового типа следует вызвать в каждом конкретном конструкторе класса-потомка, используется синтаксис **списка инициализаторов**

Если конструктор базового класса отсутствует в списке инициализации, используется конструктор базового класса по умолчанию

```
Ball::Ball(){ setName("NoName"); }
```

//еще примеры можно будет посмотреть из 4-ой лабы

Если в классе-потомке не определён ни один конструктор, то будет создан конструктор по умолчанию, который вызовет конструкторы по умолчанию всех предков.

Важно, чтобы в иерархии классов всегда были определены конструкторы по умолчанию.

Зоны ответственности конструкторов:

Конструктор **производного** класса отвечает за инициализацию всех **элементов** данных, **добавленных** к унаследованным данным из базового класса.

Конструктор **базового** класса выполняет инициализацию **унаследованных** элементов данных

Порядок вызова деструкторов:

Деструктор производного класса выполняется **перед деструктором базового** класса. В цепочках наследования произвольной длины деструкторы вызываются в **порядке обратном порядку вызова конструкторов**.

Вначале выполнится деструктор класса Ball, затем – деструктор класса Sphere.

Поскольку они явно не определены, то используются автоматические деструкторы.

Порядок вызова конструкторов и деструкторов(более подробный):

- 1.Вызов конструктора базового класса
- 2.Вызов конструктора полей
- 3.Вызов конструктора основного объекта (наследника)
- 4.Вызов деструктора основного класса (наследника)
- 5.Вызов деструкторов полей
- 6.Вызов деструктора предка

Переопределение методов. Отличие переопределения от перегрузки:

```
void Ball::showInform(){  
    cout<<" Ball type - "<<name<<". Specifications:";  
    Sphere::showInform(); //коллизия имён  
}
```

При переопределении функции базового класса в производном классе списки параметров могут не совпадать.

При этом замещающая функция переопределяет исходную функцию, но с другим списком параметров.

Перегрузка при этом **не происходит**, так как она возможна только в одном пространстве имён. Каждый класс имеет своё пространство имён.

Следовательно, производный класс вводит новое пространство имён.

//не путаем крч переопределение с перегрузкой

Вызов функций-членов класса предка из наследника:

```
void Ball::resetBall(double newRadius, const string newName) {  
    resetRadius(newRadius); // вызываем функцию чисто базового класса  
    name = newName;  
}  
  
void Ball::showInform(){  
    cout<<" Ball type - "<<name<<". Specifications:";  
    Sphere::showInform(); //здесь используем :: чтобы обозначить из какого класса вызы-  
ваем эту функцию, т.к они повторяются  
}
```

Объекты производного класса могут вызывать открытые функции-члены базового класса.

Совместимость типов: указатель/ссылка на базовый класс могут указывать на объект производного:

Указатель базового класса может указывать на **производный объект** без явного преобразования типов

Ссылка на базовый класс тоже может ссылаться на объект производного класса

//Пример

```
Ball myBall(5.0, "Volleyball");
```

...

```
Sphere &pb = myBall;
```

```
Sphere *pt = &myBall;
```

```
Sphere *p = new Ball(9.0, "basketball");
```

```
pb.showInform();
```

```
cout << pb.getRadius() << endl;
```

```
pt->showInform();
```

```
cout << pt->getVolume() << endl;
```

```
p->showInform();
```

```
cout << p->getVolume() << endl;
```

Указатель или ссылка **базового** типа позволяет вызывать только **функции базового** класса, поэтому воспользоваться **pb**, **pt** или *p* для вызова **функции** `getName()` производного класса нельзя.

Вызов функции **showInform()** через указатели *p*, *pt* или ссылку *pb* на базовый класс обратится к реализации этой функции в классе `Sphere`, игнорируя ее переопределение в классе `Ball`

Полиморфизм: статическое (раннее) и динамическое (позднее) связывание:

Раннее (статическое) связывание:

Это обусловлено тем, что определение, какую из переопределённых функций вызвать, происходит в момент компиляции по типу переменной-объекта или переменной-ссылки (указателя).

Определение на этапе компиляции вызываемого варианта переопределённой функции называется статическим, или ранним, связыванием.

Динамическое (позднее) связывание:

Для того чтобы обеспечить возможность определения, какая из переопределённых функций должна быть вызвана, не на основе типа переменной-ссылки или указателя, а **на основе типа объекта, на который они ссылаются**, нужен другой механизм — **динамическое, или позднее, связывание**

Определение на этапе выполнения вызываемого варианта переопределённой функции называется динамическим, или поздним, связыванием

Полиморфизм:

Позднее связывание реализует принцип полиморфизма

Полиморфизм позволяет выбирать вариант вызываемой функции в ходе выполнения программы

Виртуальные функции (virtual): механизм позднего связывания:

Позднее связывание реализуется с помощью виртуальных функций

```
class Person{
```

```
public:
```

```
    //все, как было описано выше, кроме функции showInform
```

...

```
virtual void showInform(); //виртуальная функция
```

```
private:
```

```
    //все, как было описано выше
```

```
};
```

Реализация функции showInform как для класса Person, так и для классов-наследников остаётся без изменений

```
//Вызов виртуальных функций
```

```
for ( int i=0; i<5; ++i)
```

```
    p[i] → showInform();
```

Виртуальная функция всегда виртуальна

Если объявление функции в базовом классе начинается с ключевого слова **virtual**, то это делает функцию виртуальной для **базового** класса и **всех** классов, **производных** от базового класса. **Виртуальная функция всегда виртуальна.**

Рекомендуется всегда использовать спецификатор **virtual** в объявлении виртуальных функций, **независимо от их расположения в иерархии наследования.** Это улучшает читаемость кода.

Вывод: Полиморфизм C++ работает **только через указатели и ссылки** на объекты базового класса

Чисто виртуальные функции (= 0). Абстрактные классы (нельзя создать объект):

Объект класса shape без конкретизации типа фигуры не может иметь площади.

Для того чтобы не прибегать к такому искусственному приёму, имеется возможность определять виртуальную функцию без реализации. //контекст можете посмотреть с лекции 6

Виртуальная функция без реализации называется чисто виртуальной.

Синтаксис объявления чисто виртуальной функции — **virtual <тип> <имя> (<список параметров>) = 0;**

```
class shape {
```

```
protected:
```

```
    double x,y;
```

```
public:
```

//конструкторы не создаём, так как используется автоматический конструктор по умолчанию

```
void set_param(double x0, double y0) {
```

```
    x=x0;
```

```
    y=y0;
```

```
}
```

```
virtual void show_area() =0; //чисто виртуальна функция
```

```
};
```

Если класс имеет хотя бы одну чисто виртуальную функцию, его называют абстрактным классом.

Абстрактный класс

- Для абстрактного класса **не могут быть созданы объекты**
- Такой класс может служить **только в качестве базового в системе наследования** и для создания указателей и ссылок, которые будут использованы при реализации полиморфизма.
- Если в базовом классе имеется **чисто виртуальная функция, производный класс должен иметь определение её собственной реализации.**
- Если **реализация** хотя бы одной из чисто виртуальных функций **не будет выполнена, производный класс, в свою очередь, останется абстрактным**

Абстрактные классы посредством чисто виртуальных функций описывают интерфейс, который можно использовать в других классах или функциях, ожидая, что наследники реализуют эти функции.

Полиморфный контейнер(доп.тема) — массив указателей на разные классы(допустим на объекты базового класса и его наследников)

//пример полиморфного контейнера

// 1. Абстрактный базовый класс

```
class Animal {
```

```
public: // Обязательно виртуальный деструктор, чтобы производные классы правильно удалялись!
```

```
    virtual ~Animal() = default;
```

```
    // Чисто виртуальная функция (делает класс абстрактным)
```

```
    virtual void voice() const = 0;
```

```
};
```

// 2. Производный класс "Собака"

```

class Dog : public Animal {
public:
    void voice() const override { std::cout << "Гав-гав!" << std::endl; }
};

// 3. Производный класс "Кошка"
class Cat : public Animal {
public:
    void voice() const override { std::cout << "Мяу!" << std::endl; }
};

int main() { // 4. Создаем ПОЛИМОРФНЫЙ КОНТЕЙНЕР // Вектор хранит умные указатели
    //на базовый класс
    Animal std::vector<std::unique_ptr> zoo;

    // Добавляем в него разные объекты
    zoo.push_back(std::make_unique<Dog>());
    zoo.push_back(std::make_unique<Cat>());
    zoo.push_back(std::make_unique<Dog>());

    // 5. Демонстрация полиморфизма в цикле
    std::cout << "Обходим наш полиморфный контейнер:" << std::endl;
    for (const auto& animal : zoo) {
        // Вызовется метод нужного производного класса благодаря виртуальности
        animal->voice();
    }

    // Память очистится автоматически при выходе вектора из области видимости,
    // так как мы использовали std::unique_ptr.
    return 0;
}

```

Ограничения(Еще раз по базе)

Конструкторы не могут быть виртуальными.

Производный класс не наследует конструкторы базового класса, поэтому бессмысленно делать их виртуальными.

Статические функции также не могут быть виртуальными.

Таблица виртуальных методов (VMT), указатель vptr. Цена виртуальности

Полиморфизм в C++ реализуется с помощью таблиц виртуальных функций — Virtual Methods Table(VMT)

В каждом объекте появляется **дополнительный указатель vptr** на **таблицу виртуальных методов**.

Для каждого класса создается таблица виртуальных методов, которая содержит **адреса всех виртуальных методов этого класса и всех его предков**.

Если в классе и его предках нет виртуальных методов, то в его объекте поле vptr отсутствует (реализуется принцип: не платим за то, что не используем).

Замечание:

В C++ нет общего главного класса-предка, как например, Object в PascalABC.Net, C#, Java. Это делается в целях повышения эффективности.

Как это все работает:

1. Компилятор видит вызов через указатель на базовый класс

Что делает компилятор:

- Определяет, что это вызов виртуальной функции
- Не может определить на этапе компиляции, какая именно реализация будет вызвана
- Генерирует код, который будет определять это во время выполнения

2. Структура объекта в памяти

Для класса с виртуальными функциями компилятор добавляет скрытый указатель vptr (virtual pointer) в начало объекта

Важно: vptr автоматически инициализируется конструктором класса

3. Таблица виртуальных методов (VMT)

Для каждого класса с виртуальными функциями компилятор создает одну таблицу VMT (Virtual Methods Table)

Ключевой момент: Индекс функции (в данном случае 0) одинаков для всех классов в иерархии. Компилятор заранее назначает каждой виртуальной функции уникальный индекс

//Почему индекс известен на этапе компиляции?

Компилятор строит иерархию классов и назначает индексы виртуальным функциям

Важно: Порядок виртуальных функций в VMT фиксируется на этапе компиляции и не меняется

(4, 5 шаг вниз)

4. Как компилятор преобразует вызов

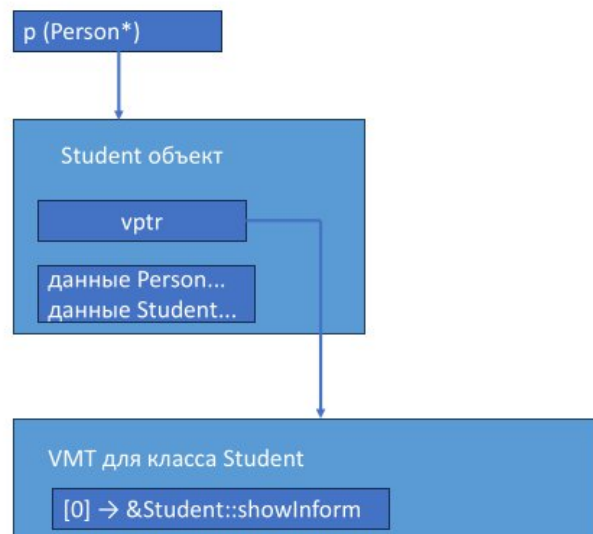
```
// То, что пишет программист:  
p->showInform();  
  
// То, что генерирует компилятор (упрощенно):  
(*(p->vptr[0]))(p);
```

Часть	Значение
p->vptr	Обращение к скрытому указателю vptr объекта, на который указывает p
p->vptr[0]	Доступ к элементу с индексом 0 в таблице VMT (адрес нужной функции)
*(p->vptr[0])	Разыменование — получение самой функции
*(p->vptr[0])(p)	Вызов этой функции с параметром p (указатель на объект становится this)

5. Что происходит во время выполнения

Шаг 5: Что происходит во время выполнения

- Вычисляется p->vptr — из объекта Student берется указатель на VMT класса Student
- Обращение по индексу — из VMT класса Student берется адрес функции с индексом 0 (это &Student::showInform)
- Вызов функции — вызывается Student::showInform(p), где p передается как this



```
Person* p = new Student(); // p указывает на объект Student
```

```
p->vptr → VMT класса Student → адрес Student::showInform
```

```
p = new Professor();
```

```
// p указывает на объект Professor
```

```
p->vptr → VMT класса Professor → адрес Professor::showInform
```

Во время выполнения адрес функции берется из VMT, т.к. `vptr` указывает на VMT того класса, объект которого реально создан. Именно поэтому вызов через один и тот же указатель `p` может приводить к разным функциям

Ключевая идея:

Компилятор создает универсальный код вызова, который во время выполнения обращается к правильной таблице виртуальных функций через указатель `vptr`, хранящийся в объекте. Это и есть механизм позднего (динамического) связывания.

Доп.тема.Последовательность выполнения

Как работает **new**: сначала выделяется память, потом вызывается конструктор.

Как работает **delete**: сначала вызывается деструктор, потом возвращается память.

Виртуальный деструктор (правило: если есть хоть одна виртуальная функция, деструктор должен быть виртуальным):

Правило:

Если в классе есть **хоть бы одна виртуальная функция**, тогда обязательно делаем его **деструктор виртуальным**.

Если базовый класс не имеет явного деструктора, а у потомков класса появятся явные деструкторы,

например, освобождающие динамическую память, то возможна ошибка утечки памяти.

Важно гарантировать, чтобы при уничтожении объекта был вызван деструктор именно того класса- наследника, к которому он относится.

Если базовый класс не требует выполнения явного деструктора, не следует полагаться на деструктор по умолчанию.

Вместо этого необходимо описать виртуальный деструктор с пустой реализацией.

Интерфейс — соглашение между функцией базового класса и наследниками по возможностям взаимодействия

//Второй вариант от Gemini

Интерфейс — это контракт(соглашение), определяющий *что* объект умеет делать (набор методов), но не определяющий *как* он это делает.

Преобразование типов в иерархии:

upcast (от наследника к базовому) – неявное(Приведение типа-наследника к базовому типу):

Фактически в конструкторе копирования в классе Student, const Person & присваиваем переменную типа const Student &.

Переменной типа предок можно присвоить переменную типа потомок, но не наоборот.

```
Person p("Иванов", 20);
```

```
Student s("Петров", 19, "ЮФУ", 3);
```

```
p = s;
```

При присваивании объекта производного класса переменной базового класса происходит **копирование только полей базового класса**, остальная часть информации объекта производного класса будет утеряна.

```
s = p; // ошибка компиляции
```

Попытка присвоить объекту класса наследника объект класса предка приведёт к **ошибке компиляции**.

При работе с указателями и ссылками на объекты предка и наследника действует аналогичное правило преобразования типов

```
Person* pp = &p;
```

```
Student* ss = &s;
```

```
pp = ss;
```

```
ss = pp; // ошибка компиляции
```

```
Person& rp = s;
```

```
Student& rs = p; // ошибка компиляции
```

Операция присваивания не наследуется

Поэтому, чтобы выполнить присваивание для private полей, унаследованных от базового класса Person, необходимо выполнить операцию присваивания, определённую в классе Person.(упоминал в реализации класса Student)

downcast (от базового к наследнику) – явное, static_cast, dynamic_cast:

В этом случае необходимо использовать явное приведение типов с помощью шаблонной функции `static_cast<тип_наследника>`

Добавим в класс `Student` функцию `get_univ()`, которая возвращает название учебного заведения, где учится студент.

Такой функции нет, и не может быть в `Person`.

```
class Student : public Person {
```

```
    ...
```

```
public:
```

```
    ...
```

```
    string get_univ() const {
```

```
        return univ;
```

```
    }
```

```
};
```

```
//примеры того как правильно использовать эту штуку
```

```
Person p = new Student("Петров", 19, "ЮФУ", 3);
```

```
p->get_univ(); // ошибка компиляции
```

При вызове `p → get_univ()` произойдёт ошибка компиляции, так как в `Person` не определена функция `get_univ()`. Для корректного вызова указатель `p` нужно привести к типу `*Student`

```
// Современный стиль:
```

```
static_cast<Student>(p)->get_univ();
```

```
// Старый стиль:
```

```
((Student)p)->get_univ();
```

Аналогичная ситуация возникает и при использовании ссылок:

```
Person & rp = *new Student("Петров", 20, "ЮФУ",  
3); static_cast<Student&>(rp).get_univ(); delete
```

Преобразование `downcast` в C++ возможно только **для указателей или ссылок** на объекты.

Корректное преобразование `downcast` возможно только **как обратное** преобразование к `upcast`.

dynamic_cast: безопасное приведение для полиморфных типов, возвращает nullptr для указателей или генерирует std::bad_cast для ссылок:

Нужно выполнить **явное приведение типа** с помощью операции **dynamic_cast <тип>** для преобразования указателя на shapeC к указателю на filled_circle

```
dynamic_cast<filled_circle *>(p[1])->set_color(10);
```

Оператор dynamic_cast может быть применён к указателям или ссылкам

Правила для dynamic_cast:

Проверка корректности приведения типов для классов с виртуальными функциями производится во время выполнения программы.

Если в классе имеются виртуальные методы, то на этапе выполнения операция dynamic_cast пытается выполнить преобразование к указанному типу данных.

В случае преобразования указателя к типу данных, который не является фактическим типом объекта, в результате будет получен **нулевой указатель**.

При работе со ссылками, если преобразование невозможно, будет сгенерировано исключение **std::bad_cast**.

Для его обработки операцию dynamic_cast следует поместить в блок try/catch. Если **виртуальных** методов в классе и его предках **нет**, то **dynamic_cast** работать **не будет**. Нужно использовать **static_cast**

Правило для полиморфных типов:

Важно: dynamic_cast работает **ТОЛЬКО** для **полиморфных** типов (классов, имеющих хотя бы одну виртуальную функцию).

Если класс **не полиморфный**, dynamic_cast вызовет **ошибку компиляции**. В этом случае используйте **static_cast**.

RTTI (Runtime Type Information): typeid, std::type_info:

Операция **dynamic_cast** использует механизм динамической идентификации типа данных RTTI

RTTI (Runtime Type Information) — это механизм, который позволяет:

- определить фактический тип объекта во время выполнения
- безопасно выполнять приведение типов в иерархии наследования

- получать информацию о типе (имя, размер)

RTTI основана на способности системы сообщать о динамическом типе объекта и предоставлять информацию об этом типе во время выполнения (в отличие от времени компиляции)

RTTI доступен только для классов, которые являются полиморфными, т.е. у них есть хотя бы одна виртуальная функция, потому что именно для них компилятор создает **VMТ**, в которой хранится информация о типе

dynamic_cast позволяет идентифицировать динамический тип переменной во время выполнения

Операция typeid() и структура type_info:

typeid() используется для определения типа переменной во время выполнения.

возвращает ссылку на объект класса std::type_info, который содержит поля, позволяющие получить информацию о типе.

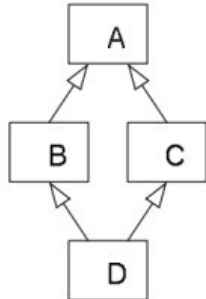
Этот класс содержит перегруженные операции == и !=, а также функцию name().

Множественное наследование, проблема ромба (diamond problem), виртуальное наследование:

Множественное наследование и его проблемы:

```
class orient {  
  
public:  
  
    void print() { std::cout << "orient" << std::endl; }  
  
};  
  
class point {  
  
public:  
  
    void print() { std::cout << "point" << std::endl; }  
  
};  
  
class vector : public point, public orient {};  
  
int main() {  
  
    vector v;  
  
    v.print();  
  
}
```

Проблема ромба



Проблема ромба (Diamond problem) — классическая проблема в языках, которые поддерживают возможность множественного наследования

В C++ решение проблемы оставлено на усмотрение программиста

Ромбовидная проблема

Ромбовидная проблема — прежде всего проблема дизайна, и её решение должно быть предусмотрено на этапе проектирования

На этапе разработки её можно разрешить следующим образом:

- вызвать метод конкретного суперкласса
- обратиться к объекту подкласса как к объекту определённого суперкласса
- переопределить проблематичный метод в последнем дочернем классе

Решение: создать контейнер, содержащий указатели (или ссылки) на объекты базового класса

Доп. тема: auto (пригодится как раз чтобы перебирать контейнер из разных объектов)

for (переменная: массив)

statement;

for (auto x: p)

tellMeAboutYourself(x);

Для пользовательских типов данных, используемых в качестве параметра цикла, рекомендуется применять **auto**

До C++ 11 auto использовалось для явного указания, что переменная имеет автоматический класс памяти (время существования ограничено блоком, в котором она определена). Однако начиная с C++ 11 все переменные по умолчанию имеют автоматический класс памяти

Начиная с C++ 11, ключевое слово auto используется для вывода типа (автоматического определения типа)

Виртуальное наследование:

Виртуальное наследование (virtual inheritance) **предотвращает** появление **множественных объектов базового класса** в иерархии наследования.

```
class Device {  
  
public:  
  
    Device() {  
        cout << "Device constructor called" << endl;  
    }  
  
    void turn_on() {  
        cout << "Device is on." << endl;  
    }  
  
};
```

```

class Computer: virtual public Device {
public:
    Computer() {
        cout << "Computer constructor called" << endl;
    }
};

class Monitor: virtual public Device {
public:
    Monitor() {
        cout << "Monitor constructor called" << endl;
    }
};

class Laptop: public Computer, public Monitor {};

int main() {
    Laptop Laptop_instance;
    Laptop_instance.turn_on();
    return 0;
}

```

Конструктор базового класса Device будет вызван только единожды, а обращение к методу turn_on() без его переопределения в дочернем классе не будет вызывать ошибку при компиляции.

Без виртуального наследования

```
//class Laptop : public Computer, public Monitor, public Device {};
```

Ошибка (активно) E0266 "Laptop::turn_on" не является однозначным

Предупреждение C4584 Laptop: базовый класс "Device" уже является базовым классом для "Computer"

Ошибка C2385 неоднозначный уровень доступа "turn_on"

Спецификаторы override и final:

virtual — начало цепочки виртуальных методов

override — метод является переопределением виртуального метода в базовом классе

final — конец цепочки виртуальных методов (производный класс не должен переопределять виртуальный метод)

Отмечайте виртуальные функции или как **virtual (начало цепочки)**, или как **override (переопределение)**, но не то и другое одновременно.

//пример использования ovveride

```
class B {
public:
    virtual void f(int) { std::cout << "B::f" << std::endl; }
};
```

```
class D : public B {
public:
    void f(int) override { std::cout << "D::f" << std::endl; }
};
```

//пример использования final

```
class B {
public:
    virtual void f(int) { std::cout << "B::f" << std::endl; }
};
class D : public B {
public:
    virtual void f(int) override final { std::cout << "D::f" << std::endl; }
};
```

Функцию, объявленную как **final**, нельзя переопределить

Доп.тема:Отношение подобия:

C++ рассматривает открытое наследование как отношение типа «является».

В рассмотренных примерах класс Student открыто наследует классу Person.

При этом в случае необходимости, компилятор неявно преобразует указатель или ссылку на объект класса Student в указатель или ссылку на объект класса Person

Используя закрытое наследование, можно реализовать отношение «подобен», или «**as-a**»

Закрытое наследование (private) как отношение "as-a" (реализовано посредством):

В противоположность открытому наследованию компиляторы в случае закрытого наследования **не преобразуют указатели или ссылки** на объекты производного класса в указатели или ссылки на объекты базового класса.

Кроме того, члены, наследуемые от закрытого базового класса, **становятся закрытыми**, даже если в базовом классе они были объявлены как защищенные или открытые.

Поэтому **для объектов наследника мы не можем вызывать функции предка**.

Закрытое наследование — смысл:

Закрытое наследование означает «реализовано посредством...»

Делая класс Derived закрытым наследником класса Base, мы заинтересованы в **использовании уже** написанного для Base **кода**

Закрытое наследование означает наследование одной **только реализации, без интерфейса**.

Закрытое наследование ничего не означает в ходе проектирования программного **обеспечения и обретает смысл только на этапе реализации**.

Закрытое наследование — это исключительно прием реализации, который является альтернативой включению (композиции).

6. Дружественность (friend)

Дружественные функции (доступ к закрытым членам класса):

Чтобы разрешить внешней функции доступ к защищённым полям объектов, необходимо её сделать **дружественной** для этих классов.

Для этого описание функции со спецификатором **friend** помещается в интерфейс каждого из классов.

Объявление **функции дружественной** классу разрешает **доступ** к защищённым полям класса **только для этой функции**.

Дружественными по отношению к рассматриваемому классу могут быть **не только внешние функции, но и функции другого класса**, или даже **другой класс** (все его функции).

На функции, объявленные дружественными, **не распространяются действия спецификаторов** (public, private, protected).

Объявление функций дружественными рекомендуется располагать либо в самом начале, либо в самом конце описания класса.

//Пример для функции

// Опережающее объявление, чтобы Warrior знал о существовании Mage

```
class Mage;
class Warrior {
private:
```

```

    int health = 150;

    int armor = 50;

public:
    // Подружили глобальную функцию с Воином
    friend bool isStronger(const Warrior& w, const Mage& m);
};

class Mage {
private:
    int health = 90;
    int manaShield = 120;
public:
    // Подружили ТУ ЖЕ глобальную функцию с Магом
    friend bool isStronger(const Warrior& w, const Mage& m);
};

// Сама функция является независимой, но видит приватные поля обоих классов
bool isStronger(const Warrior& w, const Mage& m) {
    int warriorPower = w.health + w.armor;
    int magePower = m.health + m.manaShield;
    return warriorPower > magePower;
}

```

//В большинстве случаев используется для перегрузки операторов для данного класса

Если больше ни для каких других целей геттеры и сеттеры не нужны, то вместо них можно и нужно использовать механизм дружественности.

Важно, что при этом сокращается количество вызовов функций.

Дружественные классы:

Ориентируясь на структуру вашей методички (в частности, на Главу 1, где разбирается инкапсуляция и сокрытие данных), ниже представлен подробный теоретический материал по теме «Дружественные классы и функции» для включения в учебное пособие.

Дружественные функции и классы (Friendship)

Инкапсуляция строго ограничивает доступ к закрытым (private) и защищенным (protected) членам класса извне. Однако иногда возникает необходимость предоставить отдельным сторонним функциям или другим классам прямой доступ к скрытым данным для повышения эффективности или упрощения архитектуры (например, при реализации операторов ввода-вывода или тесном взаимодействии двух зависимых классов).

Для этого в C++ используется механизм **дружественности** с помощью ключевого слова friend.

Дружественные классы(пример и инфу брал из Gemini)

Если один класс должен активно работать с внутренним устройством другого класса, можно сделать весь класс дружественным.

Дружественный класс — это класс, все методы которого автоматически становятся дружественными для другого класса и получают доступ к его закрытому интерфейсу.

```
#include #include
```

```
class Account { private: std::string m_owner; double m_balance;
```

```
public: Account(std::string owner, double balance) : m_owner(owner), m_balance(balance) {}
```

```
// Класс BankSystem теперь имеет доступ ко всему внутри Account
```

```
friend class BankSystem;
```

```
};
```

```
class BankSystem {
```

```
public:
```

```
void deposit(Account& acc, double amount) { if (amount > 0) { acc.m_balance += amount; //Прямое изменение private поля другого класса
```

```
} }
```

```
void showReport(const Account& acc) {
```

```
    std::cout << "Владелец: " << acc.m_owner << ", Баланс: " << acc.m_balance << "$" << std::endl;
```

```
}
```

```
};
```

```
int main() {
```

```
    Account myAcc("Петров", 500.0); BankSystem system;
```

```
    system.deposit(myAcc, 150.0);  
    system.showReport(myAcc);  
    return 0;  
}
```

Сравнение дружественных функций с функциями доступа (геттерами/сеттерами):

В соответствии с принципом инкапсуляции работа с защищёнными членами класса должна быть организована через открытые функции.

В рассматриваемом примере можно было бы организовать геттеры и сеттеры для доступа к закрытым членам классов.

```
double circle::s(){ return 3.14*r*r;}  
  
void circle::set_r(double r1){  
    if (r1<0) r=0;  
    else r=r1;  
}  
  
double circle::get_r(){ return r;}
```

Геттеры и сеттеры нужны для того, чтобы внешние функции, объекты могли получить безопасный доступ к закрытым полям класса.

Если больше ни для каких других целей геттеры и сеттеры не нужны, то вместо них **можно и нужно** использовать **механизм дружественности**, но при этом принцип инкапсуляции **нарушается**.

Важно, что при этом **сокращается количество вызовов функций**.

7. Статические члены класса

Статические поля: одна копия для всех объектов, инициализация вне класса:

Если данные-члены объявлены с квалификатором **static**, то для всех объектов класса поддерживается только одна копия таких данных

Статический член используется совместно всеми объектами данного класса

Для того чтобы существовал статический член, не обязательно, чтобы существовали объекты такого класса

//Пример объявления статического члена класса

```
class Counter {  
private:  
    static int count;  
public:  
    static int getCount() {  
        return count;  
    }  
    Counter() {  
        ++count;  
    }  
    ~Counter() {  
        --count;  
    }  
};
```

Инициализация и использование статического члена класса:

```
int Counter::count = 0;  
int main() {  
    cout << "before " << Counter::getCount() << endl;  
    Counter first;  
    cout << "after first " << first.getCount() << endl;  
    Counter * second = new Counter();  
    cout << "after second " << second->getCount() << endl;  
    delete second;  
    cout << "after delete second " << Counter::getCount() << endl;  
    return 0;  
}
```

Правила для статических членов класса:

Внутри класса возможно **только объявление** статического члена, но **не** его **определение**.

```
private:  
    static int count;
```

Статические данные можно **использовать** только **после определения**.

Это делается путем **нового объявления статической переменной**, причем используется оператор разрешения области видимости для того, чтобы идентифицировать тот класс, к которому принадлежит переменная.

```
int Counter::count = 0;
```

Статические члены-данные подчиняются **правилам доступа** к членам класса.

Однако **определение** статических членов-данных выполняется **вне зависимости** от спецификатора доступа.

Статические методы: не имеют указателя `this`, могут обращаться только к статическим членам:

Определение статических членов-данных класса рекомендуется располагать вместе с определением самого класса в заголовочном файле.

Статические функции-члены не могут обращаться к нестатическим данным или вызывать нестатические функции этого же класса.

Это связано с тем, что в статические функции не передаётся указатель **this** на объект, для которого вызвана функция.

Реализация статической функции записывается так же, как и реализация любой другой функции-члена класса.

При этом если реализация вне класса, то ключевое слово `static` не указывается.

Подсчёт количества объектов с помощью статического поля:

Например, определение статической функции `getCount` вне класса могло бы выглядеть следующим образом

```
int Counter::getCount() {  
    return count;  
}
```

8. Шаблоны (Templates):

Шаблоны функций и шаблоны классов:

Шаблоны классов, так же как и шаблоны функций, являются трафаретами, по которым компилятор создает **шаблонные классы** и **шаблонные методы**

- Шаблоны классов часто называют **параметризованными типами**, так как они имеют один или большее количество параметров типа, определяющих настройку шаблона класса на специфический тип данных при создании объекта класса

//Шаблоны классов

```
template< class T >

class A {

...

public:

    A();

    void f( T data );

    void g( void );

};

template< class T >

    void A<T>::f( T data ) {

        ...

    }

template< class T >

    void A<T>::g( void ) {

        ...

    }
```

//Шаблоны функций

```
template<typename T>

void saySomething(T something){

    cout << something << endl;

}
```

//отличий между typename и template в 99 процентов случаев нет(в современном стиле стоит использовать typename)

Параметры шаблонов: типы (typename/class), нетиповые параметры (константы), параметры-шаблоны:

//по данной подтеме, все кроме слайда, брал из Gemini

1. Параметры-типы (typename и class)

Это самый распространенный вид параметров. Они служат «заполнителями» (placeholder) для типов данных (таких как int, double, std::string или пользовательских структур типа Student), которые будут переданы при создании объекта.

- **Синтаксис:** Ключевые слова typename и class в контексте объявления параметров шаблона **абсолютно эквивалентны**. Исторически сначала появилось слово class, но позже было добавлено typename, чтобы подчеркнуть, что параметром может быть не только класс, но и любой встроенный тип.

```
#include <iostream>
```

```
// Объявление шаблона класса с параметром-типом
```

```
// Шаблон принимает любой тип T. Использование typename или class здесь идентично.
```

```
template <typename T>
```

```
class Box {
```

```
private:
```

```
    T value; // Поле типа T
```

```
public:
```

```
    Box(T v) : value(v) {} // Конструктор со списком инициализаторов
```

```
    T getValue() const { return value; }
```

```
};
```

```
int main() {
```

```
    Box<int> intBox(5);      // T становится int
```

```
    Box<std::string> strBox("MechMat"); // T становится std::string
```

```
    std::cout << intBox.getValue() << " " << strBox.getValue() << std::endl;
```

```
    return 0;
}
```

2. Нетиповые параметры шаблона (Параметры-константы)

Нетиповой параметр шаблона представляет собой не тип данных, а **конкретное значение (константу)**, которая должна быть известна еще на этапе компиляции программы.

- **Разрешенные типы:** Целочисленные типы (int, size_t, char, bool), указатели, ссылки, а начиная с C++20 — некоторые типы с плавающей точкой (float, double) и строковые литералы.
- **Главное свойство:** Значение нетипового параметра нельзя изменить внутри класса, оно ведет себя как константа.

Связь с курсом (Аналогия с std::array): Именно так устроен стандартный контейнер std::array<typename T, size_t N>, где N — это размер массива, задаваемый при компиляции.

```
// T — параметр-тип, N — нетиповой параметр (размер массива)
template <typename T, size_t N>
class StaticArray {
private: T m_data[N]; // Статический массив, размер которого определяется при компиляции
public:
    size_t getSize() const { return N; } // Возвращает значение параметра шаблона
    T& operator[](size_t index) { return m_data[index]; }
};

int main() { // Создаем массив из 10 целых чисел. Память выделится на стеке, а не в куче.
    StaticArray<int, 10> myArray;
    myArray[0] = 42;
    std::cout << "Размер массива: " << myArray.getSize() << std::endl;
    return 0;
}
```

3. Параметры-шаблоны (Template template parameters)

Это наиболее сложный вид параметров. Он применяется, когда параметром нашего шаблона должен стать **не просто какой-то тип, а другой шаблон**, у которого еще нет конкретных аргументов.

- **Зачем это нужно?** Например, вы пишете свой класс (скажем, аналитическую систему), и вы хотите, чтобы пользователь мог сам выбрать, в каком контейнере хранить

данные внутри (std::vector, std::list или std::deque), не привязываясь к конкретному типу данных внутри этого контейнера.

```
//пример
```

```
// Объявление параметра-шаблона.
```

```
// Container — это шаблон, который сам внутри принимает один параметр-тип (класс-контейнер)
```

```
template <typename T, template <typename...> class Container>
```

```
class DataWrapper {
```

```
private: Container m_storage; // Сконструировали контейнер Container для типа T
```

```
public:
```

```
    void add(const T& item) { // Предполагаем, что у выбранного контейнера есть метод  
        push_back m_storage.push_back(item); } };
```

```
int main() { // В качестве второго параметра передаем САМ ШАБЛОН std::vector, а не  
    std::vector
```

```
    DataWrapper<int, std::vector> vectorWrapper;
```

```
    vectorWrapper.add(10);
```

```
    // Легко меняем внутреннее хранилище на std::list без изменения кода класса
```

```
    DataWrapper<std::string, std::list> listWrapper;
```

```
    listWrapper.add("SFEDU");
```

```
    return 0;
```

```
}
```

Виды параметров шаблонов

ten Нетиповой параметр (параметр обычных типов)

```
template < int ArrayLength, typename SomeValueType >
class SomeClass {
>     SomeValueType SomeValue;
cla:     SomeValueType SomeArray[ ArrayLength ];
        ...
};

П
N

SomeClass < 20, int > SomeVariable;
SomeClass < 30, double > SomeVariable2;
```

Параметры-шаблоны

```
template< class Type, template< class > class Container >
class CrossReferences {
    Container< Type > mems;
    Container< Type* > refs;
    /* ... */
};
```

```
CrossReferences< Date, vector > cr1;
// Container = vector
// mems = vector<Date>
// refs = vector<Date*>
```

```
CrossReferences< string, set > cr2;
// Container = set
// mems = set<string>
// refs = set<string*>
```

- Container — это параметр-шаблон, который сам является шаблоном
- Container<Type> mems — использует шаблон Container с типом Type
- Container<Type*> refs — использует шаблон Container с типом Type*

Это позволяет создавать гибкие структуры, где тип контейнера задается как параметр.

Специализация шаблонов (полная и частичная):

Сгенерированные классы и функции называются специализациями

По умолчанию, шаблон предоставляет единственное определение, которое должно использоваться для всех аргументов шаблона

Версия шаблона для конкретного аргумента шаблона называется специализацией

Только специализации шаблонов содержат настоящий код

Различают сгенерированные и явные (пользовательские)

Альтернативные определения шаблона для конкретного аргумента шаблона называются специализациями, определяемыми пользователями

Явная специализация:

Явная специализация позволяет обеспечить **альтернативные реализации** шаблона.

Например, если для специфического типа данных нужен класс, который не соответствует общему шаблону класса, можно явно определить его, отменив тем самым действие шаблона для этого типа.

Так шаблон класса Stack может использоваться практически для любого типа.

Однако может возникнуть необходимость создать специфический класс Stack для некоторого типа, например, Specific.

Для этого нужно создать на выбор явную специализацию шаблона

```
template <> Stack<Specific>{  
    ...//определение специфического стека  
}  
  
//новый класс с именем Stack<Specific>  
Stack<Specific>{  
    ...//определение специфического стека  
}
```

Пример специализации для const char*

```
template <>
class Stack<const char*> {
private:
    // специальная реализация для строк
    // например, выделение памяти под копию строки
public:
    void push(const char* s) {
        char* copy = new char[strlen(s) + 1];
        strcpy(copy, s);
        // ... помещаем копию в стек
    }
    // ... остальные методы
};
```

Теперь при создании Stack<const char*> будет использоваться эта специализация, а не общий шаблон.

//можно будет подглядеть в лабах

Особенности компиляции шаблонов (код в заголовочных файлах):

Аналогично шаблонам функций, все шаблоны классов и структур должны быть помещены в **заголовочные файлы**.

Для шаблонов классов в отличие от просто классов **выносить реализации** их членов-функций в отдельный *.cpp файл **нельзя**.

Это связано с тем, что **шаблоны в языке C++ не компилируются**, потому что шаблон представляет собой не фрагмент программного кода, а лишь **инструкции для построения кода**.

Инстанцирование шаблона. Ключевое слово typename:

Подстановка конкретного типа в шаблон приводит к созданию шаблонного класса, т.е. к инстанцированию шаблона (template instantiation).

Аналогично функция инстанцируется (генерируется, конкретизируется) из шаблона функции и аргумента шаблона.

Процесс порождения функции или класса из шаблона называется инстанцированием

Количество инстанций зависит от количества используемых типов.

Пример:

```
Stack<int> intStack; // инстанцируется класс Stack<int>
```

```
Stack<double> dblStack; // инстанцируется класс Stack<double>
```

Если в коде не вызывается метод `push()` для `Stack<int>`, то этот метод не будет сгенерирован!

Конструкторы преобразования и ключевое слово `explicit`:

Конструктор преобразования (конструктор с единственным аргументом) может быть использован для преобразования объектов разных типов (включая встроенные типы) **в объекты данного класса**.

Операция преобразования (называемая также операцией **приведения**) может быть использована для преобразования **объекта одного класса в объект другого класса или в объект встроенного типа**. Такая операция преобразования должна быть **нестатической функцией-членом**. Операция преобразования этого вида **не может быть дружественной функцией**.

//Пример того, для чего нужен конструктор преобразования(frac класс дроби)

```
frac f = 2;
```

То есть запись вида `f = 2;` должна быть эквивалентна `f = frac(2);`

Любой конструктор, который **может быть вызван с одним** параметром является конструктором преобразования и **служит для неявного преобразования параметра к типу объекта данного класса**.

//еще наглядный пример

Рассмотрим такой код:

```
myvector<int> v(10), v1(10); // 10 нулей
```

```
v1 = v + 10;
```

Возможно, делая запись `v1 = v + 10;` мы подразумеваем увеличение размера вектора на 10 и как следствие получение в `v1` нового вектора из 20 нулей, или увеличение каждого элемента на 10.

На деле `v1 = v + 10;` эквивалентно `v1 = v + myvector<int>(10);`

//получили поэлементное сложение двух векторов, при условии определенной операции +

То есть мы получим еще один вектор из 10 нулей.

Для того чтобы избежать данной ошибки, необходимо запретить преобразовывать 10 к `myvector<int>(10)`.

Для этого существуют **явные конструкторы преобразования**.

Явные конструкторы преобразования

Явный конструктор преобразования задается с помощью ключевого слова **explicit**, поэтому его еще называют **explicit-конструктор**.

```
class myvector {  
    ...  
public:  
    explicit myvector(int n) {...}  
}
```

Теперь компилятор запретит выражения типа $v1 = v + 10$;

Если же необходимо сложить два вектора, тогда надо явно указывать выполняемую операцию как $v1 = v + \text{myvector}(10)$;

9. Обработка исключений (Exception Handling):

Конструкция try, catch, throw:

Порядок catch блоков имеет значение!

```
try {  
    // код, который может бросить исключение  
}  
catch(bad_cast &e) {  
    // сначала идут более специфичные исключения  
}  
catch(exception &e) {  
    // затем более общие  
}  
catch(...) {  
    // "поймать всё" — должно быть последним  
}
```

В последовательности блоков catch вначале должны обрабатываться более специфичные исключения.

Если поставить catch(...) первым, все остальные catch блоки никогда не сработают!

1. Конструкция try, catch, throw

Процесс обработки исключений строится вокруг трех ключевых слов:

- **try (попытка)** — блок кода, в котором потенциально может возникнуть ошибка.

- **throw (выброс)** — оператор, который генерирует (выбрасывает) исключение. В качестве исключения можно выбросить объект любого типа (переменную, строку, объект класса).
- **catch (перехват)** — блок кода (обработчик), который принимает и обрабатывает исключение определенного типа, если оно было выброшено внутри try.

//пример

```
#include <iostream>
```

```
#include <string>
```

// Функция, которая может сгенерировать исключение

```
double divide(double a, double b) {
```

```
    if (b == 0.0) {
```

```
        throw std::string("Ошибка: Деление на ноль!"); // Выбрасываем объект std::string
```

```
    }
```

```
    return a / b;
```

```
}
```

```
int main() {
```

```
    double x = 10, y = 0;
```

```
    try {
```

```
        // Код, выполняющийся в обычном режиме
```

```
        double result = divide(x, y);
```

```
        std::cout << "Результат: " << result << std::endl; // Пропустится, если b == 0
```

```
    }
```

```
    catch (const std::string& message) { // Перехват исключения по ссылке
```

```
        // Этот блок выполнится ТОЛЬКО если было выброшено std::string
```

```
        std::cerr << message << std::endl;
```

```
    }
```

```
    std::cout << "Программа продолжит выполнение..." << std::endl;
```

```
    return 0;
```

```
}
```

Порядок обработчиков catch (от частного к общему, catch(...)):

После одного блока try может идти несколько блоков catch для обработки разных типов ошибок.

Обработчики catch проверяются сверху вниз. Они должны располагаться **от частного к общему**.

1. **Наследование:** Если вы перехватываете объекты классов из одной иерархии (например, базовый класс `std::exception` и его наследник `std::runtime_error`), то **сначала должен идти catch для класса-потомка (частное)**, а затем для базового класса (общее). Если поставить базовый класс первым, он перехватит и «свои» ошибки, и ошибки потомков за счет полиморфизма, и до второго catch управление никогда не дойдет (компилятор выдаст предупреждение или ошибку).
2. **Универсальный перехватчик catch(...):** Три точки означают «перехватить абсолютно любое исключение». Этот блок **всегда должен стоять самым последним**.

//пример

```
#include <iostream>
```

```
#include <stdexcept> // Содержит стандартные исключения
```

```
int main() {
```

```
    try {
```

```
        // Какая-то логика, которая может выбросить разные типы ошибок
```

```
        throw std::runtime_error("Критическая ошибка логики");
```

```
    }
```

```
    // 1. Сначала перехватываем конкретного потомка (Частное)
```

```
    catch (const std::runtime_error& e) {
```

```
        std::cout << "Поймали runtime_error: " << e.what() << std::endl;
```

```
    }
```

```
    // 2. Затем перехватываем базовый класс для всех стандартных исключений (Общее)
```

```
    catch (const std::exception& e) {
```

```
        std::cout << "Поймали базовое исключение: " << e.what() << std::endl;
```

```
    }
```

```
    // 3. Самым последним ловим всё остальное (int, char, сторонние структуры)
```

```
    catch (...) {
```

```
        std::cout << "Поймано неизвестное исключение!" << std::endl;
```

```
}  
  
}
```

Исключения в конструкторах и деструкторах:

Вот все виды исключений, которые могут там произойти:

А) Стандартные исключения STL (`<stdexcept>` и `<new>`)

1. `std::bad_alloc` (Ошибка выделения памяти)

- **Когда возникает:** Самое частое исключение в конструкторах. Возникает, если внутри конструктора происходит выделение динамической памяти через оператор `new` (например, для массивов, матриц или внутренних буферов), а у системы закончилась свободная оперативная память.
- *Пример:* `buffer = new int[1000000000000];` в конструкторе класса для работы с графикой.

2. `std::invalid_argument` (Неверный аргумент)

- **Когда возникает:** Если в конструктор с параметрами переданы заведомо некорректные данные, с которыми объект не может существовать.
- *Пример:* Конструктор класса `Date(int day, int month)` выбросит его, если передать `month = 13` или `day = -5`.

3. `std::out_of_range` (Выход за пределы диапазона)

- **Когда возникает:** Если конструктор нового объекта инициализируется элементами из другого контейнера (например, по индексам или при копировании части строки), и эти индексы неверны.
- *Пример:* Инициализация поля типа `std::string` через подстроку несуществующей длины.

4. `std::length_error` (Превышение максимального размера)

- **Когда возникает:** Если в конструкторе предпринимается попытка создать внутренний стандартный контейнер (например, `std::vector` или `std::string`) размера, превышающего максимально допустимый (`max_size()`).

5. `std::runtime_error` (Общая ошибка выполнения)

- **Когда возникает:** Используется для ошибок, которые невозможно предугадать логически до запуска программы.
- *Пример:* Конструктор класса `DatabaseConnection` пытается подключиться к базе данных по сети, но сервер недоступен или указан неверный порт.

Б) Системные исключения и аппаратные сбои

Эти ошибки часто приводят не к контролируемому throw, а к системным сигналам, но в некоторых средах или при использовании сторонних библиотек они могут транслироваться в исключения:

- **Исключения ввода-вывода (`std::ios_base::failure`):** Если конструктор открывает файл для чтения конфигурации (например, класса `ConfigReader`), а файла не существует или к нему нет прав доступа.
- **Арифметика (`std::underflow_error` / `std::overflow_error`):** Ошибки переполнения или деления на ноль при вычислении начальных физических/математических параметров объекта в списке инициализации.

В) Пользовательские исключения

- Любые структуры или классы, созданные программистом для специфики предметной области (например, `struct EmptyNameException {}`; или `class NetworkTimeout : public std::exception {}`).

2. Исключения, возникающие в ДЕСТРУКТОРАХ

Деструктор вызывается при уничтожении объекта для освобождения ресурсов. Сценариев, способных породить здесь ошибку, меньше, но каждый из них **смертельно опасен** для программы.

А) Стандартные исключения и ошибки работы с памятью

1. Повторное удаление памяти (двойное освобождение / Double Free)

- **Как возникает:** Если в классе некорректно реализован (или забыт) конструктор копирования, два объекта будут указывать на один и тот же адрес в куче. При уничтожении первого объекта деструктор успешно удалит память через `delete`. При уничтожении второго объекта его деструктор вызовет `delete` по уже очищенному адресу.
- **Результат:** Обычно это вызывает немедленный крах программы (Segmentation Fault), но в некоторых реализациях может выбрасываться системное исключение среды выполнения.

2. `std::ios_base::failure` (Ошибка закрытия ресурсов)

- **Как возникает:** Если деструктор пытается корректно сохранить данные на диск или закрыть сетевой сокет (вызывая метод `.close()` или сбрасывая буфер `flush()`), а в этот момент физически отключился диск, переполнилась файловая система или пропала сеть.

3. Исключения из деструкторов внутренних полей (Объектов-компонентов)

- **Как возникает:** Если ваш класс содержит внутри себя объекты других классов, то при вызове деструктора вашего класса автоматически (в обратном порядке) начнут вызываться деструкторы этих внутренних объектов. Если деструктор хотя бы одного внутреннего поля выбросит исключение, оно полетит вверх через деструктор вашего класса.

Гарантии безопасности исключений (базовая, строгая, отсутствие).:

В C++ выделяют три уровня гарантий безопасности кода при возникновении исключений:

- **базовый** – при возникновении исключения не возникает утечек ресурсов, однако объекты могут находиться в непредсказуемом состоянии;
- **строгий** – если во время операции произошло исключение, то объект будет находиться в том же состоянии, что до начала операции;
- **без исключений** – в данном коде не может возникнуть исключений.

В примере выше — указана **базовая гарантия**

10. Умные указатели (Smart Pointers) – C++11 и новее

Проблемы с сырыми указателями: утечки памяти, висячие ссылки:

Утечкой памяти называется ситуация, когда в программе нет ни одного указателя, хранящего адрес объекта, созданного в динамической памяти.

Висячей ссылкой называется указатель, ссылающийся на уже удалённый объект.

Умные указатели:

Smart pointer — это объект, работать с которым можно **как с обычным** указателем, но в отличие от последнего, он предоставляет некоторый **дополнительный функционал** (например, автоматическое **освобождение** закреплённой за указателем области **памяти**).

ИЛИ

Smart pointer — класс (обычно шаблонный), имитирующий интерфейс обычного указателя и добавляющий некую новую функциональность, например проверку границ при доступе или очистку памяти.

Предназначение:

Умные указатели призваны для борьбы с утечками памяти, которые сложно избежать в больших проектах.

Они особенно удобны в местах, где возникают исключения, так как при последних происходит процесс раскрутки стека и уничтожаются локальные объекты.

В случае обычного указателя — уничтожится переменная-указатель, при этом ресурс останется не освобожденным.

В случае умного указателя — вызовется деструктор, который и освободит выделенный ресурс.

Семантика владения:

Чаще всего умный указатель **инкапсулирует семантику владения ресурсом**. В таком случае он называется **владеющим** указателем.

Владеющие указатели применяются для борьбы с утечками памяти и висячими ссылками.

Семантика владения для динамически созданных объектов означает, что удаление или присвоение нового значения указателю будет **согласовано с временем жизни** объекта

Виды умных указателей:

- `unique_ptr`
- `shared_ptr`
- `weak_ptr`

Все они объявлены в заголовочном файле `<memory>`

`std::unique_ptr`: эксклюзивное владение, запрет копирования, перемещение через `std::move`:

Этот указатель пришел на смену старому и проблематичному `auto_ptr`.

Основная проблема `auto_ptr` заключается в **правах владения**.

Объект этого класса теряет права владения ресурсом при копировании (присваивании, использовании в конструкторе копий, передаче в функцию по значению).

unique_ptr

В отличие от `auto_ptr`, `unique_ptr` **запрещает копирование**

```
std::unique_ptr<int> x_ptr(new int(42));  
std::unique_ptr<int> y_ptr;
```

```
// ошибка компиляции  
y_ptr = x_ptr;
```

```
// ошибка компиляции  
std::unique_ptr<int> z_ptr(x_ptr);
```

Изменение прав владения

Изменение прав владения ресурсом осуществляется с помощью вспомогательной функции `std::move` (которая является частью механизма перемещения).

```
std::unique_ptr<int> x_ptr(new int(42));  
std::unique_ptr<int> y_ptr;
```

```
// также, как и в случае с auto_ptr, права владения переходят к y_ptr, а x_ptr начинает указывать на  
// null pointer  
y_ptr = std::move(x_ptr);
```

Методы unique_ptr

Как auto_ptr, так и unique_ptr обладают методами:

- reset(), который сбрасывает права владения,
- get(), который возвращает сырой (классический) указатель.

```
std::unique_ptr<Foo> ptr = std::unique_ptr<Foo>(new Foo);
```

```
// получаем классический указатель
Foo *foo = ptr.get();
foo->bar();
```

```
// сбрасываем права владения
ptr.reset();
```

unique_ptr против auto_ptr:

unique_ptr недалеко ушел от своего предшественника в плане удобства использования, но, во всяком случае, он обезопасил от неявных смен прав владений ресурсом.

std::shared_ptr: подсчёт ссылок, совместное владение:

Это самый популярный и самый широко используемый умный указатель.

Он начал своё развитие как часть библиотеки boost.

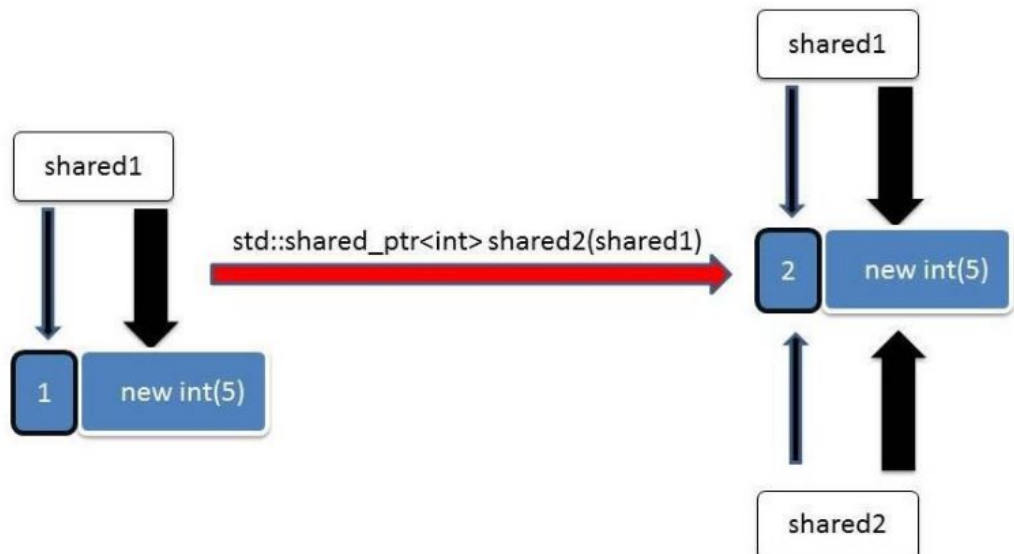
Данный указатель был столь успешным, что его включили в C++ Technical Report 1 и он был доступен в пространстве имен tr1 — std::tr1::shared_ptr<>.

Главное отличие:

В отличие от рассмотренных выше указателей, shared_ptr реализует подсчет ссылок на ресурс.

Ресурс освободится тогда, когда счетчик ссылок на него будет равен 0.

Как видно, система реализует **одно из основных правил сборщика мусора**.



Пример

```
std::shared_ptr<int> x_ptr(new int(42));
std::shared_ptr<int> y_ptr(new int(13));

// после выполнения данной строчки,

y_ptr = x_ptr;

// ресурс на который указывал ранее y_ptr (int(13)) освободится,
// а на int(42) будут ссылаться оба указателя

std::cout << *x_ptr << "\t" << *y_ptr << std::endl;

// int(42) освободится лишь при уничтожении последнего ссылающегося на него указателя
```

Методы

Также как и `unique_ptr`, и `auto_ptr`, данный класс предоставляет методы `get()` и `reset()`.

```
auto ptr = std::make_shared<Foo>(new Foo);

Foo *foo = ptr.get();
foo->bar();

ptr.reset();
```

Пример

```
#include <memory>
#include <iostream>

int test() {
    std::shared_ptr<MyObject> p1(new MyObject);
    std::shared_ptr<MyObject> p2;
    p2 = p1;
    if (p2)
        std::cout << "Hello, world!\n";
}
```

Теперь и p2 и p1 указывают на один объект, а счетчик ссылок равен 2. Когда счетчик обнуляется, объект уничтожается.

Передача как параметра

Мы можем передавать этот указатель в функцию:

```
int test(std::shared_ptr<MyObject> p1) {

    // Делаем что-то

}
```

Если передать указатель по ссылке, то счетчик не будет увеличен. Вы должны быть уверены, что объект MyObject будет жив, пока будет выполняться функция test.

Опасности:

При работе с умным указателем, следует опасаться их создания на лету.

Например, следующий код может привести к утечке памяти.

```
someFunction(std::shared_ptr<Foo>(new Foo), getRandomKey());
```

В чём же проблема?

Стандарт C++ не определяет порядок вычисления аргументов.

Может случиться так, что сначала выполнится `new Foo`, затем `getRandomKey()` и лишь затем конструктор `shared_ptr`.

```
someFunction(std::shared_ptr<Foo>(new Foo), getRandomKey());
```

если 3 1 2

Если же функция `getRandomKey()` бросит исключение, до конструктора `shared_ptr` дело не дойдет, хотя ресурс (объект `Foo`) был уже выделен.

Выход из ситуации следует в тему про фабричные функции)

Фабричные функции: `std::make_shared`, `std::make_unique`:

Выход есть

Использовать **фабричную** функцию `std::make_shared<>`, которая создает объект заданного типа и возвращает `shared_ptr` указывающий на него.

```
someFunction(std::make_shared<Foo>(), getRandomKey());
```

`make_shared` возвращает `shared_ptr`, и он является временным объектом

Стандарт C++ четко декларирует, что временные объекты уничтожаются, в случае появления исключения.

`new Foo` тоже возвращает временный объект

Однако, временным является указатель на выделенный ресурс, и в случае исключения — уничтожится указатель, при этом ресурс останется выделенным.

Когда использовать `std::shared_ptr`, а когда `std::unique_ptr`:

Если объект нужен только в одном месте, то используйте `std::unique_ptr` (чтобы защититься от непреднамеренного копирования).

Если объект понадобился в нескольких местах, то — `std::shared_ptr`.

Более глубокий анализ:

`std::unique_ptr` по своей эффективности очень близок к обычным указателям

Чего нельзя сказать о `std::shared_ptr`.

Он предоставляет больше возможностей, но за все приходится платить.

Увеличивается и расход памяти, и время доступа.

Однако накладные расходы не столь существенны, поэтому в большинстве приложений разница окажется незаметной.

Итак, smart pointers это хорошо, но есть и минусы

- небольшой оверхед (overhead)
- boilerplate-код

например,

```
shared_ptr<MyNamespace::Object> ptr = shared_ptr<MyNamespace::Object>(new MyNamespace::Object(param1, param2, param3))
```

Это частично можно решить при помощи `define` либо при помощи `typedef`

- проблема циклических ссылок

Проблема перекрёстных ссылок

```
struct Widget {  
    shared_ptr<Widget> otherWidget;  
};
```

```
void foo() {  
    shared_ptr<Widget> a(new Widget);  
    shared_ptr<Widget> b(new Widget);
```

```
    a->otherWidget = b;  
    // В этой точке у второго объекта счетчик ссылок = 2
```

```
    b->otherWidget = a;  
    // В этой точке у обоих объектов счетчик ссылок = 2  
}
```

В деструкторе уменьшатся ссылки на объекты.

У каждого объекта будет счетчик = 1 (ведь а все еще указывает на b, а b — на a).

Объекты держат друг друга и у приложения нет возможности получить к ним доступ — эти объекты потеряны.



Ещё не всё:

Представим себе случай, когда нам нужно передать указатель, не передавая права владения объектом.

Использовать для этой цели `shared_ptr` нельзя, т.к. его копирование увеличит счётчик ссылок, и в результате происходит разделение прав владения.

Использование обычного указателя также нежелательно, т.к., как уже говорилось в самом начале, невозможно проверить, ссылается указатель на существующий объект или нет.

Решение проблемы:

Специально для такого случая предусмотрен ещё один вид умных указателей: `weak_ptr`.

Этот указатель также как и `shared_ptr`, начал своё рождение в проекте boost

`std::weak_ptr`: предотвращение циклических ссылок (`shared_ptr`), метод `lock()`:

`weak_ptr` — это «слабый», не владеющий экземпляром объекта умный указатель

`weak_ptr` ссылается на экземпляр, которым владеет `shared_ptr`, но не разделяет прав владения этим экземпляром

Это гарантирует, что если экземпляр уже уничтожен, мы сможем это надёжно и безопасно проверить

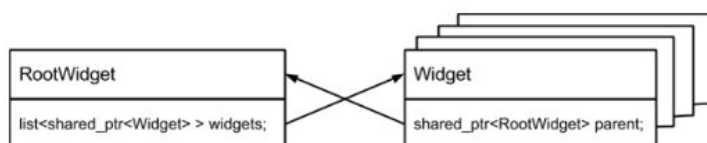
Случай, когда один объект владеет коллекцией других объектов

Одним из типичных случаев создания перекрестных ссылок является случай, когда один объект владеет коллекцией других объектов

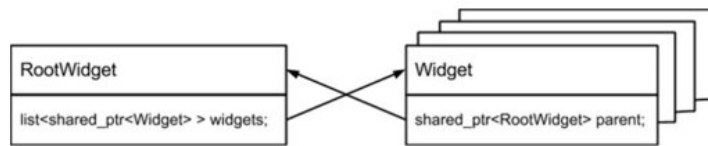
```
struct RootWidget {  
    list<shared_ptr<class Widget> > widgets;  
};
```

```
struct Widget {  
    shared_ptr<class RootWidget> parent;  
};
```

При таком устройстве каждый `Widget` будет препятствовать удалению `RootWidget` и наоборот.



Решение проблемы перекрестных ссылок



В таком случае нужно ответить на вопрос: «Кто кем владеет?».

Очевидно, что именно `RootWidget` в данном случае владеет объектами `Widget`, а не наоборот.

Поэтому модифицировать пример нужно так:

```
struct Widget {  
    weak_ptr<class RootWidget> parent;  
};
```

Слабые ссылки не препятствуют удалению объекта.

В случае возникновения в коде кольцевых ссылок, используйте `weak_ptr` для решения проблем.

Преобразование слабых ссылок

Слабые ссылки могут быть преобразованы в сильные двумя способами

1) Конструктор `shared_ptr`

```
weak_ptr<Widget> w = ...;
```

// В случае, если объект уже удален, в конструкторе `shared_ptr` будет сгенерировано исключение

```
shared_ptr<Widget> p( w );
```

2) Метод `lock`

```
weak_ptr<Widget> w = ...; // В случае, если объект уже удален, то p будет пустым указателем
```

```
if( shared_ptr<Widget> p = w.lock() ) {
```

```
// Объект не был удален – с ним можно работать
```

```
}
```

Проблема `std::auto_ptr` (deprecated):

- Реализует стратегию «одного владельца»
- При копировании владелец передается

Что означает?

```
Picture* a = new Picture();
Picture* b = a;

std::auto_ptr<int> x_ptr(new int(5));
std::auto_ptr<int> y_ptr;

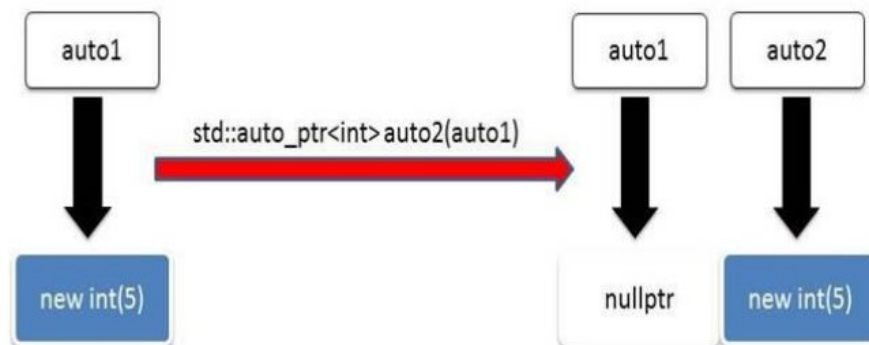
// вот этот нехороший и неявный момент

y_ptr = x_ptr;

// права владения ресурсов уходят в y_ptr и x_ptr начинает указывать на null pointer

// после этого x_ptr перестает указывать на объект

std::cout << *x_ptr << std::endl;    // error
```



Неудобства при работе с контейнером:

Это очень неудобно, при работе с контейнером из умных указателей.

Банальное

```
std::vector<std::auto_ptr<int> > vec;
```

```
// ...
```

```
std::auto_ptr<int> tmp = vec[0];
```

сделает элемент вектора невалидным.

11. Отношения между классами:

Наследование ("is-a"): - смотреть в 5-ой главе

Включение/композиция ("has-a"):

Каждый ресурс, под который выделяется память в конструкторе, обычно стремятся обернуть объектом класса, контролирующим этот ресурс, что упрощает код

В этом случае между классами возникает не отношение наследование, а отношение включения («has-a»). Оно означает, что один класс содержит в качестве члена объект другого/других классов

Закрытое наследование ("as-a" или "реализовано через"): читать в 5-ой главе

12. STL (Standard Template Library или Стандартная библиотека шаблонов):

Обобщённый подход:

- **Обобщённое программирование (generic programming)** заключается в описании структур данных и алгоритмов в терминах типов, подставляемых в качестве параметров в эти алгоритмы и структуры
- Обобщённое программирование является одним из проявлений полиморфизма
- Обобщённый подход в языке C++ основывается на шаблонах функций и шаблонах классов

Шесть основных видов компонентов STL

- **контейнеры** — объекты, которые хранят коллекции других объектов
- **итераторы** — объекты доступа к содержимому контейнеров; они исполняют роль посредников между контейнерами и алгоритмами
- **алгоритмы** — позволяют организовать обработку объектов вне зависимости от вида их организации в контейнерах

- **адаптеры** — используются для изменения интерфейсов других компонентов STL
- **функторы** (функциональные объекты) — объекты, каждый из которых скрывает функцию с целью передачи её в качестве параметра в алгоритмы
- **аллокаторы** — используются контейнерами для выделения памяти объектам контейнеров и её освобождения

Главное об обобщённых алгоритмах STL:

Поскольку они **могут использоваться со многими или даже со всеми контейнерами**, отпадает необходимость в определении соответствующих функций-членов у отдельных контейнеров, что **снижает размер кода и упрощает интерфейсы** контейнеров. Тем не менее, если эффективность алгоритма сильно зависит от внутренней реализации, он обычно реализуется как **функция-член шаблона класса**

Нет наследованию в шаблонах:

Разработчики отказались от использования наследования при реализации библиотеки STL

В частности, деструкторы в шаблонах классов не являются виртуальными

В связи с этим **не рекомендуется разрабатывать собственные шаблоны контейнеров через наследование шаблонов STL**

Общая характеристика STL:

Вся библиотека состоит **только из заголовочных файлов**

В контейнерах можно использовать любые типы данных

Поскольку **алгоритмы работают через итераторы**, они могут быть применены к **любым контейнерам, допускающим использование соответствующих итераторов**. Этим обеспечивается **универсальность**

Однако шаблоны приводят к **большему времени компиляции**

Хотя **исполняемый код** может получиться **более эффективным**

Использование библиотеки STL **увеличивает скорость написания программ**, поскольку предоставляет готовые решения по организации данных и реализации алгоритмов

Это позволяет создавать **компактный код** программы

С другой стороны это влечёт **сложность отладки из-за тяжёлых сообщений об ошибках**

Если коротко

- 1.Компактность
- 2.Эффективность в ущерб безопасности
- 3.Не используются наследование и полиморфизм для эффективности
- 4.Универсальность
- 5.0 байт в откомпилированном виде т.к. вся библиотека состоит из заголовочных файлов
- 6.Тяжелые сообщения об ошибках

Контейнеры:

Коллекция или контейнер — объект программы, содержащий в себе набор значений одного или различных типов и позволяющий обращаться к этим значениям

//одно из определений на выбор

Контейнер — это набор однотипных элементов, реализующий абстрактный тип данных. Простейшим прототипом контейнера в классическом языке C++ является массив

Назначение коллекции — служить **хранилищем** программных объектов и **обеспечивать доступ к ним**.

Обычно коллекции используются для хранения групп **однотипных объектов**, подлежащих **стереотипной обработке**.

Для **обращения** к конкретному **элементу** коллекции могут использоваться различные функции, **в зависимости от логической организации коллекции**.

Реализация может допускать выполнение отдельных **операций над коллекциями в целом**. Наличие операций над коллекциями во многих случаях может существенно упростить программирование.

STL вводит целый ряд разнообразных типов контейнеров:

- последовательные
- ассоциативные
- контейнеры-адаптеры
- псевдоконтейнеры

Какой контейнер выбрать?

Эффективность операций для различных контейнеров разная и зависит от внутреннего представления контейнера

Поэтому для каждой конкретной задачи нужно выбирать наиболее подходящий контейнер, исходя из того, какие именно операции при решении задачи будут использоваться чаще всего

Если предполагается большое количество операций удаления/вставки в произвольных местах контейнера, то рекомендуется использовать двунаправленный список `list` или однонаправленный `forward_list`.

Последовательные: `vector`, `list`, `deque`, `array`:

- `vector`
- `array`
- `list`
- `forward_list`
- `deque`

Сохраняется тот порядок, в котором элементы добавляются в контейнер, т.е. в них организуется хранение элементов в линейном порядке

`vector` - является реализацией динамического массива

`array` - та же семантика, что и у массивов фиксированного размера

`list` - двунаправленный список

`forward_list` - однонаправленный список

`deque` - двусторонняя очередь

`vector`:

Использование

в задачах, требующих доступ к элементу по индексу

Стоимость операций

- операции вставки и удаления в конце вектора выполняются за постоянное время
- вставка и удаление внутри вектора имеют линейную сложность

`array`:

начиная с версии C++11, с целью повышения эффективности в случае работы со статическим массивом

нетиповой параметр шаблона, позволяющий выделить память на этапе компиляции

```
array<int, 4> Myarray;
```

```
array<int, 4> Myarray1 = {5,6,7,8};
```

размер и эффективность `array<T,N>` такие же, как у статического массива `T[N]` в отличие от остальных контейнеров, `array` не позволяет управлять размещением элементов через аллокаторы, и для него не определены операции, изменяющие размер массива

предоставляет некоторые возможности стандартных контейнеров, такие как:

- знание собственного размера,
- поддержка присваивания,
- итераторы произвольного доступа и т.д.

Итераторы для `vector` и `array`:

Контейнеры `vector` и `array` поддерживают итераторы **произвольного доступа**:

- как **прямой**,
- так и **обратный**

`deque`:

позволяет **быстро** выполнять операции **вставки/удаления** элементов **в начале и в конце** контейнера

в отличие от `vector` и `array` контейнер `deque` обычно реализован с помощью **двустороннего списка массивов фиксированного размера**

расширение `deque` дешевле, чем расширение `vector`, потому что оно не требует копирования существующих элементов в новый участок памяти

Стоимость операций

вставки/удаления в начале и конце двусторонней очереди — постоянная

произвольного доступа — постоянная

вставки/удаления внутри контейнера — линейная

Доступ к элементам контейнеров `vector`, `array` и `deque` по индексу реализован двумя способами:

- перегруженной операцией `operator`
- функцией `at(i)`

Обе реализации подобны.

Отличие заключается лишь в том, что функция `at(i)` контролирует выход за границу массива, выбрасывая в случае ошибки исключение `out_of_range`.

Вектор — резюме:

- Вектор – это контейнер, который является последовательным, т.е. организует **хранение** элементов в **линейном** порядке
- Вектор поддерживает **итераторы произвольного** доступа
- Операции вставки и удаления в конце вектора выполняются за постоянное время
- Операции вставки и удаления внутри вектора имеют линейную сложность
- Управление памятью выполняется автоматически

Контейнер `list<T>`:

Предоставляет двунаправленные итераторы, так что его можно использовать с алгоритмом **reverse**

```
list<int> listl;
```

```
// Код для вставки значений в listl
```

```
reverse(begin(listl),end(listl)); // Обращение порядка значений в списке
```

Возможность предоставления контейнером двунаправленного итератора зависит от внутренней организации контейнера.

Контейнер `list<T>` реализован как двусвязный список, поэтому предоставляет возможность итератору реализовать операцию `operator--` за константное время.

В случае односвязного списка эффективно реализовать `operator--` (с константным временем работы) невозможно

Ассоциативные: `set`, `multiset`, `map`, `multimap`:

- `map<K, V>` — по сути, бинарное дерево поиска, то есть скорость поиска $\log(n)$
- `set<K>` — множество, так же является бинарным деревом поиска
- `multimap<K, V>` — `map` с повторяющимися ключами

- `multiset<K>` — set с повторяющимися значениями
- `unordered_map<K, V>` — хеш-таблица $O(1)$
- `unordered_set<K>` — хеш-таблица
- `unordered_multiset<K>` — хеш-таблица
- `unordered_multimap <K,V>` — хеш-таблица

Плюсы и минусы данной категории контейнеров:

Важно, чтобы базовые операции для ассоциативных контейнеров:

- вставка `insert`,
- удаление `erase`,
- поиск `find`

выполнялись как в среднем, так и в худшем случае за время $O(\log n)$.

//далее к чему-то шло сравнение сбалансированного дерева и хэш-таблиц, поэтому я это пропустил, нро это упоминалось в 10 лекции слайд 4

Ассоциативные контейнеры **обладают многими свойствами**, присущими **последовательным** контейнерам, поскольку они **поддерживают обход элементов** данных **в виде линейной последовательности**.

Они предоставляют **двунаправленные итераторы**, обход с использованием которых даёт отсортированный порядок элементов в случае с упорядоченными контейнерами.

//Немного подробностей, связанных с `set`

Операции сравнения:

Множество `set` не должно содержать повторяющихся значений.

Тем не менее, реализовывать операцию сравнения ключей на равенство не требуется, поскольку **проверка на совпадение реализуется через две операции `operator<`**.

Определение эквивалентности $x == y$ сводится к выражению

$!(x < y) \& \& !(y < x)$ или $!((x < y) || (y < x))$

Компараторы:

Бывают случаи, когда необходимы **сравнения по разным ключам**, а операция `operator<` реализует только одно сравнение.

В этих случаях есть возможность **определить класс для сравнения** значений пользовательского типа.

Такие классы называются **компараторами**.

Функция-член **insert** классов **set** и **multiset** получает единственный аргумент и **вставляет копию этого аргумента**.

В случае **multiset** элемент всегда вставляется и **результатом вставки** является **позиция** итератора **iterator**, указывающая во множестве на элемент с заданным значением ключа.

Для класса **set** дополнительно указывается, **был ли уже** этот элемент во множестве **или** он был **добавлен** в текущей операции. Поэтому в классе **set** **тип возвращаемого значения** имеет вид **pair<iterator, bool>**.

Поскольку при обращении по индексу возможно неявное добавление элемента в **map** с указанием только ключа, то **для типа ассоциированного значения необходим конструктор по умолчанию**.

//Доп тема из Gemini

1. std::set и std::multiset (Множества)

Хранят только уникальные ключи (в случае **set**) или дублирующиеся ключи (в случае **multiset**), автоматически сортируя их по возрастанию (используя **operator<**).

- Главное отличие **multiset**: Так как элементы могут дублироваться, метод **insert()** в **multiset** всегда возвращает просто итератор на вставленный элемент. В обычном **set** метод **insert()** возвращает **std::pair<iterator, bool>**, где **bool** показывает, произошла ли вставка или элемент уже был в множестве.

Полезные методы для поиска в условиях дубликатов (**multiset**):

- **count(key)** — возвращает количество элементов с таким ключом (в **set** вернет 0 или 1, в **multiset** — реальное количество).
- **lower_bound(key)** — возвращает итератор на первый элемент, который $\geq$ **key**.
- **upper_bound(key)** — возвращает итератор на первый элемент, который $>$ **key**.
- **equal_range(key)** — возвращает пару итераторов **std::pair** (начало и конец диапазона одинаковых элементов).

//пример

```
#include <iostream>
```

```
#include <set>
```

```
int main() {
```

```

std::multiset<int> ms = {1, 2, 2, 2, 3, 4};

// Находим диапазон всех двоек
auto range = ms.equal_range(2);

// Удаление конкретного диапазона дубликатов
ms.erase(range.first, range.second);

// Внимание: если написать ms.erase(2), то удалятся ВСЕ двойки сразу.
}

```

2. std::map и std::multimap (Словари / Ассоциативные массивы)

Хранят пары «ключ-значение» (std::pair<const Key, Value>), отсортированные по ключу.

Критически важные отличия между map и multimap:

1. **Уникальность ключей:** В map ключи строго уникальны. В multimap может быть несколько пар с абсолютно одинаковыми ключами, но разными значениями.
2. **Отсутствие operator[] в multimap:** В std::multimap **не существует оператора квадратных скобок [] и метода .at()**. *Обоснование для экзамена:* Если у вас есть три элемента с ключом "ID_1", компилятор не сможет определить, к какому именно значению вы хотите обратиться по индексу multimap["ID_1"].

Как работать с std::multimap: Добавление элементов происходит только через insert с явным созданием пары, а получение значений — через перебор диапазона equal_range

```

//пример
#include <iostream>
#include <map>
#include <string>

int main() {
    // Пример: Студент и предметы, по которым у него долги
    std::multimap<std::string, std::string> debts;

    // Вставка элементов (перегрузки [] нет!)
    debts.insert(std::make_pair("Иванов", "Матанализ"));
}

```

```

debts.insert({"Иванов", "Линейная алгебра"}); // C++11 стиль
debts.insert({"Петров", "Физика"});

// Поиск всех долгов студента "Иванов"
std::string student = "Иванов";
auto range = debts.equal_range(student);

std::cout << "Долги студента " << student << ":" << std::endl;
for (auto it = range.first; it != range.second; ++it) {
    std::cout << " - " << it->second << std::endl; // it->first — это ключ, it->second — значение
}
}

```

Неупорядоченные: `unordered_set`, `unordered_map` (брал из Gemini) + это скорее, как доп. тема, т.к в лекциях особо не освещалась:

В отличие от `std::set` и `std::map`, которые внутри построены на базе **сбалансированных бинарных деревьев поиска** (красно-черных деревьев) и поддерживают элементы в отсортированном порядке, неупорядоченные контейнеры используют структуру данных **Хеш-таблица**.

1. Внутреннее устройство и принцип работы

- **Хеш-функция:** Каждый элемент (или ключ) преобразуется хеш-функцией в целочисленный индекс. По этому индексу элемент помещается в определенную ячейку памяти — **корзину (bucket)**.
- **Разрешение коллизий:** Ситуация, когда разные ключи выдают один и тот же хеш-индекс, называется *коллизией*. В STL для разрешения коллизий применяется метод цепочек (связанные списки внутри каждой корзины).
- **Load Factor (Коэффициент загрузки):** Это отношение числа элементов к числу корзин. Если оно превышает 1.0 (или заданный `max_load_factor`), происходит **рехеширование (rehash)** — увеличение количества корзин и перестройка всей таблицы, что временно требует времени $O(n)$.

2. Сложность операций (Временная сложность)

В методичке для `set/map` указано время $O(\log n)$ как для среднего, так и для худшего случая. Для неупорядоченных контейнеров ситуация иная:

2. Сложность операций (Временная сложность)

В методичке для `set` / `map` указано время $O(\log n)$ как для среднего, так и для худшего случая. Для неупорядоченных контейнеров ситуация иная: `DOCX`

Операция	В среднем (Average case)	В худшем случае (Worst case)
Поиск (<code>find</code>)	$O(1)$ — константное время	$O(n)$ — линейное (если все элементы попали в одну корзину)
Вставка (<code>insert</code>)	$O(1)$	$O(n)$ (при плохой хеш-функции или во время <i>rehash</i>)
Удаление (<code>erase</code>)	$O(1)$	$O(n)$

3. Главные отличия: Упорядоченные vs Неупорядоченные

Критерий	<code>std::set</code> / <code>std::map</code> <code>DOCX</code>	<code>std::unordered_set</code> / <code>std::unordered_map</code> <code>DOCX</code>
Структура	Сбалансированное дерево <code>DOCX</code>	Хеш-таблица <code>DOCX</code>
Порядок элементов	Всегда отсортированы (по возрастанию / компаратору)	Произвольный (зависит от хеш-функции)
Итераторы	Двунаправленные (<code>BidirectionalIterator</code>) <code>DOCX</code>	Однонаправленные (<code>ForwardIterator</code>)
Требования к ключу	Необходим <code>operator<</code> (или компаратор) <code>DOCX + 1</code>	Необходима хеш-функция (<code>std::hash</code>) и <code>operator==</code>

4. Особенности работы с `std::unordered_set`

Используется, когда нужно быстро проверять наличие уникальных элементов в коллекции, а их порядок не важен.

5. Особенности работы с `std::unordered_map`

Хранит пары «ключ-значение» (`std::pair<const Key, Value>`). В отличие от обычного `map`, работает быстрее, но тратит больше оперативной памяти под структуру корзин.

Важно (Аналогия с методичкой): Так же, как и для `std::map`, при обращении по индексу через `operator[]`, если ключа нет в таблице, элемент **неявно создается** со значением по умолчанию. Поэтому для константных контейнеров используйте метод `.at()`.

6. Требования к пользовательским типам (Для экзамена/лабораторных)

Если для встроенных типов (`int`, `double`, `std::string`) в STL уже написаны хеш-функции, то для собственного класса (например, структуры `Student` из вашей методички) неупорядоченный контейнер «из коробки» работать не будет.

Чтобы использовать свой тип в качестве ключа, компилятору нужно предоставить две вещи:

1. **Перегруженный `operator==`** (чтобы различать объекты при коллизии).
2. **Специализацию шаблона `std::hash<T>`** (чтобы вычислять хеш-код).

Адаптеры контейнеров: `stack`, `queue`, `priority_queue`:

ограничивают интерфейс базовых контейнеров —

- `queue`
- `priority_queue`
- `stack`

Мощь STL заключается в том, что **алгоритмы** ничего не знают про **контейнеры**, а контейнеры ничего не знают про алгоритмы.

Для того, чтобы склеить контейнеры и алгоритмы в STL существуют **итераторы**.

Адаптеры итераторов:

Адаптеры — шаблонные классы, которые обеспечивают изменение поведения при сохранении интерфейса.

Например, для двунаправленных итераторов и итераторов произвольного доступа существуют адаптеры — **обратные (реверсивные) итераторы** — `reverse_iterator`.

Они переопределяют операции увеличения (уменьшения) таким образом, что перемещение происходит в противоположном направлении. Для инициализации реверсивных итераторов используются операции `rbegin()` и `rend()`.

//Также немного доп.тема,брал из Gemini

1. std::stack (Стек)

- **Принцип работы: LIFO** (Last In, First Out — «последним пришел, первым ушел»). Элементы добавляются и удаляются только с одного конца (вершины).
- **Базовый контейнер по умолчанию:** std::deque (может быть заменен на std::vector или std::list).
- **Основные операции ($O(1)$):**
 - push(val) — положить элемент на вершину.
 - pop() — удалить элемент с вершины (в C++ ничего не возвращает!).
 - top() — получить ссылку на элемент на вершине.
 - empty() — проверка на пустоту.
 - size() — размер.

//Пример

```
#include <iostream>
#include <stack>
#include <vector>

int main() {
    // Стек поверх std::vector (вместо стандартного deque)
    std::stack<int, std::vector<int>>> s;
    s.push(10);
    s.push(20);

    std::cout << s.top() << std::endl; // Выведет 20
    s.pop();                          // Удалит 20
    std::cout << s.top() << std::endl; // Выведет 10
}
```

2. std::queue (Очередь)

- **Принцип работы: FIFO** (First In, First Out — «первым пришел, первым ушел»). Классическая очередь: добавление в конец, удаление из начала.
- **Базовый контейнер по умолчанию:** std::deque (может быть заменен на std::list, но не на std::vector, так как у вектора нет эффективной операции удаления из начала pop_front).

- **Основные операции ($O(1)$):**
 - `push(val)` — встать в конец очереди.
 - `pop()` — удалить элемент из начала очереди.
 - `front()` — посмотреть первый элемент.
 - `back()` — посмотреть последний элемент.

3. `std::priority_queue` (Очередь с приоритетом)

- **Принцип работы:** Элементы извлекаются не в порядке очереди, а на основании их **приоритета**. По умолчанию элемент с максимальным значением (наивысшим приоритетом) всегда оказывается на вершине.
- **Внутреннее устройство:** Внутри организована структура данных **Двоичная куча (Binary Heap)**.
- **Базовый контейнер по умолчанию:** `std::vector` (требует произвольного доступа к элементам).
- **Сложность:** Вставка `push()` и удаление `pop()` занимают логарифмическое время $O(\log n)$, а просмотр максимума `top()` — константное $O(1)$.

//Пример

```
#include <iostream>
```

```
#include <queue>
```

```
int main() {
```

```
    // Очередь с приоритетом (по умолчанию на максимум)
```

```
    std::priority_queue<int> pq;
```

```
    pq.push(5);
```

```
    pq.push(12);
```

```
    pq.push(3);
```

```
    while (!pq.empty()) {
```

```
        std::cout << pq.top() << " "; // Выведет элементы по убыванию: 12 5 3
```

```
        pq.pop();
```

```
    }
```

```
}
```

Доп.тема: Требования к контейнерам, которые можно использовать с обобщёнными алгоритмами библиотеки, Требования к типам элементов контейнера:

Требования к типам элементов контейнера:

Как правило, элементы, хранимые в контейнерах STL, могут быть практически **любого типа**, если их **можно копировать**, т.е. **имеют семантику значения**.

Если необходимо использовать собственный тип, то для него должны быть обязательно определены:

- конструктор без параметров,
- конструктор копии,
- деструктор,
- операция присваивания.

Дополнительно в некоторых случаях требуется определить `operator==` и `operator<`.

Например, для каких-то типов контейнеров или для выполнения некоторых операций с контейнерами или их элементами.

Требования к контейнерам, которые можно использовать с обобщёнными алгоритмами библиотеки:

Должны иметь:

- конструктор, создающий пустой контейнер
- конструктор копирования
- деструктор, который освобождает всю память, использованную контейнером, и вызывает деструктор каждого элемента в контейнере (деструктор контейнера не виртуален)

Возможно:

- конструктор с параметрами – итераторами, задающими диапазон элементов в другом контейнере
- частные виды конструкторов в зависимости от типа контейнера

//Общий вид конструктора для контейнера с элементами типа T

// создание пустого контейнера

`container<T> c;`

```
// создание копии контейнера
```

```
container<T> c2(c);
```

```
// создание контейнера с инициализацией элементами из диапазона [b,e) другого контейнера
```

```
// диапазон задается через итераторы
```

```
container<T> c3(b,e);
```

Итераторы: понятие, категории (ввода, вывода, однонаправленные, двунаправленные, произвольного доступа).:

Итератор – объект, позволяющий программисту перебирать все элементы коллекции без учета особенностей ее реализации. Он реализует доступ к элементам коллекции

Использование итераторов в обобщённом программировании позволяет реализовать универсальные алгоритмы работы с контейнерами.

```
//определение на выбор
```

Итератор — это любой тип, к которому применимы операции:

- продвижение: ++
- обращение к элементу, на который указывает итератор: *
- проверка равенства (==) и неравенства (!=)

```
//еще одно определение на выбор
```

Итераторы — посредники между контейнерами и обобщенными алгоритмами

Однако по причинам эффективности **невозможно** обеспечить возможность работы **каждого** обобщённого **алгоритма с каждым контейнером**. Чтобы определить **применимость алгоритма к контейнеру**, необходимо знать **тип итератора**, удовлетворяющего минимальным требованиям алгоритма, и тип итератора, предоставляемый контейнером

Примерами коллекций (контейнеров), по которым может перемещаться итератор, могут быть:

- список,
- очередь,
- множество,
- ассоциативный массив
- и др.

Итератор и указатель:

Итератор похож на указатель своими основными операциями:

- указание одного отдельного элемента в коллекции объектов (доступ к элементу);
- изменение своего значения так, чтобы указывать на следующий элемент (перебор элементов).

Также должны быть определены

- способ создания итератора, указывающего на первый элемент коллекции;
- способ узнать, достигнут ли конец коллекции.

В зависимости от используемого языка и цели, итераторы могут поддерживать дополнительные операции или определять различные варианты поведения.

Предназначение итераторов:

Главное предназначение итераторов заключается в предоставлении возможности пользователю **обращаться к любому элементу контейнера при сокрытии внутренней структуры контейнера** от пользователя

Это позволяет контейнеру **хранить элементы любым способом** при допустимости **работы** пользователя с ним **как с простой последовательностью** или списком

Проектирование класса итератора тесно связано с соответствующим классом контейнера

Обычно **контейнер предоставляет функции создания итераторов**

Возможности итераторов:

- Указание одного отдельного элемента в коллекции объектов (доступ к элементу)
- Изменение своего значения так, чтобы указывать на следующий/предыдущий элемент (перебор элементов)
- Могут поддерживать дополнительные операции или определять различные варианты поведения

Встроенные итераторы:

В тех случаях, когда коллекция представляет собой структуру, основанную на указателях, итератор может быть реализован в виде дополнительного поля – текущего указателя.

НЕДОСТАТКИ

- В этом случае следует заботиться о его состоянии при любых модификациях коллекции, даже если его не придётся использовать.
- Кроме того, такой встроенный итератор обычно только один, а могут

возникнуть задачи, в которых потребуется наличие двух, трёх или даже большего количества итераторов.

```
struct el {  
    int item;  
    el* next;  
};  
class list {  
private:  
    el* head  
    el* tail;  
    el * cur;  
    ...  
};
```

Итераторы-объекты:

Итераторы-объекты имеют **преимущество** по сравнению со встроенным текущим указателем, так как позволяют создавать для одной коллекции **любое количество** итераторов, действующих независимо.

Таким образом, итератор позволяет **вынести** рабочий **указатель** за **пределы коллекции**, но при этом **дает возможность перемещаться** по объектам, содержащимся в коллекции, аналогично тому, как текущий указатель позволяет перемещаться внутри коллекции.

Разновидности итераторов(Иерархия категорий):

- 1.однонаправленные;
- 2.обратные (реверсные);
- 3.двунаправленные итераторы;
- 4.итераторы ввода и вывода;
- 5.константные итераторы (защищающие контейнер или его элементы от изменения).

Основные типы итераторов(Направление и модификация):

- итераторы ввода
- итераторы вывода

- однонаправленные (последовательные) итераторы
- двунаправленные итераторы
- итераторы произвольного доступа

Перегруженные операции для итераторов:

- Операция доступа к элементу коллекции по ссылке. Такая операция может быть реализована путём перегрузки операции разыменования *
- Операции перемещения вперёд и назад по коллекции объектов. Чаще всего для этого перегружаются операции ++ и --
- Операции == и != обычно перегружаются для проверки равенства итераторов

Другие операции над коллекцией:

Через итератор могут быть реализованы другие операции над коллекцией, например:

- вставка элемента после позиции итератора
- удаление элемента в позиции итератора

При реализации таких операций очень важно оговорить правила поведения итератора после выполнения операции.

Допустимые для итераторов операции:

1. ++

Тип итератора определяет набор допустимых для него операций.

Для всех типов итераторов определены операции ++ и *.

Операция ++ имеет одинаковую семантику.

2. *

Операция * для итераторов ввода возвращает значение элементов коллекции, **не допуская их изменение.**

Для итератора **вывода** операция * предназначена для изменения элементов коллекции, **не допуская их прочтение.**

Для остальных типов итераторов операция * позволяет выполнять оба действия над элементами коллекции

//Двунаправленные и произвольного доступа дополнительно имеют операцию --.

3. +, -

Итераторы произвольного доступа, кроме того, имеют операцию смещения на произвольное количество элементов.

Правила для итераторов:

Каждый итератор, имеющий большой набор операций может использоваться вместо более ограниченного в операциях итератора.

Алгоритмы библиотеки STL поддерживают принцип наименьших требований.

Однонаправленные и двунаправленные итераторы:

Однонаправленный итератор:

Это итератор, который **объединяет** свойства **входного и выходного** итераторов, тем самым обеспечивая возможность чтения и записи.

При этом обход элементов контейнера происходит в одном направлении.

Однонаправленные итераторы обладают также возможностью **сохранить** значение итератора и использовать сохраненное значение **для повторного прохода** из той же позиции.

Это позволяет использовать однонаправленные итераторы в **многопроходных** алгоритмах.

Контейнер **forward_list** имеет **однонаправленный** итератор **forward_list::iterator**.

Последовательные контейнеры **vector**, **list**, **deque**, а также **массивы**, имеют **двунаправленные** итераторы, которые **обладают свойствами однонаправленных**.

Поэтому к ним тоже применимы алгоритмы, использующие однонаправленные итераторы.

Двунаправленный итератор:

Аналогичен однонаправленному, кроме того, он допускает обход в обоих направлениях.

То есть, двунаправленные итераторы должны поддерживать все операции однонаправленных итераторов, а кроме них — операцию **--**, делая возможным обход последовательности в обратном направлении.

И префиксный, и постфиксный **operator--** должны выполняться за константное время.

Возможность обхода структуры данных в обратном порядке важна потому, что без неё некоторые алгоритмы не могут эффективно работать.

Например, алгоритм STL **reverse** может использоваться для обращения порядка элементов в последовательности только при наличии двунаправленных итераторов.

Итераторы произвольного доступа:

Должны поддерживать все операции двунаправленных итераторов, плюс нижеперечисленные

- Прибавление и вычитание целых чисел: $r+n$, $n+r$, $r-n$, $r+=n$ и $r-=n$
- Доступ к n -му элементу при помощи выражения $r[n]$, которое означает $*(r+n)$
- Вычитание итераторов $r-s$
- Сравнения $r<s$, $r>s$, $r<=s$ и $r>=s$

Функции `begin()`, `end()`, `rbegin()`, `rend()` + базовые функции, которые контейнеры должны предоставлять и т.д и т.п:

В классе коллекции для использования итератора создаются две функции:

- `iterator begin()` – инициализация итератора ссылкой на первый элемент коллекции;
- `iterator end()` – значение, сообщающее, что итератор достиг конца коллекции.

Или это могут быть внешние функции

Аналогия с диапазонами массивов !!!

Операции над контейнерами:

Все контейнеры должны поддерживать операции присваивания и сравнения (`=`, `==`, `<`, `<=`, `!=`, `>`, `>=`). Операции сравнения применимы только к контейнерам одного типа и выполняются в лексикографическом порядке

Чтобы контейнеры можно было обрабатывать алгоритмами STL, необходимо наличие функций, возвращающих итераторы:

- `iterator begin()` — возвращает итератор, указывающий на первый элемент контейнера
- `const_iterator begin() const` — возвращает константный итератор, указывающий на первый элемент контейнера
- `iterator end()` — возвращает итератор, указывающий на позицию, следующую за последним элементом контейнера
- `const_iterator end() const` — возвращает константный итератор, указывающий на позицию, следующую за последним элементом контейнера

Все контейнеры должны предоставлять ещё ряд функций:

`bool empty() const` — возвращает `true`, если контейнер пуст

`void clear()` — очищает контейнер, делая его размер = 0

size_type max_size() const — возвращает максимальное количество элементов, которое может содержать контейнер

size_type size() const — возвращает количество элементов, в текущий момент хранящихся в контейнере

void swap(ContainerType c) — обменивает между собой содержимое двух контейнеров одного типа

v.push_back(a) — добавление в конец:

имеет постоянную сложность

если вектор заполнен (размер равен ёмкости), выполнение функции push_back автоматически приводит к увеличению ёмкости

если заранее известно количество вводимых элементов, то вместо v.push_back(a) можно использовать операцию обращения по индексу к элементу массива

//пример

```
vector<int> v(10);
for (i=0; i<v.size(); ++i)
    cin>>v[ i ];
```

Обратный (реверсивный) итератор v.rbegin():

```
bool b = true;
auto itb = begin(v);
auto itr = rbegin(v);
while (b && itb < itr.base()) {
    b = (*itb) == (*itr);
    itb++;
    itr++;
}
cout << (b ? "yes" : "no");
```

- Использование обратного итератора **позволяет избежать дополнительной проверки на непустоту** вектора.
- Типы итераторов itb (прямой) и itr (обратный) — **разные**, поэтому прямое сравнение между ними неприменимо.

- Типы итераторов `itb` и `itrb` — разные, поэтому сравнение для них неприменимо. Но **обратный** итератор можно **привести к базовому**, используя функцию **`base()`**

//объявление итераторов

```
auto itb = v.begin(); auto itrb = v.rbegin();
```

```
auto itb = begin(v); auto itrb = rbegin(v);
```

Методы, работающие с указателями и диапазонами:

`a.erase(pt)` удаление элемента по указателю `pt`

`a.erase(p1, p2)` удаление диапазона `[p1, p2)`

```
score.erase(begin(score), begin(score)+2);
```

`a.insert(p,t)` - вставка значения `t` перед указателем `p`

`a.insert(pt, p1,p2)` - вставка диапазона `[p1, p2)` из другого вектора перед итератором `pt`

```
vector<int> old;
```

```
vector<int> new;
```

```
...
```

```
old.insert(old.begin(),3);
```

```
old.insert(old.end(),5);
```

```
new.insert(new.begin(), old.begin(), old.end());
```

```
new.insert(new.end(), old.begin()+1, old.end()-1);
```

Функция `insert(++itb, ' ')` :

выполняет **вставку в позицию перед итератором**, поэтому необходимо сдвинуть итератор `++itb` возвращает итератор на вставленный элемент

Итераторы вставки (`back_inserter`, `front_inserter`, `inserter`):

Существует три типа итераторов вставки, параметризованные типом контейнера:

- `insert_iterator` – использует функцию `insert()` класса `Container`
- `back_insert_iterator` — использует функцию `push_back()`
- `front_insert_iterator` – использует функцию `push_front()`

Объявление итераторов вставки:

Например, если `v` — вектор целых чисел, то объявление для него итератора вставки перед вторым элементом будет выглядеть следующим образом.

```
insert_iterator<vector<int>> > it(v,next(begin(v)));
```

Чтобы было удобнее использовать вводятся шаблоны функций, возвращающие нужные итераторы

- `back_inserter`
- `front_inserter`
- `inserter`

Так для нашего примера объявление будет выглядеть следующим образом

```
auto it = inserter(v, next(begin(v)));
```

//примеры

```
vector<int> v(10); //123
```

```
vector<int> v1;
```

```
copy(begin(v), end(v), back_inserter(v1));
```

```
//123
```

```
vector<int> v(10); //123
```

```
list<int> l1;
```

```
copy(begin(v), end(v), front_inserter(l1));
```

```
//321
```

```
vector<int> v(10); //123
```

```
vector<int> v1; //123
```

```
copy(v.begin(), v.end(), inserter(v1, next(begin(v1))));
```

```
//112323
```

//Примеры использования `insert`

```
vector<int> vecInt(3,100); ///Создаем вектор из 3 элементов и заполняем его значением 100
```

```
vector<int>::iterator it;
```

```
it = begin(vecInt);///Итератор указывает на vec[0]
```

```
vecInt.insert(it,200);///Вектор расширяется теперь до 4 элементов
```

```
///И первым элементом записывается 200
```

```
it = vecInt.begin() + 3;
```

```
vecInt.insert(it,300);///Вектор расширяется теперь до 5 элементов
```

```
///Итератор указывает на 4 элемент (0-элемент+3-элемента)
```

```
///И четвертым элементом записывается 300
```

```
vecInt.insert(it+1,900);///Вектор расширяется теперь до 6 элементов
```

```
///И 5 элементом записывается 900
```

Потоковые итераторы (istream_iterator, ostream_iterator) или итераторы ввода/вывода:

Важной причиной включения итераторов ввода/вывода в STL является возможность **выделения алгоритмов**, которые могут **использоваться с итераторами, связанными с потоками ввода-вывода**.

Такие итераторы предоставляются классами STL **istream_iterator** (для ввода) и **ostream_iterator** (для вывода).

Итераторы ввода:

Итераторы ввода предназначены для получения информации **из последовательности в алгоритм**, при этом значение в последовательности **не может быть изменено**.

Под последовательностью в данном случае понимаются или контейнер или поток ввода.

Итераторы ввода поддерживают операцию ++ и используются **в однократных алгоритмах**.

```
//немного примеров
```

```
Для входных итераторов из a == b не следует ++a == ++b
```

```
Допустим 123456
```

```
istream_iterator<char> a(cin); — указывает на 1
```

```
istream_iterator<char> b=a; — указывает на 1
```

```
++a; — указывает на 2
```

```
++b; — указывает на 3
```

Алгоритмы, работающие с входными итераторами, не должны пытаться скопировать значение итератора и использовать его для повторного прохода по одной и той же позиции.

Тип значения, возвращаемого итератором, не обязан быть ссылочным, поскольку алгоритмы, работающие со входными итераторами, не должны пытаться посредством них выполнять присваивания.

Итераторы вывода:

Выходные итераторы (итераторы вывода) обладают **противоположной входным** итераторам функциональностью. Они позволяют **записывать значения в последовательность**.

Под последовательностью понимается либо контейнер, либо выходной поток.

Как и в случае итераторов ввода, **алгоритмы**, использующие итераторы вывода, должны быть **однопроходными**.

Операции равенства и неравенства для таких итераторов могут быть не определены.

Алгоритмы, использующие итераторы вывода, могут работать с выходными потоками для размещения данных посредством класса `ostream_iterator`, а также с итераторами и указателями вставки.

Единственное корректное применение операции `operator*` к итераторам вывода — в левой части операции присваивания.

Выходной итератор в алгоритме `copy` для вывода символов в поток `cout`

```
copy(text.begin(),text.end(),ostream_iterator<char>(cout,""));
```

Рекомендации по выводу и вводу:

```
vector v(10);
```

```
copy(v.begin(), v.end(), ostream_iterator(cout, " "));
```

```
vector v;
```

```
copy(istream_iterator(cin), istream_iterator(), back_inserter(v));
```

Алгоритмы STL (основные):

Обобщенность алгоритмов:

Определение алгоритмов в терминах итераторов, а не непосредственно структур данных, делает возможной независимость алгоритмов от последних.

Так **алгоритмы** становятся **особенно обобщенными**, способными работать с большим количеством разных структур данных.

Ни одна другая библиотека не способна на такое расширение, как STL.

Алгоритмы STL:

- Алгоритмы представляют собой внешние функции
- Большинство алгоритмов определено в заголовочных файлах `<algorithm>` и `<functional>`

algorithm:

algorithm — заголовочный файл в стандартной библиотеке языка программирования C++, включающий набор **функций** для выполнения **алгоритмических операций** над **контейнерами** и над другими последовательностями

functional: (также будет являться ключевым определением для подглавы std::function)

functional — заголовочный файл в стандартной библиотеке языка программирования C++, предоставляющий набор шаблонов классов для работы с **функциональными объектами**, а также набор **вспомогательных классов** для их **использования в алгоритмах** стандартной библиотеки

Основные группы алгоритмов:

- Алгоритмы можно разбить на следующие группы:
 - Алгоритмы, не модифицирующие последовательность
 - Алгоритмы, модифицирующие последовательность
 - Алгоритмы сортировки

Немодифицирующие: **find, find_if, count, count_if, for_each, search, equal:**

Алгоритм **binary_search**:

Алгоритм `binary_search` на таких контейнерах как **векторы, деки** и **массивы**, имеет наибольшую эффективность — **время работы $O(\log N)$** , т.к. их итераторы являются итераторами **произвольного доступа**.

Однако **требование** алгоритма — наличие **однонаправленного** итератора, позволяет использовать его и для других контейнеров, например, списков.

Но время работы увеличится, т.к. имея указатели на начало и конец списка, единственный способ добраться до элемента посередине — это обойти элементы один за другим.

Это займет время, пропорциональное длине списка.

Алгоритмы, не модифицирующие последовательность

<u>all_of</u>	Test condition on all elements in range
<u>any_of</u>	Test if any element in range fulfills condition
<u>none_of</u>	Test if no elements fulfill condition
<u>for_each</u>	Apply function to range
<u>find</u>	Find value in range
<u>find_if</u>	Find element in range
<u>find_if_not</u>	Find element in range (negative condition)
<u>find_end</u>	Find last subsequence in range
<u>find_first_of</u>	Find element from set in range
<u>adjacent_find</u>	Find equal adjacent elements in range
<u>count</u>	Count appearances of value in range
<u>count_if</u>	Return number of elements in range satisfying condition
<u>mismatch</u>	Return first position where two ranges differ
<u>equal</u>	Test whether the elements in two ranges are equal
<u>is_permutation</u>	Test whether range is permutation of another
<u>search</u>	Search range for subsequence
<u>search_n</u>	Search range for elements

for_each():

Apply function to range Применить функцию к диапазону

```
void MyFunc(MyType x);  
vector<MyType> v;  
vector<MyType>::iterator p;  
for (p=v.begin(); p!= v.end(); ++p)  
    MyFunc(*p);
```

//Возможная реализация

```
template<typename InIt, typename Fun>  
Fun for_each(InIt b, InIt e, Fun f) {  
    while(b != e) {  
        f(*b);  
        ++b;  
    }  
    return f;  
}
```

Использование

```
for_each(begin(v), end(v), MyFunc);
```

Использование:

```
Fun for_each(InIt first, InIt last, Fun f);
```

```
InIt = list::iterator;
```

```
Fun = void(*)(int);
```

```
// Печать элемента контейнера
```

```
void print(int i) { cout << i << " "; }
```

```
list l{1, 3, 5};
```

```
for_each(begin(l), end(l), print);
```

```
// Тот же код можно написать с использованием лямбда выражений
```

```
for_each(begin(l), end(l), [](int i){cout << i << " ";});
```

Ещё один for_each:

Нам нужен for_each, который будет умножать элементы контейнера на произвольное число

```
void twice (int & x) {
```

```
    x *= 2;
```

```
}
```

```
for_each(begin(v), end(v), twice)
```

```
// с помощью лямбд
```

```
for_each(begin(v), end(v), [](int& x){x *= 2;})
```

find()

Find value in range Найти значение в диапазоне

```
template<typename InIt, typename T>
InIt find(InIt first, InIt last, const T &value);
```

Возможная реализация

```
template<typename InIt, typename T>
InIt find(InIt b, InIt e, T t) {
    while(b != e)
        if(*b == t)
            return b;
        ++b;
    return b;
}
```

Использование

```
int a[]{3, 1, 5, 7};
auto it = find(begin(a), end(a), 5);
if (it != end(a))
    cout << "нашли" << *it;
else
    cout << "нет";
```

Использование

```
list<int> l;
...
list<int>::iterator p=find(begin(l), end(l), 42);
auto p = find(begin(l), end(l), 42);
```

find_if()

Find element in range Найти элемент в диапазоне

```
template<typename InIt, typename Pred>
InIt find_if(InIt first, InIt last, Pred pred);
```

Возможная реализация

```
template<typename InIt, typename
Pred>
InIt find_if(InIt b, InIt e, Pred p) {
    while(b != e)
        if(p(*b))
            return b;
        ++b;
    return b;
}
```

find_if()

Использование

```
int a[] {3, 1, 5, 7}
auto it = find_if(begin(a), end(a), [](int x) -> bool {return x%2 == 0})
if (it != end(a))
    cout << "нашли" << *it;
else
    cout << "Нет";
```

Использование

```
list<int> l;
bool less_than_7(int v) { return v<7; }
list<int>::iterator p = find_if(begin(l), end(l), less_than_7);
```

find_end()

Find last subsequence in range **Найти последнюю подпоследовательность в диапазоне**

```
template<typename FwdIt1, typename FwdIt2>
FwdIt1 find_end(FwdIt1 first1, FwdIt1 last1, FwdIt2 first2, FwdIt2 last2);
```

Пример

```
int array[12] = {7, 3, 3, 7, 6, 8, 0, 3, 7, 6, 5, 5};
int subarr[3] = {3, 7, 6};
int* found_it;
found_it = find_end(array, array+12, subarr, subarr+3);
```

Пример

если первая последовательность – Mississippi,
а вторая – ss,
то find_end() возвращает итератор, указывающий на первую s во втором вхождении ss.

find_first_of()

Find element from set in range **Найти элемент из набора в диапазоне**

```
template<typename FwdIt1, typename FwdIt2>
FwdIt1 find_first_of(FwdIt1 first1, FwdIt1 last1, FwdIt2 first2, FwdIt2 last2);
```

Пример. Поиск первой гласной

Первая последовательность synesthesia

Вторая последовательность aeiou

Пример

```
int x[]={1,3,4};
```

```
int y[]={0,2,3,4,5};
```

```
int* p = find_first_of(x,x+3,y,y+5); // p=&x[1]
```

```
int* q = find_first_of(p+1,x+3,y,y+5);        // q=&x[2]
```

count() и count_if()

Count appearances of value in range **Подсчет появления значения в диапазоне**

```
template<typename InIt, typename Type>
iterator_traits::distance_type count(InIt first, InIt last, const Type& value);
```

Return number of elements in range satisfying condition

Возвращает количество элементов в диапазоне, удовлетворяющих условию

```
template<typename InIt, typename Pred>
iterator_traits::distance_type count_if(InIt first, InIt last, Pred pred);
```

Пример

```
int n = count(p,p+size,'e');
```

Пример

```
bool equals_a(char* v) { return *v=='a'; }
```

```
int n=count_if(p,p+size,equals_a);
```

all_of:

Test condition on all elements in range

```
all_of(b, e, pred);
```

Возвращает true если все элементы удовлетворяют заданному предикату

equal():

Test whether the elements in two ranges are equal

```
template<typename InIt1, typename InIt2>
```

```
bool equal(InIt1 first, InIt1 last, InIt2 first2);
```

```
bool equal(b, e, b1) {  
    while(b != e) {  
        if (*b != *b1)  
            return false;  
        ++b;  
        ++b1;  
    }  
    return true;  
}
```

Пример

```
bool b = equal(begin(li), end(li), begin(vd));
```

search:

Search range for subsequence

Ищет подпоследовательность в последовательности

```
search(b, e, b1, e1);
```

```
template<typename FwdIt1, typename FwdIt2>
```

```
FwdIt search(FwdIt1 first1, FwdIt1 last1, FwdIt2 first2, FwdIt2 last2);
```

Пример

в слове Mississippi подпоследовательность iss встречается дважды,
и search() возвращает итератор, указывающий на начало первого вхождения

Пример

```
string str="How old are you?";
```

```
string s="old";
```

```
char* p=search(begin(str), end(str), begin(s), end(s));
```

Модифицирующие: copy, copy_if, transform, replace, remove, unique, reverse:

Алгоритмы, модифицирующие последовательность

copy	Копирует последовательность, начиная с первого элемента
copy_if	Копирует определенные элементы диапазона
copy_backward	Копирует последовательность, начиная с последнего элемента
swap	Меняет местами два элемента
swap_ranges	Меняет местами элементы двух последовательностей
iter_swap	Меняет местами два элемента, на которые указывают итераторы
transform	Выполняет операцию над каждым элементом в последовательности
replace	Заменяет элементы с указанным значением
replace_if	Заменяет элементы при выполнении предиката
replace_copy	Копирует последовательность, заменяя элементы с указанным значением
replace_copy_if	Копирует последовательность, заменяя элементы при выполнении предиката
fill	Заменяет все элементы данным значением
fill_n	Заменяет первые n элементов указанным значением
generate	Заменяет все элементы результатом операции
generate_n	Заменяет первые n элементов результатом операции

Copy():

Копирует последовательность, начиная с первого элемента

Возможная реализация

```
template<typename ItIn, typename ItOut>
```

```
void copy(ItIn b, ItIn e, ItOut b1) {
```

```
    while (b != e) {
```

```
        *b1 = *b;
```

```
        ++b;
```

```
        ++b1;
```

```
    }
```

```
}
```

```
vector<int> v{1, 5, 3};
```

```
list<int> l;
```

```
copy(begin(v), end(v), begin(l));
```

// Итератор будет иметь нулевое значение, поэтому при попытке обратиться к нему произойдет ошибка

//copy() можно использовать также и для вывода элементов контейнера

Выходной итератор в алгоритме copy для вывода символов в поток cout

copy(text.begin(),text.end(),ostream_iterator<char>(cout,""));

параметрами которого являются итераторы на начало и конец копируемого контейнера и итератор на начало контейнера, куда производится копирование

Для **вывода в поток** третьим параметром должен быть **итератор вывода в поток ostream**
ostream_iterator<char>(cout,""))

copy_if

Copy certain elements of range

Возможная реализация

```
template<class InputIt, class OutputIt, class UnaryPredicate>
OutputIt copy_if(InputIt first, InputIt last, OutputIt d_first, UnaryPredicate pred) {
    while (first != last) {
        if (pred(*first))
            *d_first++ = *first;
        first++;
    }
    return d_first;
}
```

Использование

```
std::copy_if(to_vector.begin(), to_vector.end(), std::ostream_iterator<int>(std::cout, " "),
            [](int x) { return (x % 2) != 0; });
```

reverse_copy

Копирует последовательность в обратном порядке

Возможная реализация

```
template<class BidirIt, class OutputIt>
OutputIt reverse_copy(BidirIt first, BidirIt last, OutputIt d_first) {
    while (first != last) {
        *(d_first++) = *(--last);
    }
    return d_first;
}
```

Использование

```
std::vector<int> v({1,2,3});
print(v);
std::vector<int> destination(3);
std::reverse_copy(std::begin(v), std::end(v), std::begin(destination));
print(destination);
std::reverse_copy(std::rbegin(v), std::rend(v), std::begin(destination));
print(destination);
```

Output: 1 2 3 3 2 1 1 2 3

copy_backward

Копирует последовательность, начиная с последнего элемента

Возможная реализация

```
template< class BidirIt1, class BidirIt2 >
BidirIt2 copy_backward(BidirIt1 first, BidirIt1 last, BidirIt2 d_last) {
    while (first != last) {
        * (--d_last) = *(--last);
    }
    return d_last;
}
```

Использование

```
vector<int> from_vector;
for (int i = 0; i < 10; i++) {
    from_vector.push_back(i);
}
vector<int> to_vector(15);
copy_backward(from_vector.begin(), from_vector.end(), to_vector.end());
for (auto i: to_vector) { std::cout << i << " "; }
```

Output: 0 0 0 0 0 0 1 2 3 4 5 6 7 8 9

Алгоритм replace:

Заменяет в диапазоне все элементы с заданным значением другим значением

Ему необходим однонаправленный итератор

```
int a[10];

for (int i=0; i<10; ++i)
    cin >> a[i];

// Замена всех элементов массива, равных 5, на 6
replace(a, a+10, 5, 6);

-----

deque<char> deq;
char c;
for (int i=0; i<10;++i) {
    cin>>c;
    deq.push_back(c);
    deq.push_front(c);
}

// Замена всех 'e' в deq на 'o'
replace(deq.begin(), deq.end(), 'e', 'o');
```

Поскольку тип `deque<T>::iterator` также удовлетворяет всем требованиям к однонаправленным итераторам, этот алгоритм можно использовать и для замены элементов в деке

transform

```
template< class InputIt, class OutputIt, class UnaryOperation >
OutputIt transform( InputIt first1, InputIt last1, OutputIt d_first, UnaryOperation unary_op )

template< class InputIt1, class InputIt2, class OutputIt, class BinaryOperation >
OutputIt transform( InputIt1 first1, InputIt1 last1, InputIt2 first2, OutputIt d_first, BinaryOperation binary_op )
```

Применяет заданную функцию к одному диапазону и сохраняет результат в другой диапазон, начинающийся с `d_first`

В первом варианте унарная операция `unary_op` применяется к диапазону `[first1, last1)`.
Во втором варианте бинарная операция `binary_op` применяется к элементам из двух диапазонов: `[first1, last1)` и начинающемуся с `first2`.

Параметры

<code>first1, last1</code>	—	первый диапазон элементов для изменения
<code>first2</code>	—	начало второго диапазона элементов для изменения
<code>d_first</code>	—	начало целевого диапазона, может совпадать с <code>first1</code> или <code>first2</code>

Возможная реализация transform

Первый вариант

```
template<class InputIt, class OutputIt, class UnaryOperation>
OutputIt transform(InputIt first1, InputIt last1, OutputIt d_first, UnaryOperation unary_op) {
    while (first1 != last1) {
        *d_first++ = unary_op(*first1++);
    }
    return d_first;
}
```

Второй вариант

```
template<class InputIt1, class InputIt2, class OutputIt, class BinaryOperation>
OutputIt transform(InputIt1 first1, InputIt1 last1, InputIt2 first2, OutputIt d_first, BinaryOperation binary_op)
{
    while (first1 != last1) {
        *d_first++ = binary_op(*first1++, *first2++);
    }
    return d_first;
}
```

//пр
и-
ме-

ры

```
#include <string>
#include <algorithm>
#include <iostream>

int main() {
```

```
    std::string s("hello");
```

```

std::transform(begin(s), end(s), s.begin(), std::toupper);

std::cout << s;

}

```

```

transform(v.begin(), v.end(), v.begin(), [](int x){return x + 1;});
vector v1{1, 2, 3};
int a[] {2, -2, 8};
list l;
transform(begin(v1), end(v1), a, back_inserter(l), std::max)

```

std::remove, std::remove_if

template< class ForwardIt, class T >	
ForwardIt remove(ForwardIt first, ForwardIt last, const T& value);	(until C++20)

template< class ForwardIt, class T >	
constexpr ForwardIt remove(ForwardIt first, ForwardIt last, const T& value);	(since C++20)

template< class ForwardIt, class UnaryPredicate >	
ForwardIt remove_if(ForwardIt first, ForwardIt last, UnaryPredicate p);	(until C++20)

template< class ForwardIt, class UnaryPredicate >	
constexpr ForwardIt remove_if(ForwardIt first, ForwardIt last, UnaryPredicate p);	(since C++20)

Возможная реализация remove

```
template< class ForwardIt, class T >
ForwardIt remove(ForwardIt first, ForwardIt last, const T& value) {
    first = std::find(first, last, value);
    if (first != last)
        for(ForwardIt i = first; ++i != last; )
            if (!(*i == value))
                *first++ = std::move(*i);
    return first;
}
```

Возможная реализация remove_if

```
template<class ForwardIt, class UnaryPredicate>
ForwardIt remove_if(ForwardIt first, ForwardIt last, UnaryPredicate p) {
    first = std::find_if(first, last, p);
    if (first != last)
        for(ForwardIt i = first; ++i != last; )
            if (!p(*i))
                *first++ = std::move(*i);
    return first;
}
```

```
int main() {
```

```
    std::string str1 = "Text with some  spaces";
    str1.erase(std::remove(str1.begin(), str1.end(), ' '), str1.end());
    std::cout << str1 << '\n';
```

```
    std::string str2 = "Text\n with\tsome \t whitespaces\n\n";
    str2.erase(std::remove_if(str2.begin(), str2.end(), [](unsigned char x){return std::isspace(x);}), str2.end());
    std::cout << str2 << '\n';
```

```
}
```

Сортировка и операции с множествами: `sort`, `set_intersection`, `set_union`:

//это так, к слову

В некоторых случаях (например, когда элементы данных представляют собой **большие структуры**) **сортировка** последовательности элементов может быть выполнена более эффективно путем **вставки их в мультимножество и обхода мультимножества**, чем обобщённым алгоритмом сортировки или соответствующей функцией-членом списка.

В библиотеке имеются алгоритмы для выполнения теоретико-множественных операций:

`includes`,

`set_union`,

`set_intersection`,

`set_difference`,

`set_symmetric_difference`.

Эти операции работают с любыми **отсортированными** контейнерами, включая ассоциативные. В случае неупорядоченных контейнеров алгоритмы для теоретико-множественных операций завершаются ошибкой времени выполнения.

Порядок сортировки(`sort`)

Например, чтобы **изменить порядок сортировки** вектора с возрастания на убывание, можно **вместо** написания своего **компаратора** использовать **реверсивные итераторы**.

// сортирует вектор в порядке возрастания

```
sort( v.begin(), v.end() );
```

// сортирует вектор в порядке убывания

```
sort( v.rbegin(), v.rend() );
```

Функторы (функциональные объекты) и лямбда-выражения (C++11): синтаксис {}, списки захвата ([=], [&], [a], [&b]), `mutable` + Класс `std::function`:

Однако, лямбда-функция это не только анонимная функция, вызываемая в месте своего использования.

Помимо прочего, лямбда-функция может использовать переменные, доступные в ее контексте.

То есть переменные, доступные в том же контексте, где и описана лямбда-функция, доступны внутри лямбда-функции.

Это называется **замыкание**.

Цитата из Википедии:

Замыкание (англ. **closure**) в программировании — функция, в теле которой присутствуют ссылки на переменные, объявленные вне тела этой функции и не в качестве её параметров (а в окружающем коде).

Говоря другим языком, замыкание — функция, которая ссылается на свободные переменные в своём контексте.

//примеры + примеры можно также взять из 10-11 лаб

```
std::vector v;
```

```
const double epsilon;
```

```
std::transform(v.begin(), v.end(), v.begin(), [epsilon](double d) -> double {  
    if (d < epsilon) { return 0;}  
    else { return d;}  
});
```

```
void func5(std::vector& v, const double& epsilon) {  
    std::transform(v.begin(), v.end(), v.begin(), [epsilon](double d) -> double {  
        if (d < epsilon) { return 0; }  
        else { return d; }  
    }  
}
```

Помимо всего прочего, можно захватывать не только переменную. Можно захватывать ссылку на переменную.

Помимо имен захватываемых переменных, можно указывать «правила захвата», обозначающие

«&» и «=»:

- [**&epsilon**] — захват ссылки на переменную;
- [**&**, **epsilon**] — показывает, что по умолчанию захват переменных будет происходить по ссылке но

для переменной **epsilon** захват должен производиться по значению;

- [**=**, **&epsilon**] — показывает, что по умолчанию захват переменных будет происходить по значению, но для переменной **epsilon** захват должен производиться по ссылке;

- `[x, &y]` — захват переменной `x` по значению, и переменной `y` по ссылке.

Сумма элементов контейнера с помощью `for_each`:

```
int sum = 0;
for_each(begin(v), end(v), [&sum](int x){sum += x;})
```

Объекты функции (функторы):

На самом деле в C++ лямбда выражения являются синтаксическим сахаром.

В действительности лямбда функции с захваченными переменными переводятся в объекты функций. **Объектом функции** называется **объект класса с перегруженной операцией `operator()`**.

```
class SumHelper {
public:
    int sum;
    SumHelper(): sum(0) {}
    void operator()(int x) {
        sum += x;
    }
}

auto f = for_each(begin(v), end(v), SumHelper());
cout << f.sum;
```

На самом деле `SumHelper()` в `for_each` — это не функция, а экземпляр класса, у которого перегружен `operator()`

Присваивание лямбда функций переменной:

```
#include <functional>
using namespace std;

// Здесь auto принимает тип std::function<int(int)>, поэтому можно написать так:
std::function<int(int)> g = [](int x){return x*x;};

cout << g(5);
```

13. Продвинутые темы C++:

constexpr: вычисление на этапе компиляции, отличие от const:

constexpr — спецификатор типа, введённый в стандарт программирования языка C++11 для обозначения константных выражений, которые могут быть вычислены во время компиляции кода.

Изначально стандартом языка C++11 на его применение был наложен ряд существенных условий и ограничений, однако большинство из них было снято после выхода стандарта C++14.

Разница между const и constexpr

Данная программа компилируется

```
#include <iostream>
```

```
struct A {  
    constexpr static double x = 10.0;  
};
```

```
int main() {  
    A a;  
    return 0;  
}
```

А данная программа нет.

```
#include <iostream>
```

```
struct A {  
    const static double x = 10.0;  
};
```

```
int main() {  
    A a;  
    return 0;  
}
```

Когда спецификатор constexpr используется для функций

Существенное значение также имеет место, когда этот спецификатор, constexpr, используется для функций.

Только функции-члены класса могут иметь квалификатор const, который имеет отношение к объекту, для которого вызывается данная функция-член класса.

Обычные функции не могут быть константными.

Спецификатор constexpr введен для того, чтобы заставить компилятор на этапе компиляции создавать объекты и использовать их как константы времени компиляции.

lvalue и rvalue: понятие, ссылки на rvalue (&&):

Любое выражение в C++ является или левосторонним (lvalue), или правосторонним (rvalue)

Выражение **lvalue** — это именующее выражение

Выражение **rvalue** — это значащее выражение

//Более классная формулировка

Выражение **lvalue** — это обычно объект, который имеет имя. Все переменные являются lvalue

А выражение **rvalue** — это временный безымянный объект, не существующий за пределами того выражения, которое его создало

Ковариантные возвращаемые типы виртуальных функций:

Если **тип возвращаемого** значения **виртуальной** функции является **указателем** или **ссылкой** на класс, **переопределяющие** функции могут возвращать **указатель** или **ссылку** на **производный** класс.

Это называется **ковариантными возвращаемыми типами**.

//пример

```
class filled_circle: public circle {
public:
    filled_circle (double r, int c): circle(r), color(c) { }
    ~filled_circle() {}

    filled_circle* clone() const override {
        return new filled_circle(radius, color);
    }

    void debug (ostream & out) const {
        circle::debug(out);
        out<<" color = "<<color <<endl;
    }

private:
    int color;
};
```

Синглтон: понятие, запрет копирующего конструктора и оператора присваивания:

На практике иногда **создаваемые неявно функции** могут привнести **нежелательную функциональность**, от которой нужно избавиться.

Например, нужно создать класс, для которого должен существовать **только один экземпляр**. Такие классы называются **синглтонами**.

Для них рекомендуется запретить операцию присваивания и конструктор копирования.

С другой стороны, рекомендуется наряду с другими конструкторами всегда иметь конструктор без параметров, даже если его тело является пустым.

Управление генерацией стандартных конструкторов и функций:

```
class A {  
private:  
    int x;  
public:  
    A(int i) {...}  
    // Данная запись указывает на необходимость сгенерировать конструктор без параметров  
    A() = default;  
    // Запретить генерацию конструктора копии по умолчанию  
    A(const A&) = delete;  
    // А так можно запретить генерацию operator=  
    A& operator = (const A&) = delete;  
}
```

14. Практические задачи

Реализация класса Date:

```
/* date.h */

class Date {
private: // это слово писать не обязательно, но желательно
    int y, m, d;

public:
    // Конструктор класса с параметрами
    Date(int d, int m, int y) {
        this->d = d;
        this->m = m;
        this->y = y;
    }

    // Конструктор класса без параметров
    Date() {
        d = 0;
        m = 0;
        y = 0;
    }

    ...

    void add_days(int n);
};
```

//конструктор для этого класса также можете подглядеть в теме: конструктор со списком инициализаторов\

//Дополнение к этому классу

```
class Date{
private:

    void add_days(int n);

    ...

public:

    void operator+= (int n) { // inline
        add_days(n); //реализацию я не нашел, но скорее всего к переменным y, m, d
        добавляются некоторые значения и т.д
    }

    //bool operator==(Date d1, Date d2) типичная ошибка
    bool operator==(Date const & other) {
        return d == other.d &&
            m == other.m &&
            y == other.y;
    }

    bool operator!=(const Date &d2){
```

```

        return !(*this==d2);
    }

    Date operator++(int) {
        Date d = *this;
        //Date d = Date (*this);
        add_days(1); // ~ ++(*this);
        return d;
    }

```

```

    Date & operator++() {
        add_days(1);
        return *this;
    }

```

```

    friend ostream & operator<<(ostream & os, Date const & d){
        return os << d.d << '.' << d.m << '.' << d.y << '\n';
    }

```

```

};

```

//если все приколы отсюда реализовать, то мы видим что, этот код выглядит так, будто Date — встроенный тип

```

Date d(17, 10, 14), d1 = d;
cin >> d1;
Date d2(31, 12, 14);
Date d3;
d += 7;
d1 = d1 - 7;
int n = d2 - d;
if (d == d1)...
d++; ++d; d1--; --d1;
d2++;

```

```
cout << d << ' ' << d1
```

Реализация класса Circle:

```
//circle.h
```

```
#pragma once
```

```
class circle {  
    private:  
        double x,y,r;  
    public:  
        circle();  
        circle(double x1, double y1, double r1);  
        double s();  
        void set_r(double r1);  
        double get_r();  
        bool equal(circle a);  
        bool operator ==(circle a);  
};
```

```
//circle.cpp
```

```
#include <cmath>
```

```
#include "circle.h"
```

```
circle::circle(double x1=10, double y1=10, double r1=10){  
    x=x1;  
    y=y1;  
    r=r1;  
}
```

```
double circle::s(){ return 3.14*r*r;}
```

```
void circle::set_r(double r1){  
    if (r1<0) r=0;  
    else r=r1;  
}
```

```
double circle::get_r(){ return r;}
```

Реализация классов Person – Student (наследование):

```
class Person {  
private:  
    string name;  
    int age;  
public:  
    Person(const string& name = "noname", int age = 18): name(name), age(age) {}  
    friend ostream & operator<< (ostream & os, const Person & p) {  
        os << p.name << " " << p.age << endl;  
        return os;  
    }  
};
```

```
class Student: public Person {  
private:  
    string univ;  
    // Университет  
    int m;  
    int* marks;  
    // Оценки  
public:  
    Student() : univ("МГУ"), m(3){  
        marks = new int[m];  
        for (int i = 0; i<m; ++i)  
            marks[i] = 0;  
    }
```

```
Student(const string& name, int age, const string& u, int m): Person(name, age), univ(u), m(m)  
{
```

```

marks = new int[m];
for (int i = 0; i < m; ++i)
    marks[i] = 0;
}

```

```

Student(const Student& s): Person(s), univ(s.univ), m(s.m){
    marks = new int[s.m];
    for (int i = 0; i < m; ++i)
        marks[i] = s.marks[i];
}

```

```

~Student() { delete[] marks; }

```

```

Student& operator=(const Student& s) {
    if (&s != this) {
        delete[] marks;

        Person::operator=(s); //обязательно указывать для того, чтобы мы передали
                               //данные базовому классу(оператор копирующего
                               //присваивания для класса Person генерируется
                               //автоматически и такой вариант будет работать, т.к там нет
                               //динамической памяти)

        univ = s.univ;
        m = s.m;
        marks = new int[m];
        for (int i = 0; i < m; ++i)
            marks[i] = s.marks[i];
    }
    return *this;
}

```

```

void setMark(int i, int mark) {
    if (i < 0 || i >= m)
        throw - 1;
    marks[i] = mark;
}

```

```

}

string get_univ() const {

    return univ;

}

friend ostream & operator<< (ostream & os, const Student & s) {

    os<<(Person)s;

    os << s.univ << endl;

    for (int i = 0; i < s.m; ++i)

        os << s.marks[i] << " ";

    os << endl;

    return os;

}

};

```

Поскольку **операция вывода в поток является внешней**, то к ней невозможно применить разрешение области видимости. Поэтому используется **явное приведение типа**.

Наличие **динамически выделяемой памяти** в классе Student для int* marks создаёт дополнительные проблемы

В этом случае нельзя использовать создаваемые по умолчанию функции: **конструктор без параметров, конструктор копии, деструктор и операцию присваивания**.

Необходимо предусмотреть их реализацию в классе.

//Измененная реализация Student-Person с использованием класса Array

```

class Student: public Person {
private:
    string univ;
    // Университет
    Array marks;
    // Оценки
public:
    Student() : marks(3), univ("МГУ") {}

    Student(const string& name, int age, const string& u, int m):
    Person(name, age), univ(u), marks(m) { }

```

```

void setMark(int i, int mark) {
    if (i < 0 || i >= marks.length())
        throw - 1;
    marks[i] = mark;
}

```

```

friend ostream & operator<< (ostream & os, const Student & s)
{
    os<<(Person)s;
    os << s.univ << endl;
    for (int i = 0; i < s.marks.length(); ++i)
        os << s.marks[i] << " ";
    os << endl;
    return os;
}

```

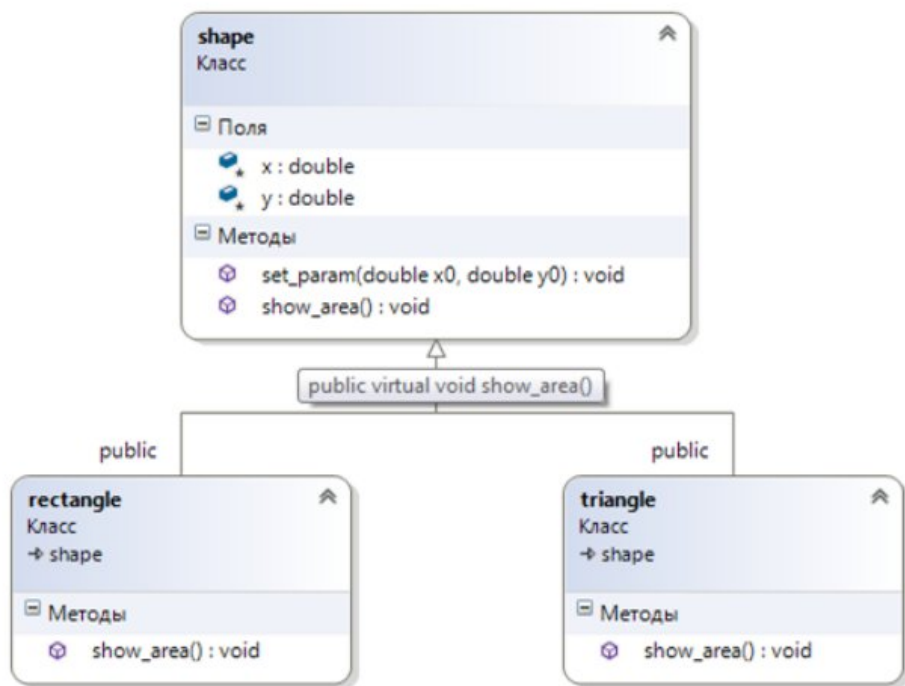
```

friend ostream & operator<< (ostream & os, const Student & s)
{
    os<<(Person)s;
    os << s.univ << endl;
    os << marks << endl;
    return os;
}
};

```

Реализация классов геометрических фигур с полиморфизмом (shape, circle, rectangle):

В этом примере наследование **не** имеет целью **расширение** базового класса. Каждый из классов-наследников **реализует свое собственное поведение** на основе виртуальной функции, объявленной в базовом классе.



//Базовый класс shape

```
class shape {
```

```
protected:
```

```
    double x,y;
```

```
public:
```

```
    //конструкторы не создаём, так как используется автоматический конструктор по
    //умолчанию
```

```
    void set_param(double x0, double y0) {
```

```
        x=x0;
```

```
        y=y0;
```

```
    }
```

```
    virtual void show_area(){
```

```
        cout<<"Area calculation for this class is undefined";
```

```
        cout <<endl;
```

```
    }
```

```
};
```

//потомки rectangle и triangle

```
class triangle: public shape {
```

public:

```
//используется автоматический конструктор по умолчанию
```

```
//вызывающий конструктор по умолчанию базового класса
```

```
void show_area() {
```

```
    cout<<endl<<"triangle with height "<<x<<" and base "<<y;
```

```
    cout<<endl<<"has an area "<<x*0.5 *y<<endl;
```

```
}
```

```
};
```

```
class rectangle : public shape {
```

public:

```
//используется автоматический конструктор по умолчанию
```

```
//вызывающий конструктор по умолчанию базового класса
```

```
void show_area() {
```

```
    cout<<endl<<"rectangle with sides "<<x<<" * "<<y;
```

```
    cout<<endl<<"has an area "<<x*y<<endl;
```

```
}
```

```
};
```

Реализация класса-счётчика объектов (Counter):

```
class Counter {
```

private:

```
    static int count;
```

public:

```
    static int getCount() {
```

```
        return count;
```

```
}
```

```
    Counter() {
```

```
        ++count;
```

```
}
```

```
    ~Counter() {
```

```
        --count;
```

```
    }  
};
```

Реализация класса Array (динамический массив):

```
class Array {  
private:  
    int n;  
    int *A;  
public:  
    Array();  
    //конструктор по умолчанию  
    Array(int _n, int x = 0); //конструктор с параметром по умолчанию  
    Array(const Array &B); //конструктор копии  
    int length() const;  
    //функция для нахождения размера массива  
    void resize(int nsize);  
    //изменение размера массива  
    Array operator + (const Array &B);  
    Array &operator += (const Array &B);  
    Array operator + (const int x);  
    Array &operator += (const int x);  
    Array &operator = (const Array &B);  
    int& operator [] (int i);  
    int operator [] (int i) const;  
    ~Array();  
    friend ostream & operator << (ostream &out, const Array &B);  
};
```

//Array.cpp

```
Array::Array(): {  
    n=10;  
    A=new int[n];
```

```
        for (int i=0; i<n; ++i) A[i]=0;
    }
```

```
Array::Array(int _n, int x = 0): n(_n) {
    A=new int[n];
    for (int i=0; i<n; ++i) A[i]=x;
}
```

```
Array::~~Array() {
    delete[] A;
}
```

```
int Array::length() const{
    return n;
}
```

```
void Array::resize(int nsize) {
    int * ndata = new int[nsize];
    int sz = (n < nsize) ? n : nsize;
    for (int i=0;i<sz;++i)
        ndata[i] = A[i];
    delete[] A;
    A = ndata;
    n = nsize;
}
```

```
Array::Array (const Array &B){
    n=B.n;
    A=new int[n];
    for (int i=0; i<n; i++)
        A[i]=B.A[i];
}
```

```
}
```

```
int& Array::operator [] (int i){  
    return A[i];  
}
```

```
int Array::operator [] (int i) const{  
    return values[i];  
}
```

```
//для массива и целого числа
```

```
Array Array::operator+(const int x){  
    Array C(n);  
    //можно: Array C(n,0);  
    for (int i = 0; i<n; i++)  
        C.A[i] = A[i] + x;  
    return C;  
}
```

```
//для двух массивов
```

```
Array Array::operator+(const Array &B){  
    if (n != B.n)  
        throw (1);  
    Array C(n);  
    //можно: Array C(n,0);  
    for (int i=0; i<n; i++)  
        C.A[i]=A[i]+B.A[i];  
    return C;  
}
```

```
//для массива и числа
```

```
Array& Array::operator += (const int k){
```

```

        for (int i=0; i<n; i++)
            A[i]=A[i]+k;
        return *this;
    }

```

//для двух массивов

```

Array& Array::operator += (const Array &B){
    if (n != B.n)
        throw (1);
    for (int i=0; i<n; i++)
        A[i]=A[i]+B.A[i];
    return *this;
}

```

//Последние два варианта реализации + и += для двух массивов предпочтительней(но необязательно)

```

Array Array::operator + (const Array &B){
    Array C(n);
    for (int i=0; i<n; i++)
        C.A[i]=A[i]+B.A[i];
    return C;
}

```

```

Array Array::operator + (const Array &B){
    Array C(*this);
    C+=B;
    return C;
}

```

```

Array& Array::operator = (const Array &B) {
    if (this != &B) {
        delete[] A;
        n=B.n;
    }
}

```

```

        A=new int[n];
        for (int i=0; i<n; i++)
            A[i]=B.A[i];
    }

    return *this;
}

```

Реализация класса String(Брал из учебника):

```

//STR.h

#pragma once

#include <iostream>

using namespace std;

class Str {

    friend ostream & operator << (ostream &, const Str &);

    friend istream & operator >> (istream &, Str &);

public:

    Str(const char * = "");

    Str(const Str &);

    //конструктор преобразования

    //конструктор копии

    ~Str();

    //деструктор

    Str & operator= (const Str &); //присваивание

    Str & operator+= (const Str &); //сцепление (конкатенация)

    bool operator!() const;

    //проверка строки на пустоту

    bool operator==(const Str &) const; //проверка на равенство

    char & operator[] (int);

    //получение ссылки на символ

    char operator[] (int) const; //получение символа по номеру

    Str operator() (int, int); //получение подстроки

    int getLength() const;

```

```

        //определение длины строки
        operator char*() const;
        //операция приведения к char*
private:
        char * sPtr;
        int length;
};
//указатель на начало строки
//длина строки

//STR.cpp
// Определение некоторых функций элементов класса Str
#include <iostream>
#include <cstring>
#include <cassert>
#include <iomanip>
#include "str.h"
// конструктор преобразования: преобразовывает char* в Str
Str::Str(const char* s) {
        length=(int)strlen(s);
        sPtr=new char[length + 1];
        assert(sPtr != 0);
        //завершение, если память не выделена
        strcpy(sPtr,s);
}

// конструктор копии
Str::Str(const Str & copy) {
        length=copy.length;
        sPtr=new char[length + 1];
        assert(sPtr != 0);
        //завершение, если память не выделена

```

```

        strcpy(sPtr,copy.sPtr);
    }

// деструктор
Str::~Str() {
    delete [] sPtr;
}

//операция присваивания
Str & Str::operator =(const Str & right) {
    if (&right != this) {
        delete [] sPtr;
        length = right.length;
        sPtr = new char[length + 1];
        assert(sPtr != 0);
        strcpy(sPtr, right.sPtr);
    }
    return *this;
}

strcpy(sPtr, right.sPtr);
{
    return *this;
}

```

//сцепление (конкатенация) На практике иногда создаваемые неявно функции могут привести нежелательную

функциональность, от которой нужно избавиться.

Например, нужно создать класс, для которого должен существовать только один экземпляр.

Такие классы называются **синглтонами**.

Для них рекомендуется запретить операцию присваивания и конструктор копирования.

С другой стороны, рекомендуется наряду с другими конструкторами всегда иметь конструктор без параметров, даже если его тело является пустым.

```

Str & Str::operator +=(const Str & right) {
    char * tempPtr = sPtr;
    length += right.length;
    sPtr = new char [length+1];
    assert(sPtr!=0);
    strcpy(sPtr, tempPtr);
    strcat(sPtr, right.sPtr);
    delete [] tempPtr;
    return *this;
}

```

//проверка строки на пустоту

```

bool Str::operator !() const {
    return length ==0;
}

```

//проверка двух строк на равенство

```

bool Str::operator ==(const Str & right) const {
    if (length!=right.length)
        return 0;
    return strcmp(sPtr, right.sPtr) == 0;
}

```

//получение ссылки на символ

```

char & Str::operator[] (int ind) {
    //проверка, не находится ли индекс вне диапазона
    assert(ind >= 0 && ind < length);
    return sPtr[ind];
    //создание L-величины
}

```

```

//получение символа
char Str::operator[] (int ind) const{
//проверка, не находится ли индекс вне диапазона
    assert(ind >= 0 && ind < length);
    return sPtr[ind];
//создание R-величины
}

//получение подстроки заданной длины, начинающейся с заданного
//индекса
Str Str::operator() (int ind, int sublength) {
    //проверка, что индекс в диапазоне и длина подстроки >=0
    assert(ind >= 0 && ind < length && sublength>=0);
    Str sub;
    //определение длины подстроки
    if ((sublength==0) || (ind+sublength>length))
        sub.length=length-ind+1;
    else{
        sub.length=sublength+1;
        //выделение памяти для подстроки
        sub.sPtr=new char[sub.length];
        assert(sub.sPtr!=0);
        //копирование подстроки в новый Str
        strncpy(sub.sPtr,&sPtr[ind],sub.length);
        sub.sPtr[sub.length]='\0'; //завершение новой строки sPtr
        return sub;
    }

    //определение длины строки
    int Str::getLength() const {
        return length;
    }
}

```

```
//операция приведения к char*
```

```
Str::operator char*() const {  
    char *c=new char[length];  
    assert(sPtr != 0);  
    strcpy(c,sPtr);  
    return c;  
}
```

```
//перегруженная операция вывода данных
```

```
ostream &operator << (ostream & output, const Str & s) {  
    output<<s.sPtr;  
    //возврат ссылки на поток для возможности многократного  
    //последовательного использования операции <<  
    return output;  
}
```

```
//перегруженная операция ввода данных
```

```
istream &operator >> (istream & input, Str & s) {  
    char temp[100]; //буфер для хранения входных данных;  
    input>>setw(100)>>temp;  
    s=temp;  
    return input;  
}
```

```
// String.cpp
```

```
#include "Str.h"
```

```
int main() {  
    char *s=new char[100];  
    cin>>s;  
    Str s1("Hello"),s2(s), s3;  
    cin>>s3;  
    cout<<s1<<endl;
```

```

    cout<<s2<<endl;
    cout<<s3<<endl;
    s=s3;
    cout<<s<<endl;
    cin>>s;
    return 0;
}

```

Реализация класса дроби (frac):

//Реализацию брал из своей лабы, поэтому называется Fract

Экземпляр класса frac хранит число f в виде отношения m/n , где m и n целые числа типа `int`.

В данном классе необходимо реализовать конструктор преобразования типов и операцию преобразования объекта `frac` к типу `double`. //реализация представлена в 3 главе

//Fract.h

```
#include <iostream>
```

```
class Fract {
```

```
private:
```

```
    int numen, denom;
```

```
    void simplify();
```

```
public:
```

```
    // Конструктор по умолчанию
```

```
    Fract() : numen(0), denom(1) {}
```

```
    // Конструктор с параметрами
```

```
    Fract(int a, int b = 1) : numen(a) {
```

```
        denom = (b != 0) ? b : 1;
```

```
        simplify();
```

```
    }
```

```
    // Арифметические операторы
```

```
Fract operator+(const Fract& other);  
Fract operator-(const Fract& other);  
Fract operator*(const Fract& other);  
Fract operator/(const Fract& other);
```

```
// Операторы сравнения
```

```
bool operator==(const Fract& other);  
bool operator!=(const Fract& other);  
bool operator>(const Fract& other);  
bool operator<(const Fract& other);  
bool operator>=(const Fract& other);  
bool operator<=(const Fract& other);
```

```
// Операторы ввода-вывода
```

```
friend std::ostream& operator<<(std::ostream&, const Fract&);  
friend std::istream& operator>>(std::istream&, Fract&);
```

```
};
```

```
//Fract.cpp
```

```
#include <iostream>
```

```
#include "fract.h"
```

```
// --- Арифметические операторы ---
```

```
Fract Fract::operator+(const Fract& other) {  
    Fract temp;  
    temp.denom = this->denom * other.denom;  
    temp.numen = this->numen * other.denom + other.numen * this->denom;  
    temp.simplify();  
    return temp;  
}
```

```
Fract Fract::operator-(const Fract& other) {
```

```

Fract temp;
temp.denom = this->denom * other.denom;
temp.numen = this->numen * other.denom - other.numen * this->denom;
temp.simplify();
return temp;
}

```

```

Fract Fract::operator*(const Fract& other) {
    Fract temp;
    temp.denom = this->denom * other.denom;
    temp.numen = this->numen * other.numen;
    temp.simplify();
    return temp;
}

```

```

Fract Fract::operator/(const Fract& other) {
    Fract temp;
    // Деление — это умножение на перевернутую дробь
    temp.numen = this->numen * other.denom;
    temp.denom = this->denom * other.numen;
    temp.simplify();
    return temp;
}

```

// --- Операторы сравнения ---

```

bool Fract::operator==(const Fract& other) {
    // Так как дроби всегда вызываются с simplify(), они гарантированно приведены к общему
    // виду
    return this->numen == other.numen && this->denom == other.denom;
}

```

```
bool Fract::operator!=(const Fract& other) {  
    return !(*this == other);  
}
```

```
bool Fract::operator>=(const Fract& other) {  
    return this->numen * other.denom >= this->denom * other.numen;  
}
```

```
bool Fract::operator<=(const Fract& other) {  
    return this->numen * other.denom <= this->denom * other.numen;  
}
```

```
bool Fract::operator>(const Fract& other) {  
    return !(*this <= other); // Строго больше — это отрицание "меньше или равно"  
}
```

```
bool Fract::operator<(const Fract& other) {  
    return !(*this >= other); // Строго меньше — это отрицание "больше или равно"  
}
```

```
// --- Вспомогательные методы ---
```

```
void Fract::simplify() {  
    if (denom == 0) return; // Защита от деления на ноль  
  
    if (denom < 0) {  
        numen = -numen;  
        denom = -denom;  
    }
```

```
    int a = std::abs(numen);
```

```
    int b = std::abs(denom);
```

```

while (b != 0) {
    a = a % b;
    std::swap(a, b);
}

numen /= a;
denom /= a;
}

// --- Операторы ввода-вывода ---
std::ostream& operator<<(std::ostream& os, const Fract& fr) {
    os << fr.numen << " / " << fr.denom;
    return os;
}

std::istream& operator>>(std::istream& is, Fract& fr) {
    is >> fr.numen;
    is >> fr.denom;
    fr.simplify();
    return is;
}

```

Реализация бинарного дерева поиска (рекурсивные структуры): (реализован второй вариант из лекции, т.к он есть и в лабе +он рекомендуется)

Немного лирики, доп.тема:

Рекурсивное определение данных возникает, когда структура данных содержит ссылку на объект того же типа.

Примеры рекурсивных структур:

- **Односвязный список** — содержит одну ссылку на следующий элемент. Для таких структур обычно эффективнее использовать **итеративные алгоритмы**.
- **Бинарное дерево** — содержит две ссылки (на левое и правое поддеревья). Для таких структур рекурсивные алгоритмы естественнее и проще в реализации.

Рекурсивное определение бинарного дерева:

- Первый способ определения класса бинарного дерева состоит в том, чтобы определить в нём указатели на два поддеревя и открытые рекурсивные функции для работы с деревом.
- Второй способ. Рекурсивное определение бинарного дерева можно реализовать как класс, содержащий внутри себя указатель на узел дерева (корень) и рекурсивное описание узла дерева.

При этом открытые функции-члены класса не будут рекурсивными.

//Реализация

```
#include
```

```
using namespace std;
```

```
class Tree { private: struct TNode; typedef TNode* node_ptr;
```

```
struct TNode {  
    int data;  
    node_ptr lt, rt;
```

```
    TNode(int val, node_ptr l = nullptr, node_ptr r = nullptr)  
        : data(val), lt(l), rt(r) {}  
};
```

```
node_ptr root;
```

```
void delTree(node_ptr t) {  
    if (t != nullptr) {  
        delTree(t->lt);  
        delTree(t->rt);  
        delete t;  
    }  
} // <-- Была пропущена скобка
```

```
void add(node_ptr& t, int a) {  
    if (t == nullptr) {  
        t = new TNode(a);  
    } else if (t->data > a) {  
        add(t->lt, a);  
    } else {  
        add(t->rt, a);  
    }  
}
```

```
void printLKR(node_ptr t, ostream& os) const {  
    if (t) {
```

```

        printLKR(t->lt, os);
        os << t->data << " ";
        printLKR(t->rt, os);
    }
}

```

```

public: Tree() : root(nullptr) {}

```

```

Tree(const Tree& t) {
    // Внимание: метод copy(...) должен быть определен в private секции,
    // иначе здесь будет ошибка компиляции.
    copy(t.root, root);
}

```

```

~Tree() {
    delTree(root);
}

```

```

void addNode(int a) {
    add(root, a);
}

```

```

friend ostream& operator<<(ostream& os, const Tree& t) {
    t.printLKR(t.root, os);
    return os;
}

};

```

//Особенности(доп.тема)

В примере типы `TNode` (узел) и `node_ptr` (указатель на узел) определены внутри класса, что подчёркивает **инкапсуляцию внутренней** организации класса `Tree`.

Именно в определении типа **`TNode`** присутствует **рекурсия**.

Поэтому **рекурсивные функции** оперируют с указателями на узел дерева и должны быть объявлены как **`private`**.

Для доступа к ним используются **открытые функции** класса, которые **вызывают рекурсивные функции**, передавая в качестве параметра указатель на корень дерева.

Такое описание класса допускает существование пустого дерева.

Конструктор без параметров создаёт пустое дерево.

Если предусмотреть операцию удаления узла дерева, то можно получить в процессе её выполнения пустое дерево.

Для безопасной работы с таким деревом рекомендуется иметь операцию проверки дерева на пустоту

Также доп.тема: Callback-функции

Callback — это функция, которая передается как параметр в другую функцию, чтобы быть вызванной в определенный момент.

//пример

```
typedef double (*BinOp) (double, double);
```

Реализация двусвязного списка и итератора к нему + прочие ал-госы:

```
struct node{
int data;
node* next;
node* prev;
node(int data, node* next, node* prev) {
    this->data = data;
    this->next = next;
    this->prev = prev;
}
};

ostream & operator<<(ostream & out, const node& X) {
    out << X.data;
    return out;
}
```

```
class listIterator; //предварительная инициализация
```

```
class List {
public:
    class Error {
    public:
        void what() {
```

```

        cout << "List is empty" << endl;
    }

};

List();
List(const List& l);
~List();
bool isEmpty() const;
void inHead(int val);
void inTail(int val);
int getFirst()const;
int getLast()const;
void delFirst();
void delLast();
listIterator begin() const;
listIterator end() const;
friend ostream& operator<<(ostream& os, const List& l);
//в полной реализации могут быть еще методы
private:
    node *dummy;
    Error err;
    friend class listIterator;
};

```

```

List::List(const List& l): List() {
    dummy = new node(0, nullptr, nullptr);
    dummy->next = dummy;
    dummy->prev = dummy;
    if (l.dummy->next!=l.dummy) {
        node* q = l.dummy->next;
        while (q != l.dummy) {
            inTail(q->data);
            q = q->next;
        }
    }
}

```

```
    }  
}  
}
```

```
List::List() {  
    dummy = new node(0, nullptr, nullptr);  
    dummy->next = dummy;  
    dummy->prev = dummy;  
}
```

```
List::~~List() {  
    if (dummy->next!=dummy) {  
        node* t = dummy->next;  
        while (t!=dummy) {  
            node* cur = t;  
            t = t->next;  
            delete cur;  
        }  
    }  
    delete dummy;  
}
```

```
bool List::isEmpty() const {  
    return (dummy->next == dummy);  
}
```

```
void List::inHead(int val) {  
    node* t = new node(val, dummy->next, dummy);  
    if (dummy->next==dummy) {  
        dummy->prev = t;  
        t->next = dummy;  
    }  
}
```

```

        else {
            t->next->prev = t;
        }
        dummy->next = t;
    }

void List::inTail(int val) {
    if (dummy->next == dummy)
        inHead(val);
    else {
        node* t = new node(val, dummy, dummy->prev);
        t->prev->next = t;
        dummy -> prev = t;
    }
}

```

```

int List::getFirst() const {
    if (dummy->next!=dummy)
        return dummy->next->data;
    else
        throw err;
}

```

```

int List::getLast()const {
    if (dummy->next!=dummy)
        return dummy->prev->data;
    else
        throw err;
}

```

```

void List::delFirst() {
    if (dummy->next!=dummy) {

```

```

node* t = dummy->next;
if (t->next != dummy) {
    dummy->next = t->next;
    dummy->next->prev = dummy;
}
else {
    dummy->next = dummy;
    dummy->prev = dummy;
}
delete t;
}
else
    throw err;
}

```

```

void List::delLast() {
    if (dummy->next!=dummy) {
        node* t = dummy->prev;
        if (t->prev!=dummy) {
            dummy->prev = t->prev;
            t->prev->next = dummy;
        }
        else{
            dummy->next = dummy;
            dummy->prev = dummy;
        }
        delete t;
    }
    else
        throw err;
}

```

```

ostream& operator<<(ostream& os, const List& l) {
    if (l.dummy->next!=l.dummy) {
        node* p = l.dummy->next;
        while (p != l.dummy) { os << *p << " "; p = p->next; }
        os << endl;
    }
    return os;
}

```

```

class listIterator {
public:
    class Error {
    public:
        void what() {
            cout << "Iterator error" << endl;
        }
    };

    listIterator(const List* s, node* e):collection(s), cur(e) {}

    const int operator *() {
        if (cur == collection->dummy) throw Error();
        return cur->data;
    }

    listIterator operator++(); //префиксный ++
    listIterator operator--(); //префиксный --
    int operator == (const listIterator& ri) const;
    int operator != (const listIterator& ri) const;

private:
    const List* collection;
    node* cur;
};

```

```
listIterator listIterator:: operator++() {
    if (cur!=collection->dummy) {
        cur = cur->next;
        return *this;
    }
    else throw Error();
    // или collection.isEmpty() или cur достиг end()
}
```

```
listIterator listIterator:: operator--() {
    if (cur->prev != collection->dummy) {
        cur = cur->prev;
        return *this;
    }
    else throw Error();
    // или collection.isEmpty() или cur достиг begin()
}
```

```
int listIterator:: operator==(const listIterator& ri) const {
    return ((collection == ri.collection) && (cur == ri.cur));
}
```

```
int listIterator:: operator!=(const listIterator& ri) const {
    return !(*this == ri);
}
```

```
listIterator List::begin() const {
    return listIterator(this, dummy->next);
}
```

```
listIterator List::end() const {
    listIterator iter(this, dummy);
```

```

        return iter;
    }

int main(){
    List l1;
    l1.inHead(100);
        for (int i = 0; i < 5; ++i) l1.inHead(i);
        for (int i = 5; i < 10; ++i) l1.inTail(i);
        cout << " list \n" << l1 << endl;
        List l2(l1);
        cout << " list \n" << l2 << endl;
        cout << sum(l2.begin(), l2.end()) << endl;
        listIterator mt = max(l2.begin(), l2.end());
        cout << *mt << endl;
        reverseprint(l2.begin(), mt);
        reverseprint(mt, l2.end());
        return 0;
    }

```

Нахождение суммы элементов списка, расположенных между двумя итераторами:

```

int sum(listIterator b, listIterator e) {
    int sum = 0;
    while (b != e){
        sum+= *b;
        ++b;
    }
    return sum;
}

```

Нахождение итератора, ссылающегося на максимальный элемент списка:

```

listIterator max(listIterator b, listIterator e) {
    listIterator maxx = b;
    while (b != e){
        if ( *b > *maxx )
            maxx = b;
        ++b;
    }
    return maxx;
}

```

Вывод в обратном порядке элементов списка, расположенных между двумя итераторами:

```

void reverseprint(listIterator b, listIterator be){
    while (be!=b) {
        cout << *(--be) << ' ';
    }
    cout << endl;
}

```

```

void reverseprint(listIterator b, listIterator be){
    --be;
    while (be!=b) {
        cout << *be << ' ';
        --be;
    }
    cout << endl;
}

```

Реализация шаблонного класса Stack:

//Stack.h(здесь и объявления функций и их реализация)

```

template <typename T>
class Stack {

```

```

private:
    T * start;
    //указатель на стек
    int head;
    //положение вершины стека
int size; //размер стека
public:
//конструктор с параметром по умолчанию для размера стека
    Stack(int = 10);
    ~Stack() { delete[] start; } //деструктор
    void push(const T&);
    //добавление элемента в стек
    T pop(); //извлечение элемента из стека
    T top(); //просмотр элемента на вершине стека
    bool isEmpty() const { //true, если стек пустой
        return head == -1;
    }
    bool isFull() const { //true, если стек полон
        return head == size - 1;
    }
}

class StackERR {...};

};

class stackEmpty { };
class stackFull {
...
};

//конструктор
template <typename T>

```

```

Stack<T>::Stack(int n) {
    //определение «разумного» размера стека
    size = n>0 && n<1000 ? n : 10;
    start = new T[size];
    head = -1;
}

// добавление объекта в стек
//в случае переполнения стека выбрасывается исключение
template <typename T>
void Stack<T>::push(const T& x) {
    if (isFull()) throw stackFull();
    start[++head] = x;
}

//извлечение элемента из стека
template <typename T>
T Stack<T>::pop() {
    if (isEmpty()) throw stackEmpty();
    T x = start[head--];
    return x;
}

// просмотр элемента на вершине стека
template <typename T>
T Stack<T>::top() {
    if (isEmpty()) throw stackEmpty();
    T x = start[head];
    return x;
}

```

//main.cpp

```

int main() {
Stack<int> intStack(5);
try{
    for (int i=0; i<5; i++) intStack.push(10-i);
    while (!intStack.isEmpty())
        cout<<intStack.pop()<<' ' ;
        cout<<endl;
        cout<<intStack.top(); // для проверки исключения
    }
    catch(stackFull e) {
        cout<<"ERROR="<<e.getCode()<<endl;
    }
    catch(Stack<int>::StackERR e) {
        cout<<"ERROR="<<e.getCode()<<endl;
    }
    return 0;
}

```

Реализация класса Matrix на основе vector<vector>:

```

template <typename T>
class myvector {
private:
    int size;
    T * vect;
public:
    matrix(int m=1, int n=1): rows(m), mdata(m){
        for (int i = 0; i < m; i++)
            mdata[i].resize(n);
    }

    myvector(int s = 1): size(s){
        vect = new T[s];
    }
}

```

```
}
```

```
myvector(myvector<T>&& v){
```

```
    size = v.size;
```

```
    vect= v.vect;
```

```
    v.vect = nullptr;
```

```
}
```

```
myvector(const myvector<T> & v): size(v.size){
```

```
    vect = new T[v.size];
```

```
    for (int i = 0; i<v.size; ++i)
```

```
        vect[i] = v.vect[i];
```

```
}
```

```
~myvector(){
```

```
    if (vect != nullptr)
```

```
        delete[] vect;
```

```
}
```

```
myvector<T>& operator=(myvector<T>&& v){
```

```
    if (vect != nullptr)
```

```
        delete[] vect;
```

```
    size = v.size;
```

```
    vect = v.vect;
```

```
    v.vect = nullptr;
```

```
    return *this;
```

```
}
```

```
myvector<T> operator+(const myvector<T>& v){
```

```
    if (size == v.size) {
```

```
        myvector<T> v1(size);
```

```
        for (int i = 0; i < size; ++i)
```

```

        v1[i] = vect[i] + v[i];

        return v1;
    }

    return *this; //лучше исключение использовать
}

```

```

void resize(int n){
    T* newVect = new T[n];
    int sz = (n < size) ? n: size;
    for (int i = 0; i<sz; ++i)
        newVect[i] = vect[i];
    delete[] vect;
    vect = newVect;
    size = n;
}

```

```

myvector<T>& operator [] (int i) {
    return mdata[i];
}

```

```

myvector<T> operator [] (int i) const {
    return mdata[i];
}

```

```

T& operator()(int i, int j){
    return mdata[i][j];
}

```

```

T operator() (int i, int j) const {
    return mdata[i][j];
}

```

```

};

```

Использование map для частотного словаря:

```
#include <iostream>

#include <fstream>

#include <vector>

#include <map>

#include <algorithm>

#include <iterator>

#include <string>

using namespace std;

int main() {

    ifstream ifs("source.txt");

    if (!ifs.is_open()) {

        cout << "Не могу открыть файл словаря source.txt" << endl; exit(1);

    }

    // Объявление типов и контейнеров
    typedef istream_iterator<string> string_input;
    vector<string> text;
    map<string, int> dictionary;

    // Чтение всех слов из файла в вектор
    copy(string_input(ifs), string_input(), back_inserter(text));

    // Подсчет частоты встречаемости каждого слова
    for (string s : text) {
        ++dictionary[s];
    }

    // Вывод частотного словаря на экран
    for (auto s : dictionary) {
        cout << s.first << " - " << s.second << endl;
    }

    return 0;

}
```

Использование set и set_intersection для поиска студентов-задолжников:

```
#include

class Student {
private:
    std::string name;
    std::string recordBook; // номер зачётной книжки (уникальное значение)
public:
    Student(std::string nm = "noname", std::string rB = "*****") : name(nm),
        recordBook(rB) {}

    // Оператор меньше для сравнения студентов по номеру зачётки
    friend bool operator<(const Student &p1, const Student &p2) {
        return p1.recordBook < p2.recordBook;
    }

    std::string Name() {
        return name;
    }
};

class StudentRating{
public:
    bool operator() (const Student & p1, const Student & p2){
        return p1.getrating() < p2.getrating();
    }
}

//Поскольку у разных студентов могут быть одинаковые баллы, для упорядочения по рейтин-
гу

//используем multiset с двумя параметрами: тип элементов множества и компаратор.

//main.cpp
#include <iostream>
#include <set>
#include <algorithm>
```

```

#include <string>
#include <iterator>

int main(){
    set<Student> MA, DE, PL;

    // здесь заполняются множества задолжников по: MA, DE, PL, например, с использо-
    ванием операции insert:

    MA.insert(Student("First Student", "MM11111"));

    set <int> s;

    pair<it, bool> a = s.insert(1);
    // можно auto a = s.insert(1);

    if (a.second)
        // проверяем, вставлен или уже был такой элемент
        cout <<"добавлено " << *(a.first);
    else
        cout <<"уже есть во множестве " << *(a.first);

    // Пересечение множеств MA и DE с записью результата через итератор
    it_ex set_intersection(MA.begin(), MA.end(), DE.begin(), DE.end(), it_ex);

    // Вывод студентов из expelledProj
    for (Student t : expelledProj) {
        cout << t.Name() << " ";
    }
    cout << endl;

    set<Student> expelled;

    // Пересечение expelledProj и PL с сохранением результатов в expelled
    set_intersection(
        expelledProj.begin(), expelledProj.end(),
        PL.begin(), PL.end(),
        inserter(expelled, expelled.begin())
    );

    // Вывод итогового списка отчисленных студентов
    for (Student t : expelled) {
        cout << t.Name() << " ";
    }
    cout << endl;

    return 0;
}

```

}